



中国矿业大学(北京)
CHINA UNIVERSITY OF MINING & TECHNOLOGY-BEIJING

硕士专业学位论文

三维地质建模中的布尔运算研究与应用

作者：田华威

学院：地球科学与测绘工程学院

学号：ZQT2100203097

专业学位类别：资源与环境

专业学位领域：地质工程

校内导师：武强

校外导师：范书凯

2024年6月

中图分类号: TD263

单位代码: 11413

UDC 分类号: 622

密 级: 公开

硕 士 专 业 学 位 论 文

中文题目: 三维地质建模中的布尔运算研究与应用

英文题目: Research and application of Boolean operation in 3D
geological modeling

作 者: 田华威 学 号: ZQT2100203097

专业学位类别: 资源与环境 专业学位领域: 地质工程

研 究 方 向: 煤矿信息化 学 习 方 式: 全日制

校 内 导 师: 武强 职 称: 教授

校 外 导 师: 范书凯 职 称: 教授级高工

论文提交日期: 2024 年 3 月 20 日 论文答辩日期: 2024 年 5 月 30 日

学位授予日期: 2024 年 6 月 18 日

中国矿业大学（北京）

独创性声明

本人声明所呈交的学位论文是我个人在导师指导下进行的研究工作及取得的研究成果。尽我所知，除了文中特别加以标注和致谢的地方外，论文中不包含其他人已经发表或撰写过的研究成果，也不包含为获得中国矿业大学（北京）或其他教学机构的学位或证书而使用过的材料。与我一同工作的同志对本研究所做的任何贡献均已在论文中作了明确的说明并表示谢意。

作者签名：田华威 日期：2024年6月15日

关于论文使用授权的说明

本人完全了解中国矿业大学（北京）有关保留、使用学位论文的规定，即：学校有权保留送交论文的复印件，允许论文被查阅或借阅；学校可以公布论文的全部或部分内容，可以采用影印、缩印或其他复制手段保存论文。

（保密的论文在解密后应遵守此规定）

作者签名：田华威 导师签名： 日期：2024年6月15日

摘 要

随着三维地质建模技术的快速发展，三维地质建模在数字矿山领域中的应用也更加广泛。模型之间的布尔运算是三维地质建模中关键步骤之一。针对规模庞大、构造复杂的三维地质模型，研究高效健壮的布尔运算算法对三维地质建模技术有着重要意义。

本文根据 OFF 模型的文件特点建立相应的数据结构及拓扑关系。通过构建 AABB 包围盒进行预检测，确定布尔运算的碰撞检测区域；构建层次包围盒树进行碰撞检测，以便筛除大量不相交的三角形；对剩余三角形进行精确相交测试，基于模型拓扑结构，将求得的离散交线段有序地连接成多个交线环。记录交线环经过的三角形并提取其中的内外多边形区域，运用 Ear-Clipping 算法对多边形区域作三角剖分，实现模型的重构。对于交线环未经过的三角形，基于图的广度优先遍历（BFS）算法快速完成此类三角形的内外分类，提高了布尔运算效率。根据布尔运算公式对内外三角形进行合并重组，生成最终结果。

以邢台某矿的复杂地层、巷道等三维地质模型为例进行布尔求交运算，根据地层岩性的不同为三维巷道进行岩性划分并设置相应的颜色，实现三维巷道表面岩性划分及可视化。以布尔求差运算为基础对矿区地层进行开挖，获取开挖后暴露的地层结构、空腔形状等信息。

关键词：三维地质建模；布尔运算；碰撞检测；交线环；模型重构

Abstract

With the rapid development of 3D geological modeling technology, the application of 3D geological modeling in digital mining field is more and more extensive. The Boolean operation between models is one of the key steps in 3D geological modeling. Aiming at the large scale and complex structure of 3D geological model, it is important to study the efficient and robust Boolean operation algorithm for 3D geological modeling technology.

In this thesis, the corresponding data structure and topological relationship are established according to the file characteristics of the OFF model. The collision detection region of Boolean operation is determined by constructing AABB bounding box for pre-detection. A hierarchical bounding box tree is constructed for collision detection to screen out a large number of disjoint triangles. Based on the topology structure of the model, the discrete intersection segments obtained are connected into multiple intersection loops. The triangles passed by the intersecting ring were recorded and the inner and outer polygon regions were extracted. The Ear-Clipping algorithm was used to triangulate the polygon regions to realize the reconstruction of the model. The breadth-first traversal (BFS) algorithm based on graph can quickly classify the triangles that are not traversed by the intersecting loop, and improve the efficiency of Boolean operation. At last, the inner and outer triangles are combined and reorganized according to the Boolean operation formula, and the operation result is generated.

Taking the three-dimensional geological model of complex strata and roadway in Xingtai mine as an example, Boolean calculation is carried out to divide the lithology of the three-dimensional roadway and set the corresponding colors according to the different lithology of the strata, so as to realize the lithology division and visualization of the surface of the three-dimensional roadway. The excavation of strata in mining area is carried out on the basis of Boolean difference calculation, and the exposed strata structure and cavity shape are obtained after excavation.

Key Words: three-dimensional geological modeling; boolean operation; collision detection; Intersecting loop; model reconstruction

目 录

1 引言	1
1.1 选题背景、目的及意义	1
1.2 国内外研究现状	2
1.3 研究内容	4
1.4 研究方法与技术路线	5
1.4.1 研究方法	5
1.4.2 技术路线	5
1.5 主要创新点	7
1.6 本章小结	7
2 布尔运算中模型交线求取方法	9
2.1 模型结构	9
2.1.1 模型文件格式	9
2.1.2 模型数据结构	12
2.1.3 拓扑关系建立	14
2.2 碰撞检测	16
2.2.1 数据结构	16
2.2.2 AABB 包围盒预检测	17
2.2.3 构建包围盒树	19
2.2.4 包围盒树遍历	20
2.3 相交测试	22
2.3.1 三角形位置关系	22
2.3.2 空间相交测试	23
2.4 交线环提取	25
2.5 本章小结	30
3 布尔运算中模型重构及分类	31
3.1 模型重构	31

3.1.1 多边形提取算法	31
3.1.2 多边形区域划分	33
3.1.3 多边形三角剖分	35
3.2 三角形内外分类	42
3.3 模型合并	45
3.4 本章小结	47
4 应用实例	49
4.1 研究区域背景	49
4.1.1 位置与交通	49
4.1.2 地质概况	49
4.2 应用实例	51
4.3 本章小结	69
5 结论与展望	71
5.1 结论	71
5.2 展望	71
参考文献	73
致 谢	79
作者简介	81

1 引言

1.1 选题背景、目的及意义

矿产资源对人类的生存、建设和发展至关重要^[1-2]。我国作为煤矿大国，煤矿资源对我国的社会经济发展起到了重要的作用^[3-5]。近几年来，由于大数据、数值分析、可视化、虚拟现实等技术的不断提升，数字矿山已经成为学术界关注的热点。随着信息化技术的持续进步，地理空间信息变得越发重要，其在建立三维场景和数字孪生技术等领域的运用也日益增加^[6-7]。数字矿山是一种综合了采矿工程、信息技术、系统方法等多个学科的信息系统。它是一个集理论、技术以及实践于一体的学科。数字矿山的研究、建设与应用，极大地提升了我国矿产资源的综合利用程度，促进了我国的经济与社会发展。数字矿山系统拥有全面、实时、精确监控我国矿产储量及其变动的能力，这对于推动矿产资源的科学、合理发掘与利用，以及达成有效保障具有极其重要的意义。此系统有助于促进矿产资源的可持续供应，确保矿产资源的长期利用与健康发展^[8-9]。在当前背景下，面对资源开发协调、地质灾害预防以及生态环境维护等多重复杂挑战，数字矿山系统的应用显得尤为重要^[10]。在矿业领域，可视化以及数字化技术的应用为矿山建设和资源开发带来了强大的技术支撑。通过这些技术，可以构建基于地质数据的三维矿山模型，进而为矿山的生产管理和环境保护提供科学的参考和决策依据^[11-13]。巷道、地层、断层以及陷落柱等三维地质模型的可视化作为数字矿山研究的重要组成部分，为矿产资源的勘探采掘、井下灾害模拟及逃生模拟提供了坚实的基础。巷道特指在岩层中为矿石开采所开挖的空腔，涵盖竖井、底部停车场、运输以及回风巷道等多种类型^[14]。

地理信息系统（GIS）作为一种处理空间信息并辅助决策的空间信息系统，已广泛应用于多个领域^[15-16]。依据其处理维度的不同，GIS可分为二维和三维两大类，由于计算机技术的快速发展，三维GIS受到更多关注，其在矿山领域的作用也愈加重要^[17-18]。因此，在GIS的基础上对三维地质模型的构建与相关应用进行研究变的十分重要。

三维巷道的可视化与应用在数字化矿山中是至关重要的研究领域^[19-20]。随着计算机技术在煤矿领域的快速发展，建立符合生产要求的三维巷道模型并在此基础上开展相关应用的研究尤为重要^[21-22]。地层、断层、陷落柱以及透镜体的建模以及可视化在数字矿山中也是占据重要地位。对这些三维地质模型进行布尔运算得到它们的交集、并集、差集，为之后模型的相关应用奠定基础。

布尔运算的应用根据使用场景的不同，主要可划分为两大类别。首类，即应

用于图像渲染领域的布尔运算，其在图形处理与视觉效果呈现方面发挥着重要作用；次类，则是应用于实体模型之间的布尔运算，这一类别在三维建模、物体交互及空间分析等领域具有广泛的应用价值。第二类布尔运算在三维地质建模领域中得到了广泛的应用。如王红娟、卢新明^[23]等人基于布尔运算可以改善巷道三维建模算法中特殊交叉点不能相互贯通的问题。中南大学的邹艳红^[24]运用布尔运算为研究断裂与矿化分布之间的定量关系提供了可靠依据。华中科技大学的吴佳明^[25]将其运用到地质体与结构体模型的深度融合中。

然而，在三维地质建模领域，断层、陷落柱等自然现象增加了模型的复杂性，规模庞大、构造复杂的三维地质模型之间进行布尔运算容易产生错误。现有布尔运算算法主要分为三步：交线环提取、模型重构、模型分类及合并。交线环提取需要把离散的交线对按照顺序依次连接成交线环，现有方法是基于距离完成最近点的合并，最终完成交线环提取，当距离的阈值设置不合理时，容易造成本该合并的两点无法合并，导致交线环提取失败；复杂三维地质模型进行重构时容易产生缝隙、重合等现象。针对现有技术的缺陷，本文设计了一个三维地质模型之间布尔运算的流程及其关键技术的实现方法。以区域内复杂地层、巷道等三维地质模型为例进行布尔求交运算，根据地层岩性的不同为三维巷道进行岩性划分并设置相应的颜色，实现三维巷道表面岩性划分及可视化，为辅助巷道支护设计、风险评估、施工指导奠定基础。以布尔求差运算为基础对矿区地层进行开挖，获取开挖后暴露的地层结构、空腔形状等信息。

1.2 国内外研究现状

尽管国内外很多学者已经对三维实体模型之间的布尔运算进行了深入的研究，但同一种布尔运算算法对不同类型的三维模型并不适用，如有些三维模型使用三角形为基元进行表达，而有的三维模型用四边形进行表达。至今尚未有一种通用的算法能够完全满足所有应用场景的需求。鉴于此，本文只对以三角形为基础的三维模型进行布尔运算探究。

在 3D 领域中，布尔运算可以根据其结果的精确程度被划分为两类：一类是近似布尔运算，另一类是精确布尔运算。对于近似布尔运算主要有体素表达法，该方法是最早被提出用以提升布尔运算效率^[26]。其核心在于利用体素模型在布尔运算上的快速性能。体素化的实现方法众多，包括自适应距离场方法^[27]和基于水平集的方法^[28]等。这种方法具有显著的优点，如运算速度快、稳定性高等。然而，其局限性亦不容忽视，即体素化模型的精度受限于体素尺寸，对于高精度要求，可能导致内存使用增加及体素化时间延长。针对这些问题，众多研究者进行了深入研究并提出了改进措施。例如，L.P.Kobbelt 等人^[29]采用 Marching Cube 算法从

体素模型中提取三角形特征，有效减少了布尔运算后模型特征的损失；T.Ju 等人^[30]通过对模型法向量的收集与研究，使模型表面重构效果更好，进一步提升了模型的精度和质量。青岛理工大学的李妞^[31]采用基于八叉树球形体素模型的虚拟毛胚体素化方法，提高了体素单元之间的碰撞效率。此外，还有研究者使用两种轮廓使模型的细节以及拓扑得以保持^[32-33]。尽管如此，上述算法都面临将三角网格转换为体素，然后再转换回三角网格的过程中可能丢失模型几何细节和精度的问题。

精确布尔运算是以三角形为基元的算法^[34]。如何快速有效地对模型间发生相交的三角形进行提取，是精确布尔运算效率提升的关键。这个提取过程通常被称为碰撞检测，碰撞检测根据应用场景不同所需技术也不相同，常见技术有层次包围盒法以及空间剖分法。

层次包围盒简称 BVH^[35-36]，是一种二叉树结构，用来存储模型包围盒信息。在 BVH 树中，根节点表示整个模型的包围盒，其子节点则分别代表更小的子包围盒。通过不断地细分，最终的每个叶子节点中储存三角形的子集。目前，有各种不同的包围盒，大致分为：AABB 包围盒，OBB 包围盒，包围球^[37]，K-DOPS 包围盒以及凸包包围盒。这些包围盒都定义了怎样构建去包围模型，它们之间的区别在于，越复杂的包围盒一般能够更紧密的包围模型，相交测试的次数变少，但是构建的时间花费更长。上海大学的王磊^[38]采用一种改进的混合型层次包围盒算法，通过将待检测物体表示为二叉树结构，并将其分为上下两个层级，其中上层使用 AABB 并添加一个球体到根节点，下层采用 OBB。这种算法显著提升了物体碰撞检测的准确性和效率，同时减少了误报和穿透问题。金汗均等^[39]从存储空间的角度来改进基于 AABB 树的碰撞检测算法。中北大学的杨凯祥^[40]运用 AABB 包围盒做碰撞测试从而筛除大量不相交三角形。中南大学的杨兰^[41]运用改进后的 OBB 包围盒做碰撞检测，有效提升了算法效率，并将改进后的算法应用于布尔运算。

空间剖分法涵盖了多种基于树结构的方法，如 BSP(Binary Space Partition)树^[42-43]、四叉树^[44-45]、八叉树^[46-47]，以及以数组为基础的统一网格划分^[48]方法。Naylor 和 Thibault^[49-50]最先把 BSP 树技术运用在三维模型布尔运算的碰撞检测中。BSP 树作为一种空间二叉树结构，其核心思想在于通过选取一个平面，将空间划分为两个互不重叠的部分，随后将这两部分分别作为树的子节点进行递归处理。当三角形的数量降至设定阈值以下或树的深度达到预设限制时，划分过程终止。通过递归执行这种划分，可以构建出一个完整的 BSP 树。在利用 BSP 树进行布尔运算时，首先需要将两个模型分别转换成 BSP 树的结构。接着根据选定的布尔运算符（如并集、交集或差集）对这两棵 BSP 树执行合并操作，最终合并得到的结

果就是布尔运算的输出结果^[51]。林宇鹏^[52]对基于 BSP 树的碰撞检测进行优化，从而提升了布尔运算的整体效率。八叉树是将空间进行八等分，首先确定三维模型之间共有的最小立方体，此为八叉树的根节点空间。接下来，将这个根节点空间等分为八个立方体，对存储在根节点中的两个模型的所有三角形进行遍历，并根据与这些三角形相交的子立方体节点对它们的索引进行分类和存储。随后，检查所有子节点中包含的三角形索引信息。如果一个节点中存在来自两个模型的三角形索引并且树的深度在 8 到 10 层之间以及节点的三角形数量大约为 50 个^[53]，则对该叶子节点结束细分，否则继续递归。Campen M^[54]等创新性地融合了八叉树与 BSP 树技术，使三维模型之间的布尔运算效率得到提升。熊心一^[55]采用将静态八叉树与动态八叉树相结合的方式，旨在降低时间以及空间复杂度，通过实验结果发现此算法的效率得到提升。F.R.Feito^[56]利用八叉树自身数据结构特性进行改进，使算法效率得到显著提升。上海交通大学的姜旭东^[57]对延迟切割技术与三角形分类技术进行结合，使算法效率得到提升并取得了显著成果。Franklin^[58-59]提出了使用统一网格划分算法进行相交测试，通过研究表明此算法有良好表现。Magalhes^[60-62]进一步扩展了这一算法，将其应用于布尔运算中，实现了一种既高效又稳健的布尔运算算法，为相关领域的研究与实践做出了重要贡献。

在三维模型的布尔运算过程中，需要进行大量的空间三角形相交测试。Moller^[63]提出了空间三角形相交算法，Held^[64]以及 Guigue^[65]在此基础上进行改进。为了避免离散交线段配对成交线环的过程中来自浮点运算的误差，Xiaotong^[66]利用八叉树的数据结构与模型中三角形的邻接边信息，提出了一种快速且稳健的交线环提取算法，并使整体布尔运算效率得到提升。在三维模型的布尔运算中，模型相交产生的交线环会把三角形分割为多个多边形区域，对这些三角形需要进行重构，重构的关键在于对多边形区域进行三角剖分。常用的方法是 Delaunay 三角剖分以及 Ear-Clipping 三角剖分，Livesu 等^[67]对 Ear-Clipping 算法进行研究，发现该算法在效率上有较好的表现。

1.3 研究内容

本文对当前国内外学者关于三维模型布尔运算的研究现状进行了系统性的梳理与分析，并对其中存在的问题以及解决方法进行详细阐述。针对实际三维地质模型，对布尔运算的交、并、差进行研究，并对布尔运算结果进行相关研究与应用。

(1) 布尔运算中模型交线求取方法研究。通过分析模型文件格式，建立相应的数据结构及拓扑关系。研究基于层次包围盒的碰撞检测算法以实现相交三角形的快速筛选，并对相交的三角形对进行交线段求取。研究基于模型本身拓扑结

构的方法完成交线环提取。

(2) 布尔运算中模型重构及分类研究。针对交线环经过的三角形进行重构以及内外分类；对于交线环未经过的三角形，基于 BFS 算法快速完成此类三角形的内外分类，并根据布尔运算公式对所需三角形进行合并重组。重点研究如何运用 Ear-Clipping 算法将提取的多边形剖分为三角形，以及如何对三角形进行内外分类。

(3) 应用实例研究。通过研究区域的相关地质背景，在复杂地层、巷道等三维地质模型的基础上进行布尔运算。根据地层岩性的不同为三维巷道进行岩性划分并设置相应的颜色，以实现三维巷道表面岩性划分及可视化。以布尔求差运算为基础对矿区地层进行开挖，获取开挖后暴露的地层结构、空腔形状等信息。

1.4 研究方法与技术路线

1.4.1 研究方法

根据本文研究内容，将研究方法过程分为以下几个方面：

(1) 参考文献与资料收集

查阅国内外参考文献以及相关资料，收集矿区相关三维地质模型的数据。总结前人关于三维模型间布尔运算的研究进展。

(2) 模型交线求取方法

对三维地质模型进行导入工作，研究模型数据结构以及拓扑关系的构建方法。对模型的层次包围盒算法进行研究，探讨出得到发生碰撞的三角形对的方法。基于空间三角形相交原理，对这些三角形对进行精确求交产生一些离散的交线段；基于模型本身的拓扑结构，研究把这些离散交线段连接成交线环的方法。

(3) 模型重构及分类

针对被交线环切割的三角形情况，对这些三角形的重构算法进行研究。针对未被交线环切割的三角形，研究基于图的广度优先遍历算法（BFS）区分内外三角形的方法。把重构完成的三角形以及基于 BFS 算法区分的内外三角形依据布尔运算公式进行重组，得出布尔运算结果。

(4) 可视化分析

在 MeshLab 中进行实例可视化分析，通过典型三维地质模型间的布尔运算结果验证方法的可靠性。

1.4.2 技术路线

根据本文的研究内容，对三维地质建模中布尔运算的技术路线进行研究，技术路线图如下所示，其中虚线框中代表的是布尔运算的一般步骤。

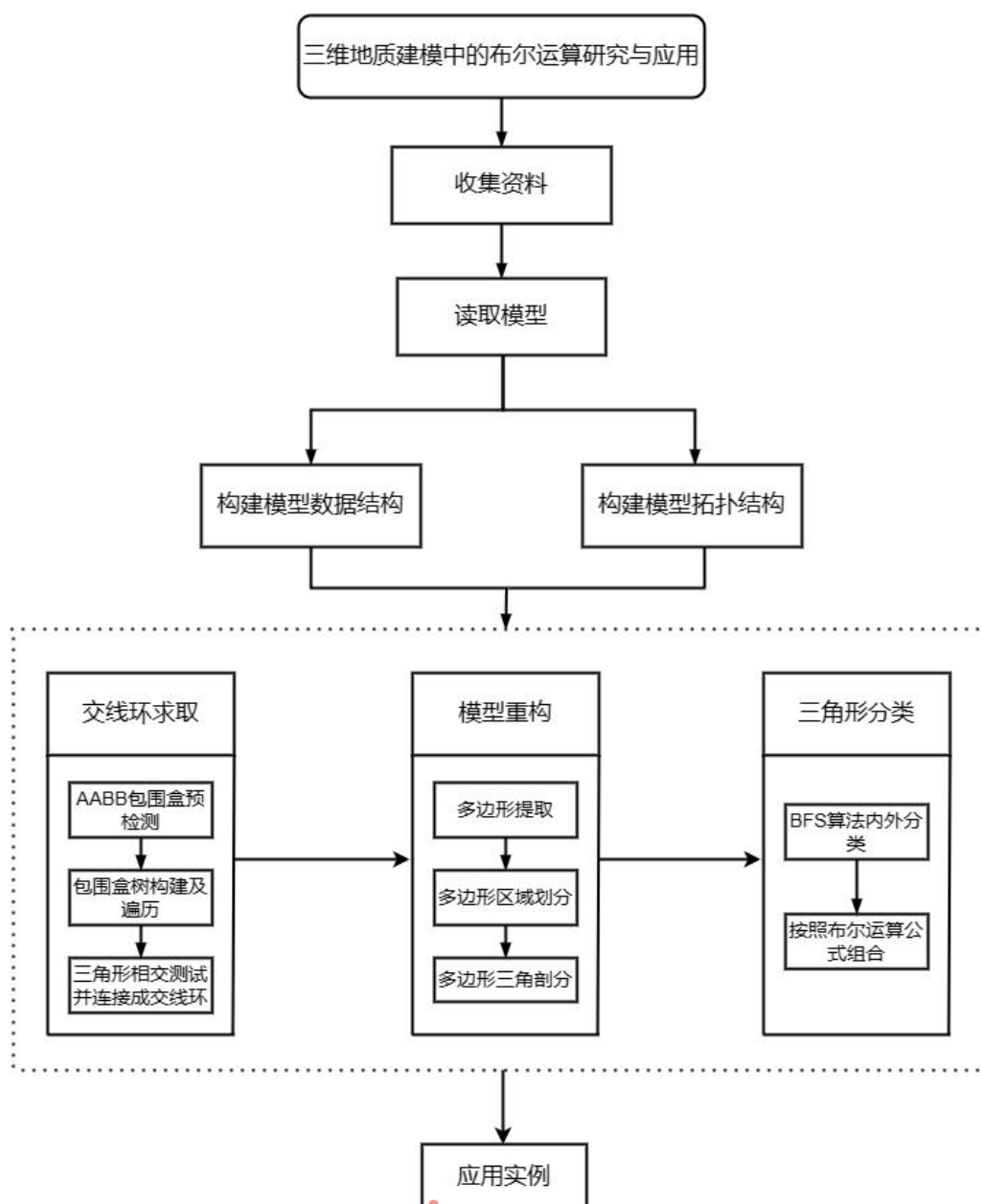


图1.1 技术路线图

Fig. 1.1 The technology roadmap

在实际场景中，比如巷道模型与地层模型相交的情况下，这时候有很多三角形处于相交碰撞之中，这时候无法对其进行三角形之间的相互分类，只能先沿着交线环对相交的三角形进行重构，重构之后才能进行之后的布尔运算。所以，布尔运算中首先要解决的问题是如何找到巷道以及地层模型中相交的三角形，如果模型中的三角形数量比较少时，可以采用暴力法对巷道中每一个三角形进行遍历，对地层三角形进行相交性判断。但是，真实的巷道以及地层模型中的三角形个数动辄达到十万或者百万以上，运用暴力法的效率十分低下。所以碰撞检测算法对三维地质模型之间的布尔运算起到重要作用。

布尔运算可以分为以下几个步骤：

(1) 运用碰撞检测筛除模型中大量不相交的三角形，保留可能相交的三角形进行三角形相交测试。

(2) 把经过三角形相交测试得到的一系列交线段按照规律配对成交线环，并把交线环经过的三角形记录下来。

(3) 对记录下的三角形进行重构并区分内外，根据布尔运算公式对三角形进行分类以及重组，得到最后结果。

1.5 主要创新点

(1) 基于模型拓扑结构，将离散交线段连接成交线环；针对交线环经过的三角形进行重构以及内外分类，模型重构性能良好，算法健壮性强。

(2) 针对包含断层、陷落柱等复杂地层与巷道相交情况，根据地层岩性的不同为三维巷道进行岩性划分并设置相应颜色，实现三维巷道表面岩性划分及可视化，为辅助巷道支护设计、风险评估、施工指导奠定基础。

1.6 本章小结

本章首先论述了选题背景、目的及意义，其次梳理并总结出当前国内外学者在三维模型布尔运算方面的研究现状，最后说明了本文的研究内容、研究方法与技术路线以及主要创新点。

2 布尔运算中模型交线求取方法

2.1 模型结构

2.1.1 模型文件格式

OFF(Object File Format)是一种使用一系列空间三角面片构成的网格来拟合三维实体的文件格式。由于其结构简洁且易于理解,处理起来相对便捷,因此逐渐成为了三维建模领域中广泛采用的数据交换格式。OFF 文件格式有以下优点: 1、开放的文件格式: OFF 文件是一种开放的文件格式,可以免费使用和编辑,可以在不同的软件 and 平台之间进行交换和共享。2、简单的文件格式: OFF 文件格式简单易懂,以 ASCII 文本行形式储存数据,便于查看和编辑。3、适用于三维模型: OFF 文件格式适用于存储三维模型,可以表示任意的多边形和曲面。4、可扩展性: OFF 文件格式具有可扩展性,可以根据需要添加新的数据类型和属性信息,便于扩展和更新。总之, OFF 文件格式是一种开放、简单、适用于三维模型存储和交换的文件格式,具有可扩展性,方便用户进行查看和编辑,同时也便于在不同软件 and 平台之间进行交换和共享。OFF 文件格式也可以通过一些软件进行转换,例如 Blender、NetBeans 等。这些软件可以将 OFF 文件格式转换为其他格式,例如 OBJ、STL、DXF 等。以下是一些常用的 OFF 文件格式转换方法: 1、OFF 到 OBJ: OFF 文件格式可以使用 Blender 软件转换为 OBJ 格式。Blender 是一款免费的三维建模软件,可以将 OFF 文件导入后转换为 OBJ 格式,也可以通过命令行工具 off2obj 进行转换。2、OFF 到 STL: OFF 文件格式可以使用 NetBeans 软件转换为 STL 格式。NetBeans 是一款免费的集成开发环境,支持 Java、C++ 等多种编程语言,也支持 OFF 文件格式的读取和转换。3、OFF 到 DXF: OFF 文件格式可以使用 AutoCAD 软件转换为 DXF 格式。AutoCAD 是一款专业的计算机辅助设计软件,支持多种文件格式,包括 DXF、DWG 等。4、使用在线工具: 3D 模型在线转换,网站提供多种模型格式文件转换,支持将 GLB、PLY、OBJ、STL、PCD 等多种格式转换为 OFF,同时支持将 OFF 转换成 GLTF、OBJ、GLB、PLY、STL 等多种格式。

OFF 文件格式通过定义物体表面的多边形来展现一个模型的几何形态。这些多边形可以由不同数量的顶点构成,并以 ASCII 文本行形式储存数据。OFF 文件的结构相对简单,由头部信息、顶点信息和面信息三部分组成。其中,头部信息包含了 OFF 文件的版本号和模型中顶点、面和边的数量;顶点信息则是描述每个顶点在三维空间中的坐标,用 $x\ y\ z$ 表示,以空格或制表符分隔,每行一个顶点;

面信息是描述每个面由哪些顶点组成的索引，用 No. id1 id2 id3 表示，以空格或制表符分隔，其中每行的第一个数字 No. 表示该面有多少个顶点，id1 id2 id3 表示此面对应的顶点索引。

在具体应用中，不同的三维模型通常需要不同的三角划分来进行表示，对于同一个模型，其划分的精度也会影响到三维模型的准确性以及逼真度，即三角面片越多，OFF 文件所描述的三维模型越细致、精确，误差也越小。因此，在三维地质建模领域，为了获得精确的三维模型，必须尽可能的增加 OFF 文件中三角面片的数量。在增加 OFF 文件中三角面片数量的同时，还需要遵循一些规则：

（1）共顶点规则

模型中每个三角面片必须和其他相邻面片共用两个顶点，即每个三角面片的顶点不能在与它相邻面片的某条边上。如图 2.1 所示。

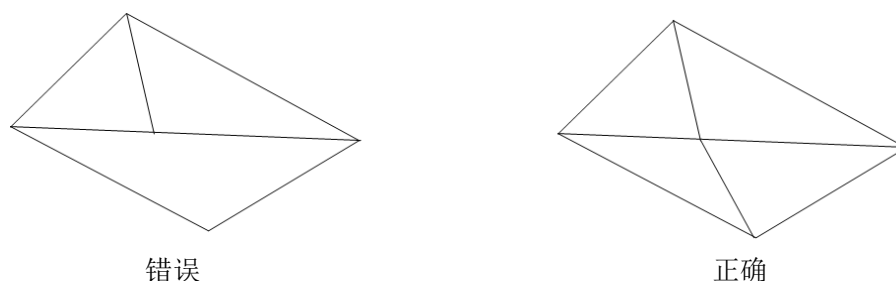


图 2.1 共顶点规则

Fig. 2.1 Common vertex rule

（2）取向规则

模型的内外是靠三角面片中顶点的排序来区分的，三角面片的顶点如果是逆时针排序则为模型的内表面，顶点如果是顺时针排序该三角面片则为模型外表面。本文规定 OFF 文件中三角面片的顶点顺序均为逆时针排列。按照右手定则，手指从顶点 point1 开始，经过 point2 并指向顶点 point3 时，大拇指的方向即为三角面片的法向量方向，即指向三维模型表面的外部。如图 2.2 所示。

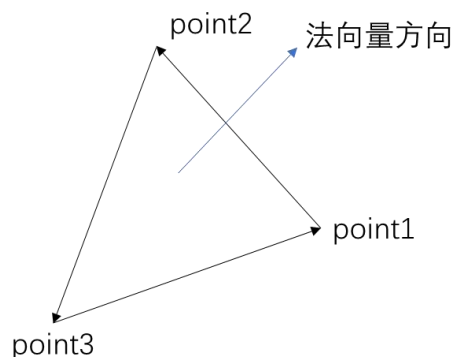


图 2.2 右手取向规则

Fig. 2.2 Right-handed orientation rule

(3) 合理性规则

三维模型中不能有重复的点、线、面，三角面片必须全覆盖模型表面，不可以有任何缝隙、孔洞等，不能出现模型自相交等情况。

ASCII 文本行形式的 OFF 文件中，每个三角面片的几何信息都由相应的关键字给出，首先以单词 OFF 作为文件的开头，第二行则分别是点、面、边的个数，可以安全地忽略边的数量。从第三行开始每行列出一个顶点的 $x y z$ 坐标，在顶点列表之后是面列表，每行表示一个面。对于面列表，每行的第一个数字表示顶点数，然后是指定顶点列表中的顶点索引号。具体如下：

```
OFF //文件名
8 12 0 //点、面、边的个数
-5.831676 -0.459705 -0.296586 //点的具体坐标
-5.831676 0.427135 -0.296586
-3.080345 -0.281107 -8.913973
-3.080345 0.605733 -8.913973
13.81167 -0.459705 -0.296586
13.81167 0.427135 -0.296586
16.563 -0.281107 -8.913973
16.563 0.605733 -8.913973
3 3 1 2 //一个三角形中点的个数以及顶点的索引
3 6 7 3
3 4 5 6
3 4 1 5
3 4 6 2
3 5 3 7
3 5 7 6
3 6 3 2
3 1 0 2
3 4 0 1
3 5 1 3
3 4 2 0
```

上述 OFF 文件表示的是一个简单的立方体，如图 2.3 所示：

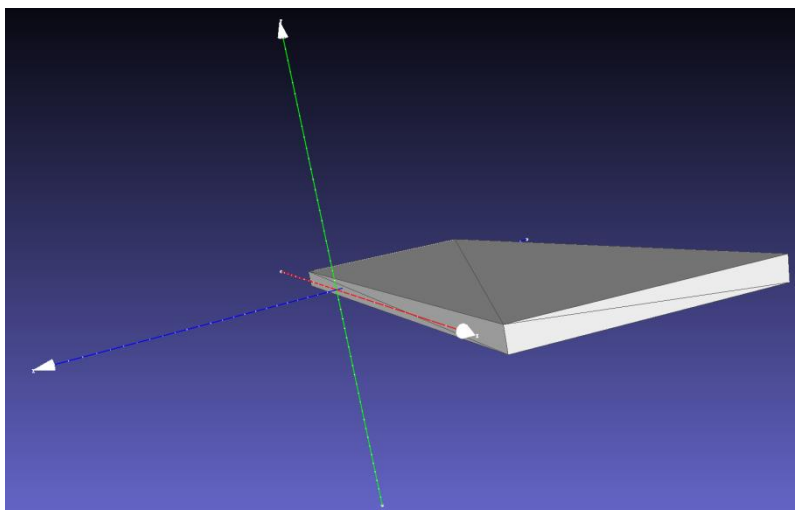


图 2.3 简单三维立方体图
Fig. 2.3 Simple 3D cube diagram

2.1.2 模型数据结构

三维地质模型包括巷道模型、地层模型、断层模型以及陷落柱等，其中地层、断层以及陷落柱模型放在一起表达，统称为地层模型。

为了更好的表达之后三维地质模型之间的布尔运算算法，本文先介绍几个基本的数据类型。由于三维地质模型都是由三角面片所组成，一个三角面片又是由三个顶点以及三条边所构成。本文把进行布尔运算的三维地质模型分别作为一个 Mesh，每个 Mesh 类里面包含最基础的成员变量 triangle、edge 以及 point。详细如下：

(1) Point 类

Point 类是布尔运算中最基础的类，是所有类的基础。该类描述的是三角形的顶点具有的基本成员变量以及基本成员方法，它的基本成员变量包括空间三角形顶点的三维坐标 x、y、z，其数据类型是 double 双精度浮点型，以及顶点的索引 id 号，数据类型是 int 整数类型。Point 类的具体形式如下：

```
Public class Point{  
    int id;  
    double x;  
    double y;  
    double z;  
}
```

(2) Edge 类

Edge 类表示的是三角形的边类，该类描述的是三角形的边具有的基本成员变

量以及基本成员方法，它的基本成员变量包括边的首尾两个顶点的编号(int startPoint、int endPoint)、边的索引号(int id)、共享此边的三角形集合(List<Integer> triangles)以及边上所含有的交点集合(List<Point> intersectionP)。其具体形式如下：

```
Public class Edge{
    int id;
    int startPoint;
    int endPoint;
    List<Integer> triangles;
    List<Point> intersectionP;
}
```

(3) Triangle 类

Triangle 类表示的是三角面片，其包括此三角形的索引(int id)、三角形的三个顶点索引(int indexPa,int indexPb,int indexPc)、三条边的索引(int indexE1,int indexE2,int indexE3)以及此三角面片相邻三个三角面片的索引(List<Integer> triangles)。其具体形式如下：

```
Public class Triangle{
    int id;
    int indexPa;
    int indexPb;
    int indexPc;
    int indexE1;
    int indexE2;
    int indexE3;
    List<Integer> triangles;
}
```

(4) Mesh 类

Mesh 类表示的是三维地质模型的整体，其基本成员变量包含有此模型的 id、name，以及此模型中所有的三角面片集合 List<Triangle> triangles、边的集合 List<Line> edges、点的集合 List<Point> points。具体情况如下：

```
Public class Mesh{
    int id;
    string name;
    List<Triangle> triangles;
```

```
List<Line> edges;  
List<Point> points;  
}
```

当基础 Mesh 类建立好后，可以通过 mesh. triangles、mesh. edges、mesh. points 去调用获取到模型中的三角形、边以及顶点的集合。

2.1.3 拓扑关系建立

由于 OFF 文件可以用 txt 文本形式打开，所以读取 txt 文件的同时可以把模型的空间拓扑关系建立起来。为了构造拓扑关系，需要根据前面介绍的基本数据结构进行拓扑预处理。因为原始数据所记录的三角形都是无序杂乱、无指向性的，所以要重新对顶点数据以及三角面片数据进行重新排序。定义集合 List<Point> points、List<Line> edges 以及 List<Triangle> triangles 用来储存模型的顶点、边以及三角面片，最终建立一个不包含重复顶点的点集合（记录顶点的坐标值以及索引号），同时建立一个边集合（用来储存边的索引号、构成此边的顶点索引以及包含此边的两个三角面片的索引）以及一个面集合（包含此三角面片的索引号、三个顶点的索引号、三条边的索引号以及相邻三角面片的索引号）。根据 OFF 文件的格式采用按行读取信息，详细步骤如下：

（1）根据 OFF 文件的格式，从第三行开始先读取顶点的 x, y, z 并对此顶点进行对象实例化以及赋 id 属性，最后把此点加入到集合 List<Point> points 中。

（2）依次读取顶点并进行(1)中操作，直到所有顶点都加入到集合 List<Point> points 中。

（3）接着读取三角面片信息，当读取到行的第一个字符为“3”时，表示此行记录的是三角面片的信息，取当前三角面片 T_i (i 为此三角形的索引号)， T_i 的三个顶点索引号为 V_1 、 V_2 、 V_3 ，首先按照逆时针读取三个顶点的索引号并对此三角面片进行对象实例化以及赋 id 属性，然后根据三个顶点 V_1 和 V_2 、 V_2 和 V_3 、 V_3 和 V_1 构建三角形的三条边 Edge1、Edge2、Edge3（索引），随后把此三角形的索引号 i 加入到每条边的邻接三角形集合中，用来构建每条边的邻接三角形，当三条边都构建完，根据边中邻接三角面片的关系，得到此三角形的邻接面片域。最后把此三角面片加入到 List<Triangle> triangles 中，三条边加入到 List<Line> edges 中。

（4）按照(3)的操作，直到所有三角面片读取完为止。

参照图 2.4 展示的部分 OFF 模型三角面片，构建相应的拓扑结构后，所得到的结果分别如表 2.1、表 2.2 和表 2.3 所示。在这些表中，使用 -1 来指代不存在的邻接三角面片。但实际上，由于模型表面是封闭的，每个三角面片都有其对应的邻接三角面片。

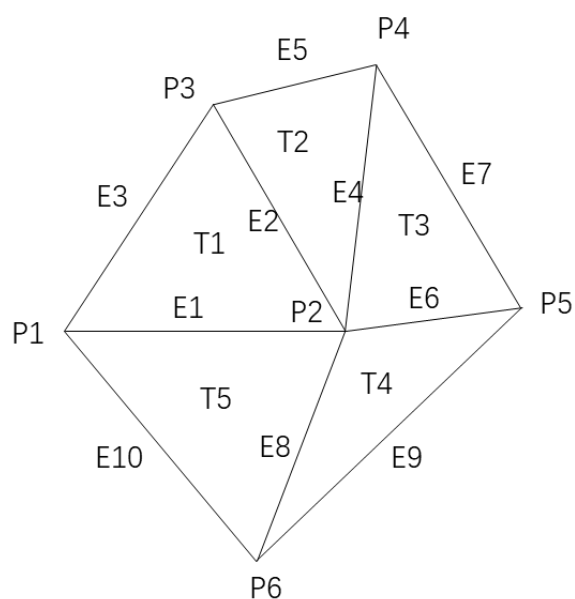


图 2.4 OFF 模型部分三角形图

Fig. 2.4 OFF model part triangle diagram

表 2.1 顶点拓扑

Table 2.1 Vertex topology

索引(id)	0	1	2	3	4	5
顶点	P1	P2	P3	P4	P5	P6

表 2.2 边的拓扑关系

Table 2.2 Topological relation of edges

索引(id)	边号	顶点索引	邻接三角面片(id)
0	E1	0 1	0 4
1	E2	1 2	0 1
2	E3	2 0	0 -1
3	E4	1 3	1 2
4	E5	3 2	1 -1
5	E6	1 4	2 3
6	E7	4 3	2 -1
7	E8	5 1	3 4
8	E9	5 4	3 -1
9	E10	0 5	4 -1

表 2.3 三角面片的拓扑关系

Table 2.3 Topological relation of triangular surfaces

索引(id)	三角面片号	顶点索引	边索引	相邻面片索引
0	T1	0 1 2	0 1 2	-1 4 1
1	T2	1 3 2	1 3 4	-1 0 2
2	T3	1 4 3	3 5 6	-1 1 3
3	T4	1 5 4	5 7 8	-1 2 4
4	T5	0 5 1	0 9 7	-1 3 0

2.2 碰撞检测

在进行模型的布尔运算时，当模型的三角面片比较稀少时，传统的方法采用对两个模型中的所有三角形进行两两相交测试，并对那些产生交点的三角形进行进一步处理，这种算法的时间复杂度是 $O(MN)$ ，其中 M ， N 分别代表两个模型中三角形的数量。现如今的模型数量往往达到十万甚至百万级别，原有的传统算法已不适用。

碰撞检测方法中的层次包围盒（AABBs）是由 Steven M. Rubin 和 Turner Whitted 在关于快速渲染复杂场景时所提出，它具有简单高效、快速碰撞检测、易于实现、良好的近似性能等特点。AABB 树作为空间索引的数据结构，目的在于降低了三角网格之间进行相交检测时所带来的不必要的时间消耗，因此 AABBs 能够提升碰撞检测的效率。

本文采用基于 AABB 层次树的快速碰撞检测算法，相比于传统的 AABB 层次包围盒碰撞算法中直接对两模型构建层次二叉树，本文先构建两模型各自的 AABB 包围盒，然后求取两个包围盒的公共部分，因为碰撞只可能发生在此相交区域，接着把公共部分当成共同的根节点区域分别构建层次包围盒树，通过自上向下原则生成叶子节点实现碰撞检测，这样做的好处在于通过第一步可以筛除大量不相交的三角形，提高碰撞效率。

2.2.1 数据结构

为了更加好表达 AABB 层次包围盒的碰撞检测算法，本文先介绍几个和碰撞检测有关的数据结构，它们是 AABB 层次包围盒碰撞检测的基础。

(1) AABB 类

```
Public class AABB{
    double minX;
    double minY;
    double minZ;
    double maxX;
```

```
double maxY;
double maxZ;
}
```

AABB 类是 AABB 层次包围盒碰撞检测的基础类，minX，minY，minZ，maxX，maxY，maxZ 这六个成员变量组成包围物体的最小立方体。

(2) Node 类

```
Public class Node{
    AABB boundingBox; //当前节点的包围盒
    List<Triangle> triangles; //当前节点的三角形集合
    Node leftChild; // 左子节点
    Node rightChild; // 右子节点
}
```

Node 类是节点类，表示 AABB 层次包围树中的各个节点，其中储存有当前节点的包围盒、当前节点的三角形集合、当前节点的左子节点、当前节点的右子节点等信息。

2.2.2 AABB 包围盒预检测

由上述可知，三维空间中知道模型的两个点(X_{min} , Y_{min} , Z_{min}), (X_{max} , Y_{max} , Z_{max}), 就可以表示出此模型的 AABB 包围盒，可以用以下集合来表示：

$$\{(x, y, z) | x \in [X_{min}, X_{max}], y \in [Y_{min}, Y_{max}], z \in [Z_{min}, Z_{max}]\} \quad (2.1)$$

从上面公式可知，AABB 包围盒本质上是一个通过沿三个坐标轴方向上的投影界定而成的立方体区域。在解析诸如巷道、地层等三维地质模型的 OFF 格式文件时，针对每一个读入的三角形顶点坐标数据，可以进行检测，看其是否超出了当前包围盒在各个坐标轴方向上预设的边界范围。一旦发现某个顶点坐标值超越了现有边界，立即依据这个新的坐标值调整包围盒的边缘位置，确保包围盒能够恰好完全包裹住这个坐标点。通过对 OFF 文件中每一处三角形顶点坐标的逐一分析与适时更新包围盒边界的过程，能够在读取文件的同时，逐步构建起模型适用的 AABB 包围盒。这种同步构建的方式极为高效，因为它几乎不会消耗额外的时间成本，在完成 OFF 文件读取之时，即可同步生成精确反映模型边界范围的 AABB 包围盒。

以巷道模型以及地层模型为例，当两模型的 OFF 文件读取完毕，这时它们的初始包围盒也构建完成，首先，需要判断这两个包围盒是否相交，依据是两个包围盒在坐标轴上的投影是否发生重合，如果没有发生重合，即巷道模型包围盒的

最小坐标值比地层模型包围盒的最大坐标值还要大，或巷道模型包围盒的最大坐标值比地层模型包围盒的最小坐标值还要小，则直接判断出两模型不相交，结束后续相关算法。反之，如果两模型的包围盒在每个坐标轴的投影都重合，则对两个包围盒求出交集部分，这个交集所代表的包围盒就是碰撞检测区域。分别遍历巷道与地层模型中的所有三角形，筛选出在此区域中的三角形并存入集合。此区域可以由下述公式所表示：

$$\begin{cases} V_x = V_{Ax}(X_{\min}, X_{\max}) \cap V_{Bx}(X_{\min}, X_{\max}) \\ V_y = V_{Ay}(Y_{\min}, Y_{\max}) \cap V_{By}(Y_{\min}, Y_{\max}) \\ V_z = V_{Az}(Z_{\min}, Z_{\max}) \cap V_{Bz}(Z_{\min}, Z_{\max}) \\ U = \{P | P.x \in V_x, P.y \in V_y, P.z \in V_z\} \end{cases} \quad (2.2)$$

其中， U 代表碰撞检测区域，也是两模型 AABB 包围盒的交集区域， P 为 U 区域中的点， A 、 B 分别代表巷道以及地层模型的包围盒， V_x 为 A 、 B 两包围盒在 x 轴上的交集， V_y 为 A 、 B 两包围盒在 y 轴上的交集， V_z 为 A 、 B 两包围盒在 z 轴上的交集。如图 2.5 所示，为部分地层与巷道的三维理论模型，图 2.6 为两模型的 AABB 包围盒，图 2.7 黄色包围盒为两模型 AABB 包围盒的交集区域，也是碰撞检测区域。

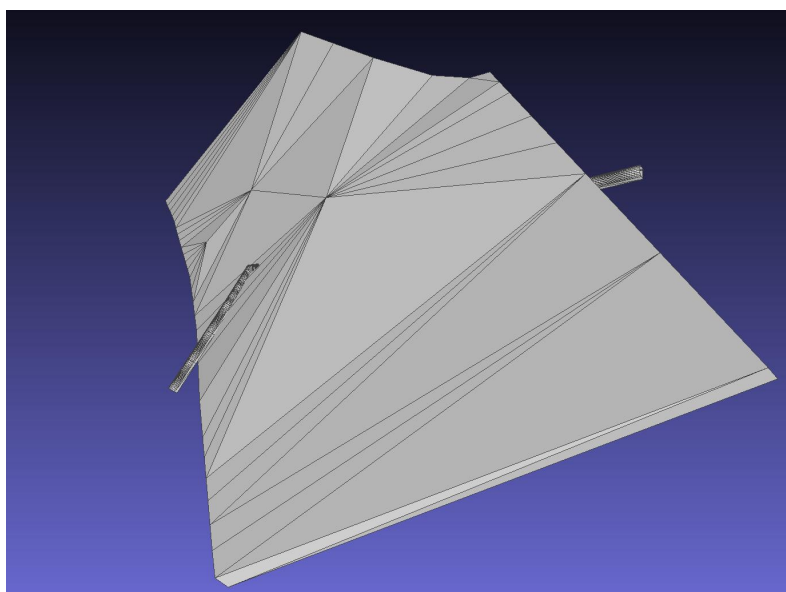


图 2.5 地层与巷道三维模型图

Fig. 2.5 3D model of strata and roadway

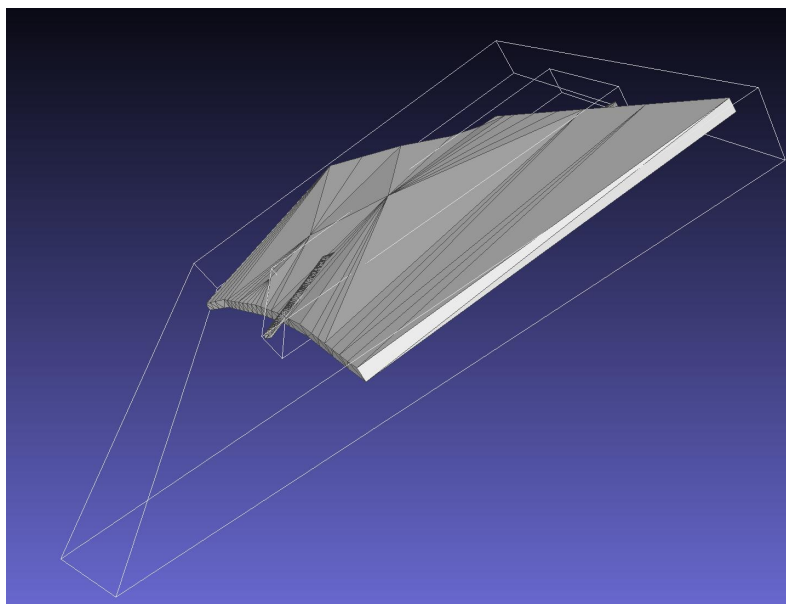


图 2.6 地层与巷道 AABB 包围盒图

Fig. 2.6 AABB bounding box diagram of strata and roadway

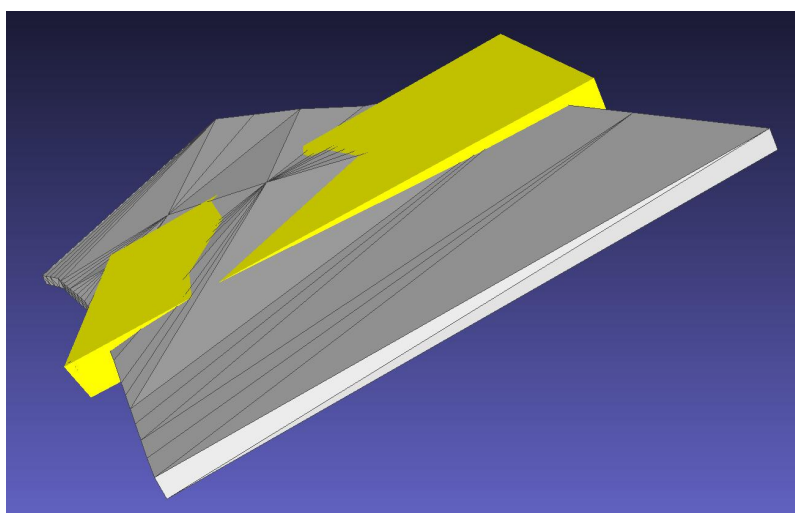


图 2.7 碰撞检测区域图

Fig. 2.7 Collision detection area diagram

2.2.3 构建包围盒树

由上述可知，两模型碰撞只可能发生在图 2.7 的黄色包围盒中，所以构建两模型各自包围盒树的算法如下：

输入：根节点 `root`

输出：包围盒树 `rootTree`

(1) 初始化左右子集。创建两个空集合 `leftTriangles` 和 `rightTriangles`，用于存储分割后的三角形。

(2) 确定切分平面。首先计算包围盒在三个坐标轴上的尺寸 dx , dy , dz ，然

后比较这三个尺寸的大小，确定最大的尺寸所对应的坐标轴 $axis$ ，遍历当前节点中所有三角形，求出它们的共同中心点在 $axis$ 轴上的平均坐标值 val 。

（3）分割三角形。遍历当前节点中所有三角形，对每个三角形计算其中心点坐标在 $axis$ 轴上的值 $pval$ ，比较 $pval$ 与 val 的大小，如果 $pval < val$ ，把当前三角形加入到 $leftTriangles$ 集合中，如果 $pval > val$ ，则把当前三角形加入到 $rightTriangles$ 集合中。

（4）更新左右子包围盒。初始化 $leftBox$ 和 $rightBox$ ，根据 $leftTriangles$ 以及 $rightTriangles$ 子集中的三角形更新对应的包围盒。

（5）创建左右子节点并递归调用。使用更新后的包围盒和对应的三角形集合创建左右子节点。如果当前节点中的三角形数量超过阈值（本文规定为 100），并且子集的三角形数量不等于当前节点（这表明确实进行了分割），则对左右子节点进行递归调用，否则停止递归。

图 2.8 为巷道以及地层构建好的层次包围盒，其中黄色代表巷道模型的层次包围盒，红色代表地层的层次包围盒。

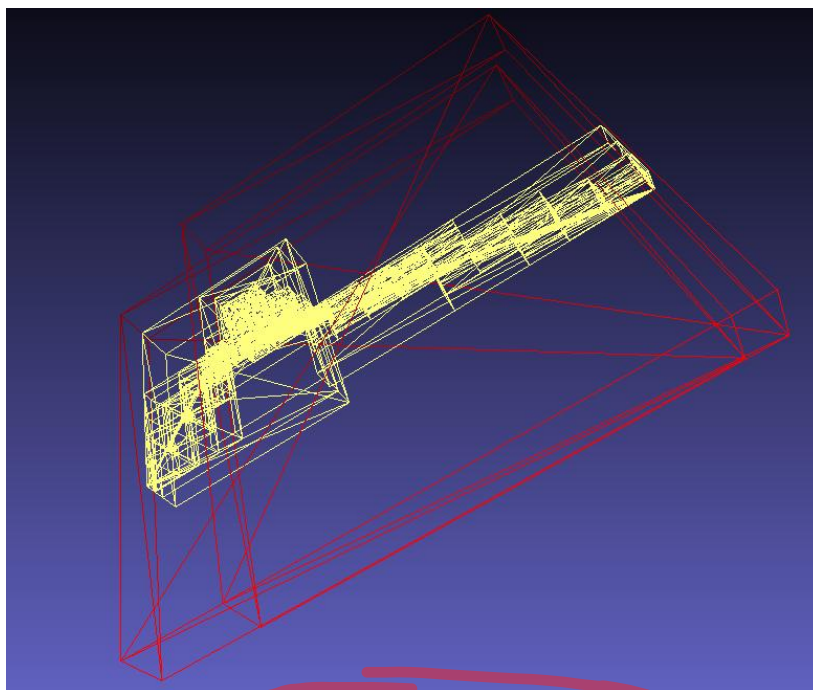


图 2.8 层次包围盒树图

Fig. 2.8 Hierarchical bounding box tree

2.2.4 包围盒树遍历

在包围盒树中，根节点内储存模型的最大包围盒以及所有三角形集合，其余每个节点内储存的包围盒以及三角形都是根节点的子集。通过对包围盒树进行遍历，可以得到两模型具体有哪些三角形对相交。这个过程是一个递归的双重遍历：

算法首先使用一个模型包围盒树的根节点去遍历另一个模型的包围盒树，如果能够遍历到叶节点，则使用该叶节点去遍历第一个模型的包围盒树，如果能到达该模型的叶子节点，则进一步对两个叶子节点中包含的三角形进行相交测试。在这个过程中，若两个节点上的包围盒发生相交，那么便需要进一步对其包含的三角形进行相交测试。假若两个节点上的包围盒不相交，则可以确定它们所包含的三角形也不会相交，因此无需执行进一步的相交测试。具体算法如下：

(1) 首先传入两颗模型的包围盒树 `nodeTree1`、`nodeTree2`。

(2) 判断当前两节点的包围盒是否相交，如果不相交，则没有发生碰撞，如果相交则跳转到第三步。

(3) 判断当前两节点是否同为叶子节点，如果同为叶子节点，则对两叶子节点所包含的三角形进行精确求交。

(4) 如果第一个节点为叶子节点，第二个节点是内部节点，分别把第一个节点以及第二个节点的左子节点和右子节点作为当前节点，跳转至第二步。

(5) 如果第一个节点为内部节点，第二个节点是叶子节点，分别把第一个节点的左子节点和右子节点以及第二个节点作为当前节点，跳转至第二步。

(6) 如果当前节点都是内部节点则执行以下操作：

- 1: 把第一个节点以及第二个节点的左子节点作为当前节点，跳转至第二步。
- 2: 把第一个节点以及第二个节点的右子节点作为当前节点，跳转至第二步。
- 3: 把第一个节点的左子节点以及第二个节点作为当前节点，跳转至第二步。
- 4: 把第一个节点的右子节点以及第二个节点作为当前节点，跳转至第二步。

可以简化为以下代码：

```
Public static void detectCollision(nodeTree1, nodeTree2){
    if(nodeTree1 与 nodeTree2 的包围盒不相交)
    {
        返回两对象不发生碰撞的结果;
    }
    if(nodeTree1 与 nodeTree2 均为叶子节点)
    {
        对 nodeTree1 与 nodeTree2 中所包含的三角形进行精确求交;
    }
    else if(nodeTree1 为叶子节点, nodeTree2 为非叶子节点)
    {
        detectCollision(nodeTree1, nodeTree2 的左子节点);
        detectCollision(nodeTree1, nodeTree2 的右子节点);
    }
}
```

```

else if(nodeTree1 为非叶子节点, nodeTree2 为叶子节点)
{
    detectCollision(nodeTree1 的左子节点, nodeTree2);
    detectCollision(nodeTree1 的右子节点, nodeTree2);
}
else{// nodeTree1 和 nodeTree2 都为非叶子节点
    detectCollision(nodeTree1, nodeTree2 的左子节点);
    detectCollision(nodeTree1, nodeTree2 的右子节点);
    detectCollision(nodeTree1 的左子节点, nodeTree2);
    detectCollision(nodeTree1 的右子节点, nodeTree2);
}
}

```

2.3 相交测试

2.3.1 三角形位置关系

两三角形在空间中存在两种状态，一种是相交，另外一种为相离，其中相交又包括共面、碰撞等奇异情况，本文不做考虑，相离即不产生交点，对于相交状态可以以交点的个数进行划分。具体可以分为图 2.9 以下三种情况：

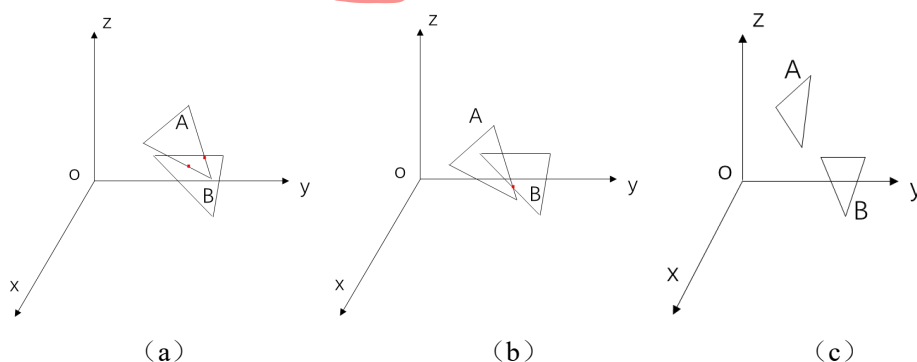


图 2.9 三角形空间位置图

Fig. 2.9 Triangular space location map

对于第一种情况，由图 2.9 (a) 可知三角面片 A 和 B 有两个交点，这两个交点连线为这两个三角形的交线，将它们保存到交线集合中，用于后续多条交线段配对成环。对于第二种情况，由图 2.9 (b) 可知只有一个交点，即 A 和 B 两个空间三角形只相交与一点，是一种退化的交线，这样的情况不必记录。第三种情况为相离，如图 2.9 (c) 所示，即三角形 A、B 的交点个数为零，两个三角形没有公共交点。

2.3.2 空间相交测试

空间相交测试分为射线与三角形相交测试以及三角形与三角形相交测试。

(1) 射线与三角形相交测试

其中 Moller Trumbore^[68]算法是射线与三角形之间判定是否相交的快速算法，首先假设射线的方程为：

$$\vec{O} + t\vec{P} = 0 \quad (2.3)$$

其中 O 为射线的起点， P 为射线的方向向量， t 为射线上的一点到起点 O 的距离。假设三角形的平面方程为：

$$(1 - b_1 - b_2)\vec{P}_0 + b_1\vec{P}_1 + b_2\vec{P}_2 = 0 \quad (2.4)$$

其中 $b_1, b_2, (1 - b_1 - b_2)$ 分别是三个点的重心坐标， P_0, P_1, P_2 为三角形的三个顶点。判定射线与三角形是否相交，只需联立方程求解 b_1, b_2 ，满足 $b_1 \geq 0, b_2 \geq 0, b_1 + b_2 \leq 1$ 则说明射线与三角形相交。联立 (2.3)，(2.4) 并整理得到以下方程：

$$\vec{O} - \vec{P}_0 = -\vec{D}t + (\vec{P}_1 - \vec{P}_0)b_1 + (\vec{P}_2 - \vec{P}_0)b_2 \quad (2.5)$$

为了简化 (2.5) 式，用 $S = \vec{O} - \vec{P}_0, E_1 = \vec{P}_1 - \vec{P}_0, E_2 = \vec{P}_2 - \vec{P}_0$ ，所以可得：

$$S = Eb_1 + E_2b_2 - \vec{D}t \quad (2.6)$$

把 (2.6) 中方程写成下述矩阵形式：

$$\begin{bmatrix} -D & E_1 & E_2 \end{bmatrix} \begin{bmatrix} t \\ b_1 \\ b_2 \end{bmatrix} = [S] \quad (2.7)$$

通过克莱姆法则，可以得到 $A_t = [S, E_1, E_2], A_{b_1} = [-D, S, E_2], A_{b_2} = [-D, E_1, S]$ ，所以可得：

$$t = \frac{\det A_t}{\det A} = \frac{\det [S, E_1, E_2]}{\det [-D, E_1, E_2]} \quad (2.8)$$

通过混合积运算可得：

$$t = \frac{(S \times E_1) \cdot E_2}{E_1 \cdot (D \times E_2)} \quad (2.9)$$

令： $S_1 = D \times E_2, S_2 = S \times E_1$ ，最终得到：

$$t = \frac{S_2 \cdot E_2}{E_1 \cdot S_1} \quad (2.10)$$

同理可得：

$$b_1 = \frac{S_1 \cdot S}{E_1 \cdot S_1} \quad (2.11)$$

$$b_2 = \frac{S_2 \cdot D}{E_1 \cdot S_1} \quad (2.12)$$

通过比较 b_1 以及 b_2 的值，看是否满足 $b_1 \geq 0$ ， $b_2 \geq 0$ ， $b_1 + b_2 \leq 1$ ，如果满足，则说明该射线与三角形相交。

（2）三角形与三角形相交测试

对于三角形对之间的相交测试，本文采用初剑^[69]提出的方法判断两空间三角形对是否相交以及进行交点坐标的求取。

假设模型 1 的三角网格中含有三角面片 $\triangle A_1A_2A_3$ ，模型 2 的三角网格中含有三角面片 $\triangle B_1B_2B_3$ ，如图 2.10 所示，现用 $\triangle A_1A_2A_3$ 的每条边和 $\triangle B_1B_2B_3$ 求交以及 $\triangle B_1B_2B_3$ 中的每条边和 $\triangle A_1A_2A_3$ 求交，详细步骤如下：

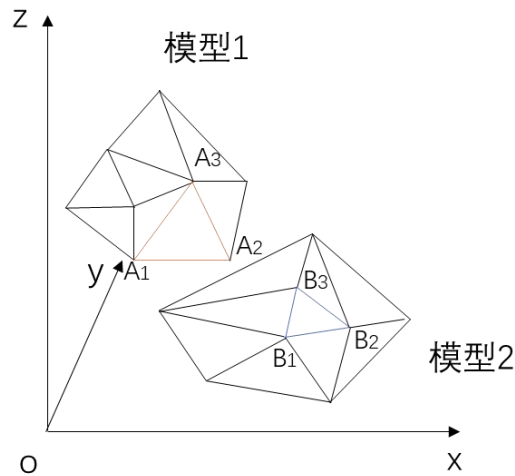


图 2.10 三角形相交测试图

Fig. 2.10 Triangle intersection test plot

1) 求线段 A_1A_2 所在直线的参数方程：

$$A = A_1 + t(A_2 - A_1) \quad (t \text{ 是参数}, -\infty \leq t \leq +\infty) \quad (2.13)$$

2) 确定 t_0 ，使点 $A_0 = A_1 + t_0(A_2 - A_1)$ 在 $\triangle B_1B_2B_3$ 的平面之上，既满足：

$$\overrightarrow{A_0B_1} \cdot (\overrightarrow{B_1B_2} \times \overrightarrow{B_1B_3}) = 0 \quad (2.14)$$

3) 通过公式 (2.14) 得到 t_0 ，如果 $0 \leq t_0 \leq 1$ ，说明得到的交点在线段 A_1A_2 上，而不在其延长线上，接下来要判断交点 A_0 是否在 $\triangle B_1B_2B_3$ 的边上或者是内部。如果符合条件， A_0 必须满足以下条件：

$$\begin{cases} \left(\overrightarrow{A_0 B_1} \times \overrightarrow{A_0 B_2} \right) \cdot \left(\overrightarrow{B_3 B_1} \times \overrightarrow{B_3 B_2} \right) \geq 0 \\ \left(\overrightarrow{A_0 B_2} \times \overrightarrow{A_0 B_3} \right) \cdot \left(\overrightarrow{B_1 B_2} \times \overrightarrow{B_1 B_3} \right) \geq 0 \\ \left(\overrightarrow{A_0 B_3} \times \overrightarrow{A_0 B_1} \right) \cdot \left(\overrightarrow{B_2 B_3} \times \overrightarrow{B_2 B_1} \right) \geq 0 \end{cases} \quad (2.15)$$

如若 A_0 符合 3) 中的条件, 证明 A_0 是三角形 $\triangle A_1 A_2 A_3$ 和 $\triangle B_1 B_2 B_3$ 的一个交点, 接下来需要完成 $\triangle A_1 A_2 A_3$ 的另外两条边 $A_2 A_3$ 以及 $A_3 A_1$ 对 $\triangle B_1 B_2 B_3$ 的相交测试, 以及 $\triangle B_1 B_2 B_3$ 的三条边对 $\triangle A_1 A_2 A_3$ 的三次测试。等完成六次测试之后, 对得到的交点进行去除重复点的操作。若得到的交点个数为:

①零个点: 证明 $\triangle A_1 A_2 A_3$ 和 $\triangle B_1 B_2 B_3$ 没有交点, 为相离状态。

②一个交点: $\triangle A_1 A_2 A_3$ 和 $\triangle B_1 B_2 B_3$ 只相交与一点, 说明形不成交线或者交线退化, 这种情况不必考虑。

③两个交点: 这两个交点相连起来即为 $\triangle A_1 A_2 A_3$ 和 $\triangle B_1 B_2 B_3$ 的交线, 储存到交线段集合中。

2.4 交线环提取

在包围盒碰撞检测筛除大量不相交的三角面片之后, 仍需要一些三角面片对进行两两求交, 求出许多杂乱的线段, 提取交线环就是把这些杂乱的线段按照正确的拓扑关系一一连接起来形成交线环, 用于后续巷道模型以及地层模型的布尔运算。其中, 求出的交点坐标因为是按照浮点数进行存储的, 所以会存在浮点误差。当模型的三角网格比较大时, 这时求出的交线段相对比较长, 浮点误差带来的影响可以忽略不记, 但是当三角形极小或者狭长时, 由于浮点运算的误差, 有可能导致交线对无法正确连接形成交线环。所以如何规避浮点运算误差是提取交线环的关键。

浮点运算误差是由于计算机在进行浮点数运算时所引入的近似值所产生的。这是因为计算机使用有限的位数来表示实数, 而实数集是无限的。因此, 在进行浮点运算时, 会出现舍入误差以及截断误差。舍入误差是指将一个实数近似表示成浮点数时所引入的误差。由于计算机只能表示有限位数的小数, 无法准确地表示无限位数的实数。当一个实数无法精确表示时, 计算机会根据规定的舍入规则将其近似表示成最接近的浮点数。这个近似值与实际值之间的差异就是舍入误差。比如考虑将 0.1 这个实数近似表示成浮点数的情况。由于计算机无法精确表示 0.1, 它会根据舍入规则将其近似的表示。在单精度浮点数中, 0.1 的近似值为 0.100000001490116119384765625。这里的差异就是舍入误差。而截断误差是指在进行浮点数运算时, 计算结果被截断为有限位数而引入的误差。由于浮点数的精

度是有限的，当进行复杂运算时，中间结果可能会超出浮点数的表示范围，计算机会将其截断为有效位数内的值。这个截断过程导致了结果的不准确性，即截断误差。比如计算 $0.1234+0.5678$ ，并将结果截断为两位小数。首先进行加法运算，得到 0.6912 ，然后将结果截断为两位小数，最终结果为 0.69 。这里截断误差是 0.0012 ，即 0.6912 和 0.69 之间的差异。这些误差可能在复杂的浮点运算中累积和放大，导致最终结果与预期结果之间存在差异。

对于交点的浮点运算误差，实际是因为三角面片在求交的过程中同一条边与其它三角形求交时产生不同的两个交点。具体情况如图 2.11 所示：三角形 $\triangle T_2$ 和 $\triangle T_3$ 分别与 $\triangle T_1$ 相交， $\triangle T_2$ 中的一条边与 $\triangle T_1$ 相交于点 P_1 ， $\triangle T_3$ 中同一条边与 $\triangle T_1$ 相交于点 P_2 ，同一条边与三角形相交应该产生同一个交点，但是由于浮点运算的误差可能导致点 P_1 和 P_2 不相等。这种误差产生的影响可能会一步步扩大，最终导致巷道与地层等三维地质模型进行布尔运算时发生错误。

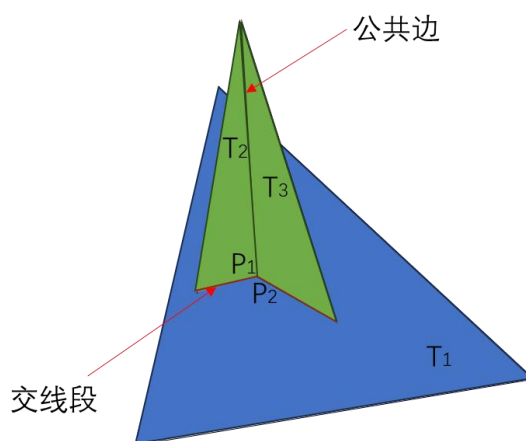


图2.11 三角形求交浮点误差问题

Fig. 2.11 Floating point error problem of triangle intersection

关于三角面片求交过程中浮点运算产生的误差，有许多学者进行相关的研究，问题的关键所在是把同一条边产生的交点合并，即点 P_1 和 P_2 合并成同一个交点，其中用的最多的是按照距离去进行合并。基于距离法的基本原理是，初始化集合 $List<Point>1$ 用来储存交线环，在存储离散交线对的集合 $List<Edge>2$ 中先取出一条交线段作为起始交线段并把首尾端点储存在 $List<Point>1$ 中，把此交线段从集合 $List<Edge>2$ 中删除，然后遍历集合中剩下的交线段，当某条交线段的端点坐标和起始交线段的端点坐标之差小于某个阈值，一般为 1×10^{-6} ，合并两个端点把另一端点储存在 $List<Point>1$ 中，并把此交线段从 $List<Edge>2$ 中删除，接着遍历剩余交线段重复上述操作，直到剩余交线段的某个端点和起始端点重合，证明第一条交线环提取完毕，如果还有剩余交线段，可以继续进行第二条、第三条……交线环的提取。

基于距离提取交线环的操作存在许多问题，首先如果遇到极度狭长或小的三角面片时，阈值为 1×10^{-6} 或许不能适用，其次每次遍历剩余交线段进行配对的操作其算法效率比较慢。

本文采用基于模型三角网格本身的拓扑结构进行相同交点合并，实现交线环的提取，由于模型本身是致密并具有拓扑关系一致性与连续性，所以三角形的一条边会被两个三角形所共有，称为公共边，根据第二章 2.1.2 中边的数据结构可知，已知三角形的一条边，可得另一个含有此边的三角面片，以及边上存储的交点信息，由于两模型的三角面片进行两两求交，公共边会被运算两次并得到两个交点，根据公共边的信息可以把这两个交点进行合并，达到消除浮点运算带来误差的目的。

这种基于模型三角网格本身的拓扑结构进行相同交点合并的算法是十分高效的，因为在读取 OFF 文件的同时，相关的点、线、面的拓扑关系已经建立起来，所以用公共边去搜索待配对的交线段是可行并高效率的。为实现上述算法，除了第二章 2.1 中的基础数据结构，现在建立相交三角形对以及它们之间的交点的数据结构。如下：

(1) IntersectionPoint 类

```
Public class IntersectionPoint{
    double x;
    double y;
    double z;
    Pair<Integer, Integer> edgeIndex;
}
```

其中 Pair 类是 java 中自带的一个泛型类，用于储存两个对象。它有两个参数类型，分别表示两个对象的类型。其中 Pair <Integer, Integer> 中的第一个 Integer 代表的是 faceId，为面的 id 号，第二个 Integer 代表的是 edgeId，为边的 id 号，所以 edgeIndex 代表交点所属的边索引号，表示位于 faceId 号三角形的第 edgeId 条边上。

(2) IntersectionTriangle 类

```
Public class IntersectionTriangle{
    Pair<Integer, Integer> faceIndex;
    Pair< IntersectionPoint, IntersectionPoint > intersectionPoints;
}
```

faceIndex 是指两个相交三角面片的信息，intersectionPoints 表示这两个相交三角面片的交点信息。

提取交线环的关键在于利用模型自身的拓扑结构进行相同交点的合并，交线环提取的具体步骤如下：

(1) 起点的选择: 初始化集合 $\text{List}\langle\text{Point}\rangle$ intersectionLoop 用来储存交线环，在存储离散交线对的集合 $\text{List}\langle\text{Edge}\rangle$ edges 中先取出一条交线段作为起始交线段，把这条交线段从 $\text{List}\langle\text{Edge}\rangle$ edges 中删除，选取起始交线段的某一端点作为起点，对应的另一点作为尾部点，把这两点按顺序加入 $\text{List}\langle\text{Point}\rangle$ intersectionLoop 中。

(2) 离散交线段的配对: 交线段的配对关键在于对公共边产生的交点进行合并，根据已连接交线环的尾部点，确定下一个要连接的交线段。具体实现可分为两步。1) 从剩余 $\text{List}\langle\text{Edge}\rangle$ edges 中找到与前面尾部点相连接的交线。2) 合并相同交点，并把交线的另一端点加入到 $\text{List}\langle\text{Point}\rangle$ intersectionLoop 中。具体的示意图可见图 2.12。假设此时的尾部点为 P_2 ，已知此时的交线段 P_1P_2 是由三角面片 $\triangle T_2$ 和 $\triangle T_1$ 相交所产生，根据交点 P_2 所在的边可以直接搜索到邻接三角形 $\triangle T_3$ ，又已知 $\triangle T_3$ 和 $\triangle T_1$ 相交所产生交线段 P_3P_4 ，故交线段 P_3P_4 是下一个需要连接的交线段，由于 P_2 、 P_3 是同一条边所产生的交点，所以把两者合并，并把 P_4 加入到 $\text{List}\langle\text{Point}\rangle$ intersectionLoop 中，重复上述操作，直到提取到的交线段的一端和首端点相同。

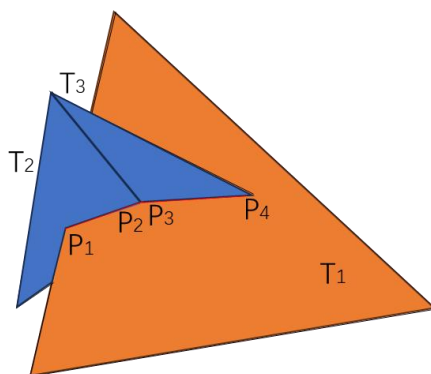


图 2.12 交线段配对

Fig. 2.12 Crossover segment pairing

对于交线环的方向，本文规定不论是巷道模型还是地层模型，它们三角面片顶点的排列顺序都为逆时针，以巷道三角形为例，当需要对巷道三角形进行重构时，每条交线段的方向等于巷道三角形的法向量叉乘地层三角形的法向量的方向。假设交线段为 P_1P_2 ，巷道三角形的法向量为 n_1 ，地层三角形的法向量为 n_2 ， $n = \vec{n_1} \times \vec{n_2}$ ， P_1 的坐标 (x_{p1}, y_{p1}, z_{p1}) ， P_2 的坐标 (x_{p2}, y_{p2}, z_{p2}) ， n 的坐标 (x_n, y_n, z_n) ，如果 $(x_{p2} - x_{p1}, y_{p2} - y_{p1}, z_{p2} - z_{p1})$ 的方向与 (x_n, y_n, z_n) 相同，则交线段的方向是由 $P_1 \rightarrow P_2$ ，反之则是 $P_2 \rightarrow P_1$ 。所以每条交线段的方向顺逆也决定了交线环方向的顺逆。

如下图所示，假设半圆拱形模型为巷道模型，不规则立方体为地层模型，图

2.13 (a) 为模型的顶面，图 2.13 (b) 为模型的底面，由图可知两模型共有三处发生相交，分别为相交 1、相交 2、相交 3。

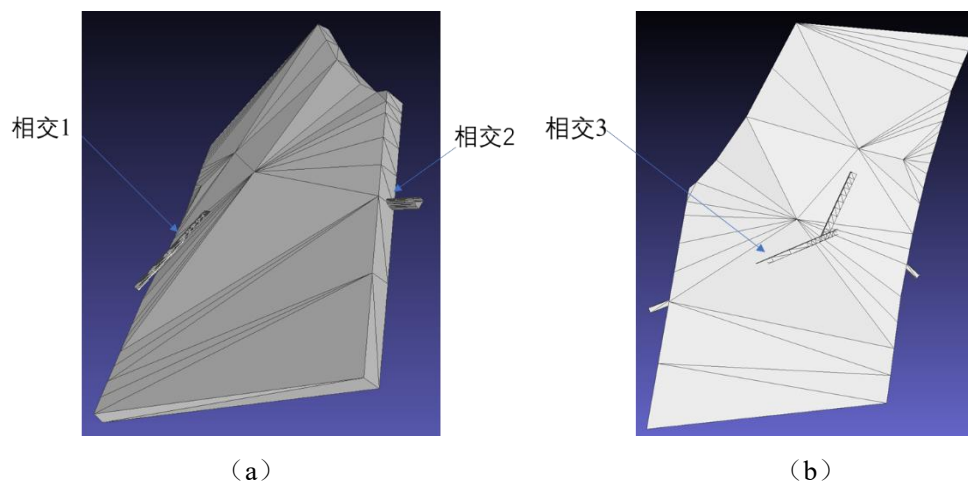


图 2.13 模型顶、底面

Fig. 2.13 Top and bottom of the model

共形成了三个交线环，交线环 1 如图 2.14 (a) 所示，交线环 2 如图 2.14 (b) 所示，交线环 3 如图 2.14 (c) 所示。总体的交线环如图 2.14 (d) 所示。因为三角面片顶点的排列顺序都为逆，所以模型的每个三角面片法向量都朝外，图中的交线环方向同为顺时针。

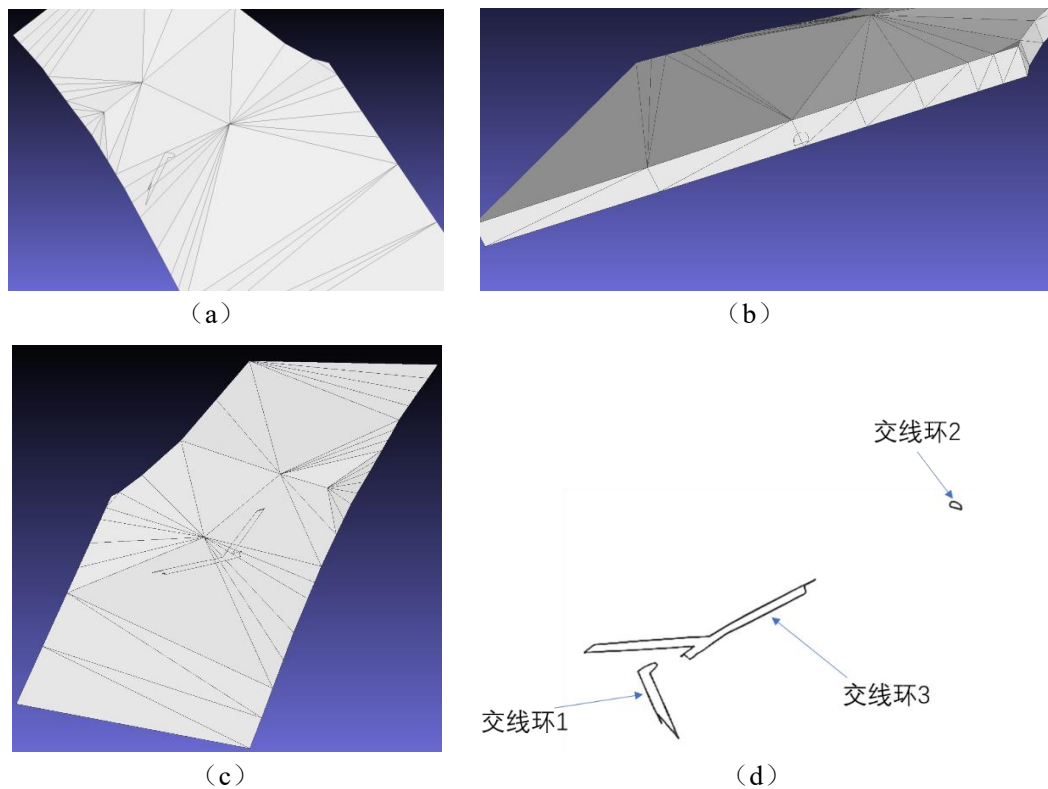


图 2.14 交线环

Fig. 2.14 Intersecting loop

2.5 本章小结

本章主要介绍在模型的布尔运算中交线如何求取，共分为以下几点：

- （1）读取模型文件，建立相应的数据结构，根据数据结构进行模型拓扑关系的建立。
- （2）基于层次包围盒算法对模型进行碰撞检测，筛除大量不相交三角形，对相交的三角形进行精确求交并得到一系列离散交线对。
- （3）基于模型本身的拓扑结构，对离散交线对进行两两连接，最后形成交线环。

3 布尔运算中模型重构及分类

3.1 模型重构

对于2.4中提取的交线环，记录交线环经过的三角形，把这些三角形进行重构是三维地质模型之间进行布尔运算的关键，对三角形进行重构大致分为以下几个步骤：

- (1) 对每个交线环经过的三角形进行多边形提取。
- (2) 对提取的多边形进行区域分类。
- (3) 对内外多边形进行三角剖分。

3.1.1 多边形提取算法

由于交线环把每个三角形切割成不同的多边形区域，所以重构三角形的第一步就是对每个三角形进行多边形区域提取。

以巷道模型和地层模型进行布尔运算为例，对于巷道三角形而言，根据地层起伏情况的不同，巷道三角形可以被切割成任意个多边形区域，当地层较为平缓时，切割出来的多边形较为简单，对于多边形的提取方法也较为简单，如图 3.1 所示，红色线段为交线环在三角形 $\triangle ABC$ 中的部分，把 $\triangle ABC$ 分割为 1, 2 两个多边形区域，对于 (a) 中两条边各有一个交点的情况，提取多边形的原理为：根据 $\text{edgeAB.intersectionP.size()=1}$ ， $\text{edgeBC.intersectionP.size()=0}$ ， $\text{edgeCA.intersection.size()=1}$ 可知，交线环和三角形 $\triangle ABC$ 分别相交于边 AB 和边 AC，交点为 a 和 c，已知边 AB 和边 AC 相交于点 A，所以把顶点 A 以及交点 a、b、c 以逆时针加入多边形集合中，得到 Aabc 为区域 1，同理，把顶点 B、C 以及交点 a、b、c 以逆时针加入集合中，得到 aBCcb 为区域 2，多边形区域 1 和 2 提取完毕。对于 (b) 中一条边有两个交点的情况，根据 $\text{edgeAB.intersectionP.size()=0}$ ， $\text{edgeBC.intersectionP.size()=2}$ ， $\text{edgeCA.intersection.size()=0}$ 可知，交线环和三角形 $\triangle ABC$ 相交于边 BC，交点为 a 和 e，所以把三角形的三个顶点以及交线的点以逆时针加入多边形集合中，得到 ABabcdeC，为区域 1，同理得到 aedcb 为区域 2，其中 $\text{edge.intersectionP.size()}$ 为边上交点的个数。

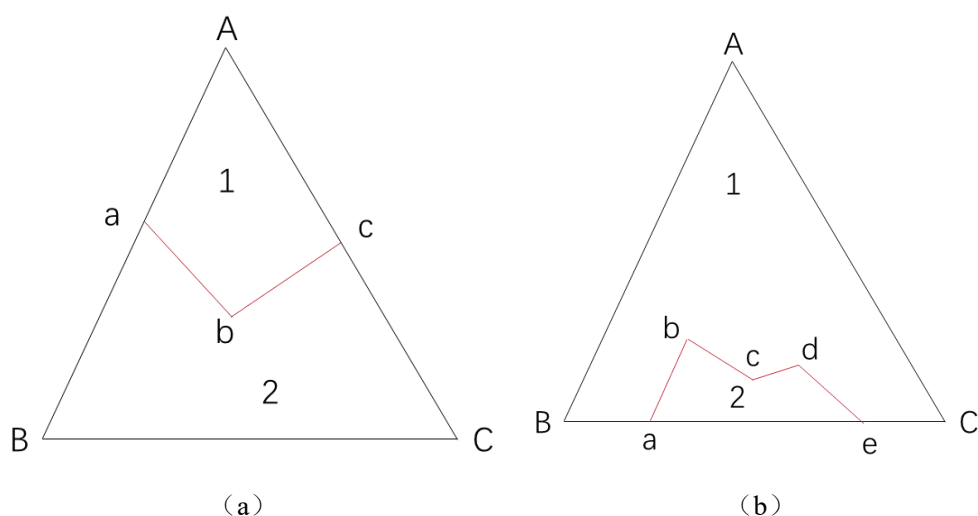


图 3.1 被切割成两个多边形

Fig. 3.1 Cut into two polygons

这种方法在面对上述简单的情况下是适用的，但是面对切割成好几个多边形的情况无法解决，真实的地层是起伏多变的，所以面对复杂的切割情况，本文采用以下算法实现多边形区域的自动提取。如图 3.2 所示：三角形 $\triangle MNO$ ，1、2 为交线段和边 MN 的两个交点，3、4、5、6 是交线段和边 MO 的交点，7、8、9、10 为交线段与边 OM 的几个交点。提取多边形的具体操作如下所示：

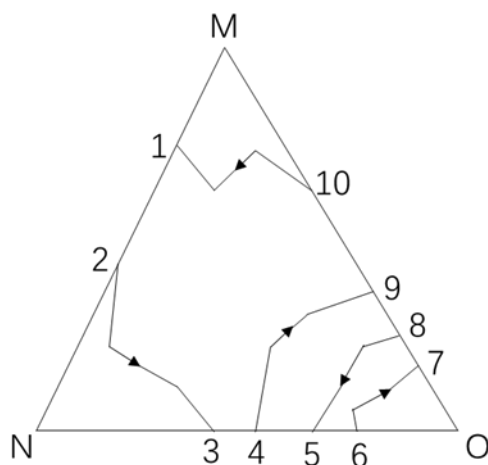


图 3.2 被切割成多个多边形

Fig. 3.2 Cut into polygons

输入：三角形 $\triangle MNO$ 以及交线段集合。

输出：逆时针排列的六个多边形。

(1) 对三角形的三个顶点以及交点 1-10 进行逆时针排序，得到 M-1-2-N-3-4-5-6-O-7-8-9-10-M 的点集。具体如下：

I: 初始化集合 $\text{List}\langle \text{Point} \rangle$ AnticlockwisePoints 用于存放逆时针点，先把顶点

M 放入集合中，然后边 MN 调用方法 `getIntersectionP()` 可以获得其上的交点集合，但是集合中的点 1、2 是无序的，接着分别求取交点 1、2 到顶点 M 的欧氏距离，并按距离从小到大的顺序依次把点 1、2 加入到集合 `AnticlockwisePoints` 中。

II: 对边 NO 以及边 OM 重复 I 中操作，直到得到 M-1-2-N-3-4-5-6-O-7-8-9-10-M 的点集。

(2) 初始化集合 `List<Point> polygon` 用于存放单个多边形，`List<List<Point>> polygons` 存放 $\triangle MNO$ 中提取到的所有多边形区域。遍历集合 `AnticlockwisePoints` 中的点，从 M 开始，当碰到交线段上的点，则沿着该线段遍历并把线段上的点加入到集合 `polygon` 中，等遍历完该交线段，接着从集合 `AnticlockwisePoints` 中该交线段的尾点的下一个点接着遍历，直到回到起点，停止遍历并标记为已访问，一个封闭多边形区域提取完毕，并把集合 `polygon` 加入到集合 `polygons` 中。

(3) 接着从集合 `AnticlockwisePoints` 中下一个点遍历并提取封闭多边形区域，直到所有点遍历完毕。所有多边形已提取完毕。提取的结果如图 3.3 所示：按照先后顺序一共分为六个封闭多边形区域，分别为 1、2、3、4、5 以及 6。这六个多边形中的点按照逆时针顺序排列。

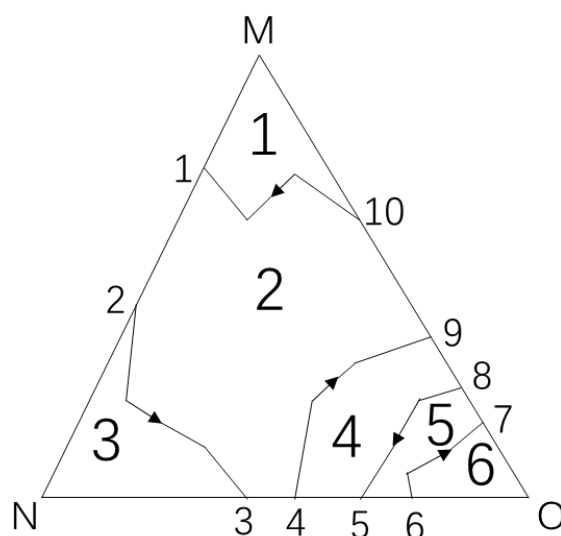


图 3.3 多边形提取

Fig. 3.3 Polygon extraction

3.1.2 多边形区域划分

三角面片被交线环切割为不同的封闭多边形区域，判断多边形区域在交线环内或者外，为下一步三角剖分后三角形的内外提供依据，多边形在交线环内，由它剖分出的三角形则是内部三角形，反之则为外部三角形。由于多边形的存储方式都为逆时针存储，所以判断多边形区域内外的关键在于对多边形中交线段上的点和原交线段进行顺序判断，如果顺序相同，说明此多边形在交线环内部，反之

则在交线环外部。具体分类步骤如下：

(1) 调用图 3.3 中提取完毕的集合 `polygons` 并对其遍历，首先对封闭多边形区域 1 进行判断，定义一个布尔类型的变量 `in`，并将它的值初始化为 `false`。

(2) 遍历 $\triangle MNO$ 中原始交线段集合，获取当前交线段的首端点 `startPoint` 以及尾端点 `endPoint`，如果封闭多边形区域 1 中包含 `startPoint` 以及 `endPoint` 则转至步骤 (3)，否则接着判断多边形区域 1 中是否包含其它交线段的首端点 `startPoint` 以及尾端点 `endPoint`。

(3) 定义变量 `int startIndex`、`endIndex`，表示该线段的起点与尾端点在多边形 1 中的索引，定义集合 `List<Point> polygonSegment`，用于储存当前交线段在多边形 1 中具体位置，多边形 1 调用 `java` 中集合的自带方法 `indexOf(Point)`，使 `int startIndex = 多边形 1.indexOf(startPoint)`，`int endIndex = 多边形 1.indexOf(endPoint)`，如果 `endIndex=0`，`polygonSegment=多边形 1.subList(startIndex, 多边形 1.size())`，如果 `startIndex=0`，`polygonSegment=多边形 1.subList(endIndex, 多边形 1.size())`，如果 `startIndex ≤ endIndex`，`polygonSegment=多边形 1.subList(startIndex, endIndex+1)`，否则 `polygonSegment = 多边形 1.subList(endIndex, startIndex + 1)`。其中 `subList` 为 `java` 语言自带的函数，用于截断操作。

(4) 对比集合 `polygonSegment` 中的第一个元素，是否与当前交线段的第一个元素相同，如果相同，把 `in` 赋值为 `true`，表示当前多边形区域在交线环的内部。否则，当前多边形区域在交线环的外部。

(5) 对其余每个封闭多边形区域重复上述(1)到(4)中操作。

根据上述算法可得，内部多边形区域为：2 和 5，外部多边形区域为：1、3、4 以及 6。如下图 3.4 所示：

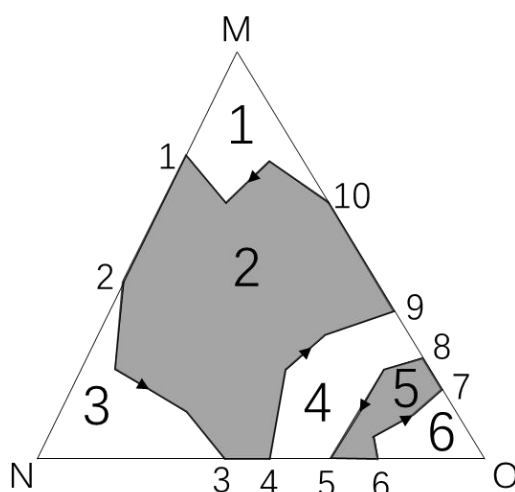


图 3.4 多边形区域划分
Fig. 3.4 Polygon partitioning

3.1.3 多边形三角剖分

把一个多边形分割为一系列三角形集合的方法称为多边形的三角剖分。常用的三角剖分算法有：Delaunay 三角剖分算法、几何分解法、拓扑细分法以及节点连接法。其中 Delaunay 三角剖分算法应用最为广泛，这种剖分技术起源于 Voronoi 图，由俄国数学家 M.G. Voronoi 在 1908 年提出。随后，Delaunay 在 1932 年提出了一种全面且实用的解决方案来处理该问题。在随后的数十年中，这个算法经历了持续的优化和提升，以适应多样化的应用需求。Delaunay 三角剖分算法的实现方式有很多种，主要可以分为两大类：增量算法和分治算法。增量算法从点集中的一个点开始，逐步添加其余点，直到所有点都被添加完毕。这类算法的常见实现包括 Green-Sibson 算法、Bowyer 算法、Lawson 算法等。分治算法则基于“分而治之”的策略，将点集分成两个大致相等的子集，然后递归地对它们进行处理，最后将结果合并。目前较为优秀的分治算法有 DeWall 算法。文献^[70]提出一种改进 Delaunay 三角剖分算法，该算法实现 Delaunay 三角网权重与损失值的实时计算，有效滤除损失值总和与不满足条件的三角边。而对于复杂多边形的三角剖分，更多采用的是由 David Eberly 提出的耳切法(Ear-Clipping)三角剖分算法，该方法不但能解决一般情况下交线环和三角面片相交出的复杂多边形区域，也能解决凹的多边形以及含有孔洞的多边形。本文采用耳切法对封闭多边形区域进行三角剖分。耳切法实现封闭多边形区域的三角剖分主要分为四个步骤：（1）对多边形区域进行投影，使其降维至二维平面。（2）由于投影的二维平面不同，投影出的二维多边形可能为顺也可能为逆，把顺时针的多边形转换为逆时针用于三角剖分。（3）用耳切法对每个内、外多边形进行剖分。（4）对剖分后的每个三角形与三维多边形进行坐标匹配（id 号匹配）。

（1）多边形的投影

投影法是一种将三维空间中的物体展现到二维平面上的技术。根据投影线的角度，投影法主要分为中心投影法和平行投影法两种类型。平行投影法的一个显著特征是投影大小与物体和投影面之间的距离无关，因此它具有良好的度量性。平行投影法中的正投影法能够精确而完整地展示物体的形状和结构，并且绘图过程相对简单，度量性也较好，因此在工程领域中得到了广泛的应用。

本文以对巷道模型上的三角形投影为例，采用正投影法先对交线环经过的巷道三角形进行投影，每个三角形的投影方向既是其上的多边形的投影方向。根据三角形在巷道的不同位置，可以大致分为以下三类进行投影：

1：交线环经过的三角形在巷道的侧壁

巷道在开挖采掘工程中一般是垂直于岩层的，所以判定三角形在侧壁只需要判定三角形的单位法向量 $|z|$ 是否接近于 0。确定三角形在侧壁之后，还需确定是

向 xz 平面投影或是向 yz 平面投影。

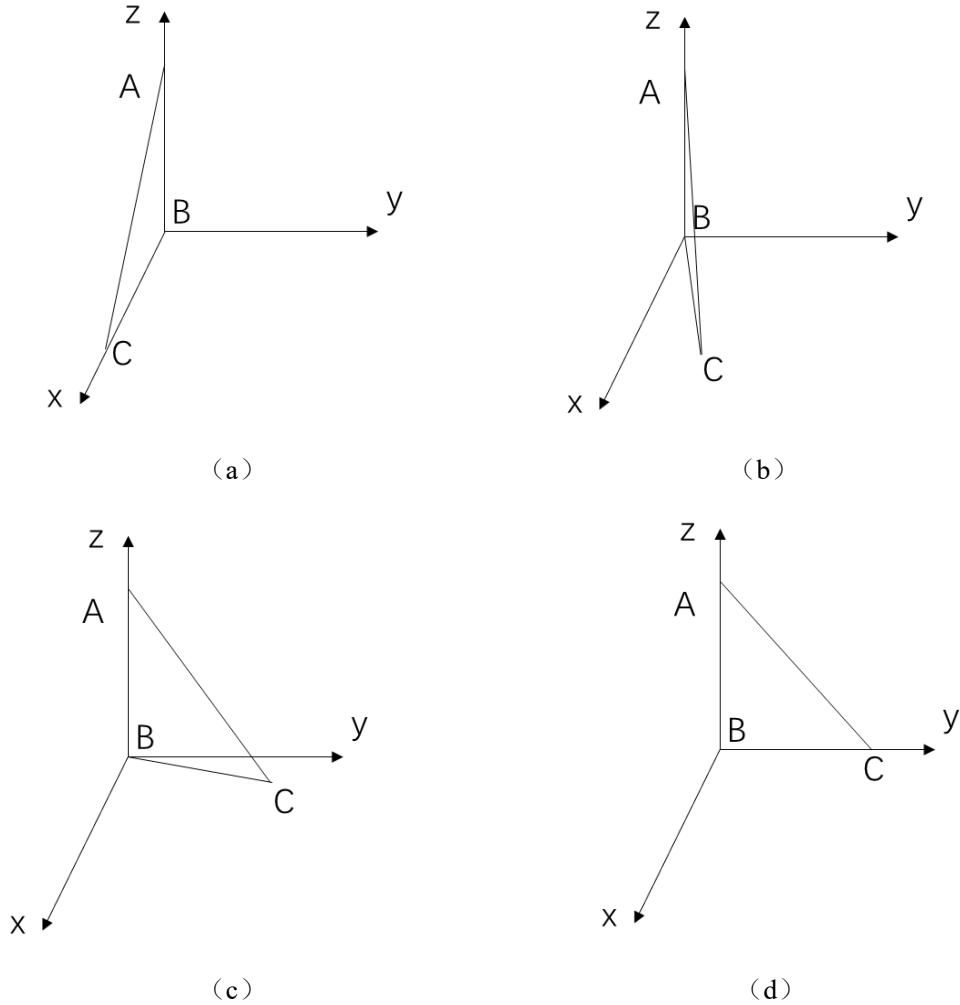


图 3.5 三角形在巷道侧壁

Fig. 3.5 The triangle is on the side wall of the roadway

图 3.5 (a) 中三角形在 xz 平面上，其中 A 的坐标为 $(0, 0, 1)$ ，B 的坐标为 $(0, 0, 0)$ ，C 的坐标为 $(1, 0, 0)$ ，A, B, C 逆时针， $\overrightarrow{AB} = (0, 0, -1)$ ， $\overrightarrow{BC} = (1, 0, 0)$ 。所以图 (a) 三角形 ABC 的单位法向量 $\vec{n}_1 = (0, -1, 0)$ ，图 3.5 (b) 中三角形靠近 xz 平面，其中 A 的坐标为 $(0, 0, 1)$ ，B 的坐标为 $(0, 0, 0)$ ，C 的坐标为 $(0.7, 0.2, 0)$ ， $\overrightarrow{AB} = (0, 0, -1)$ ， $\overrightarrow{BC} = (0.7, 0.2, 0)$ ，所以图 3.5 (b) 三角形 ABC 的单位法向量 $\vec{n}_2 = (0.2, -0.7, 0)$ ，图 (c) 中三角形靠近 yz 平面，其中 A 的坐标为 $(0, 0, 1)$ ，B 的坐标为 $(0, 0, 0)$ ，C 的坐标为 $(0.2, 0.8, 0)$ ， $\overrightarrow{AB} = (0, 0, -1)$ ， $\overrightarrow{BC} = (0.2, 0.8, 0)$ ，所以图 (c) 三角形 ABC 的单位法向量 $\vec{n}_3 = (0.8, -0.2, 0)$ ，图 (d) 中三角形在 yz 平面，其中 A 的坐标为 $(0, 0, 1)$ ，B 的坐标为 $(0, 0, 0)$ ，C 的坐标为 $(0, 1, 0)$ ， $\overrightarrow{AB} = (0, 0, -1)$ ， $\overrightarrow{BC} =$

(0, 1, 0)，所以图 (d) 三角形 ABC 的单位法向量 $\vec{n}_4 = (1, 0, 0)$ 。

所以当三角形在巷道侧壁上时，通过比较 $\vec{n}_1, \vec{n}_2, \vec{n}_3, \vec{n}_4$ 的 $|x|$ 以及 $|y|$ 的值可以得出三角形的单位法向量 $|y| > |x|$ 则向 xz 平面投影， $|x| > |y|$ 则向 yz 平面投影。

2: 交线环经过的三角形在巷道拱顶正上方或者底部

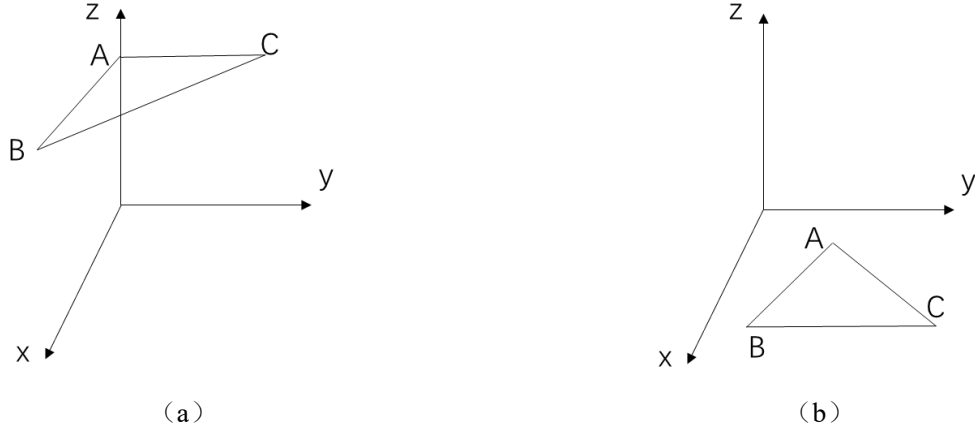


图 3.6 三角形在巷道顶部或底部

Fig. 3.6 The triangle is at the top or bottom of the roadway

图 3.6 (a)、图 3.6 (b) 三角形的单位法向量 $\vec{n}_5, \vec{n}_6$ 的 $|z|$ 近似为 1，所以向 xy 平面投影。

3: 交线环经过的三角形在巷道的侧壁与拱顶的过渡部位

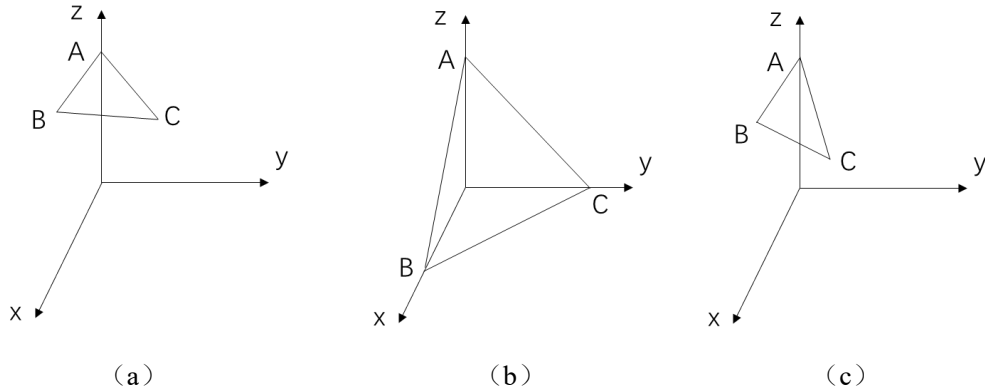


图 3.7 三角形在巷道侧壁和拱顶过渡部位

Fig. 3.7 The triangle is in the transition between the side wall of the roadway and the vault

图 3.7 (a) 中三角形靠近顶部，其中 $A(0, 0, 2)$ ， $B(1, 0, 2)$ ， $C(0, 1, 1)$ ， $\vec{AB} = (1, 0, 0)$ ， $\vec{BC} = (-1, 1, -1)$ 。所以图 (a) 的法向量 $\vec{n}_7 = (0, 0.71, 0.71)$ ，图 3.7 (b) 中三角形位于过渡部位，其中 $A(0, 0, 2)$ ， $B(2, 0, 0)$ ， $C(0, 2, 0)$ ， $\vec{AB} = (2, 0, -2)$ ， $\vec{BC} = (-2, 2, 0)$ 。所以图 (b) 的法向量 $\vec{n}_8 = (0.58, 0.58, 0.58)$ 。图 (c) 中三角形靠近侧壁，其中 $A(0, 0, 2)$ ， $B(1, 0, 2)$ ， $C(0, 0.4, 0.4)$ ， $\vec{AB} =$

$(1, 0, 0)$, $\overrightarrow{BC} = (-1, 0.4, -1.6)$ 。所以图 3.7 的 (c) 中的法向量 $\overrightarrow{n_9} = (0, 0.97, 0.24)$ 。

由上可知，当三角形的单位法向量的 $|z| \geq 0.5$ 时，三角形的投影方向为 xy 平面。当 $|z| < 0.5$ 时，由 1 中结论判断 $|x|$ 和 $|y|$ 的大小以确定投影平面。

综上所述，判定交线环所经过的三角形的投影方向为下：

I：当三角形的单位法向量的 $|z| \geq 0.5$ 时，向 xy 平面进行投影。

II：当三角形的单位法向量的 $|z| < 0.5$ 时，判定 $|x|$ 和 $|y|$ 的大小，如果 $|x| \geq |y|$ ，向 yz 平面投影，如果 $|x| < |y|$ ，则向 xz 方向投影。

根据上述结论，把给定的三维封闭多边形转化为二维多边形的算法如下：

输入：三角形 T，多边形 List<Point> polygon3d。

输出：polygon2d。

第一步：获取三角形 T 的三个顶点 A、B、C，并根据这三个顶点算出三角形 T 的单位法向量 n。

第二步：初始化集合 polygon2d，类型为 List<Point>。如果：

I：Math.abs(n.z) ≥ 0.5 ，遍历 Polygon3d，对其中每个三维点 point3d 进行二维化操作，即使 point3d.z=0，得到 point2d (point3d.x, point3d.y, 0.0)，并把 point3d 的 id 号赋值给 point2d，最后把点 point2d 加入到集合 polygon2d 中。

II：Math.abs(n.z) < 0.5 && Math.abs(n.x) $< \text{Math.abs}(n.y)$ ，遍历 Polygon3d，对其中每个三维点 point3d 进行二维化操作，即使 point3d.y=0，得到 point2d (point3d.x, 0, point3d.z)，并把 point3d 的 id 号赋值给 point2d，最后把点 point2d 加入到集合 polygon2d 中。

III：Math.abs(n.z) < 0.5 && Math.abs(n.x) $\geq \text{Math.abs}(n.y)$ ，遍历 Polygon3d，对其中每个三维点 point3d 进行二维化操作，即使 point3d.x=0，得到 point2d (0, point3d.y, point3d.z)，并把 point3d 的 id 号赋值给 point2d，最后把点 point2d 加入到集合 polygon2d 中。

第三步：输出集合 polygon2d。

得到二维多边形集合 polygon2d 用于后续操作。

(2) 多边形顺逆的检查

由于交线环经过的巷道三角形有可能在巷道的不同部位，投影后所在的投影平面不同，分别在 xy 平面，xz 平面，yz 平面。所以投影之后的封闭多边形区域有可能为逆时针，也有可能为顺时针，为了后续进行多边形三角剖分，现在把所有待剖分的多边形都转化为逆时针方向。本文采用端点判断法进行多边形顺逆时针的判断，如图 3.8 所示：假设多边形在 xy 平面上，多边形 $P_1P_2P_3P_4P_5P_6$ ，其中 $P_1(x_1, y_1)$ ， $P_2(x_2, y_2)$ ， $P_3(x_3, y_3)$ ， $P_4(x_4, y_4)$ ， $P_5(x_5, y_5)$ ， $P_6(x_6,$

y_6)。判断多边形顺逆时针的具体方法为：1) 遍历多边形所有点，找到 x 最大的点 P_2 。2) 取该点前后各一个点 P_1 以及 P_3 ，组成向量 $\vec{n_1} = (P_1, P_2)$ 、 $\vec{n_2} = (P_2, P_3)$ 。3) 判断 $\vec{n_1} \times \vec{n_2}$ 的 z 值是否大于 0，如果大于零，说明它们法向量指向 z 轴正方向，多边形为逆时针，否则为顺时针。

根据端点判断法判定图 3.8 中多边形 $P_1P_2P_3P_4P_5P_6$ 为逆时针排列。其中 “+” 号代表该顶点为“凸角”，“-”号代表该顶点为“凹角”。

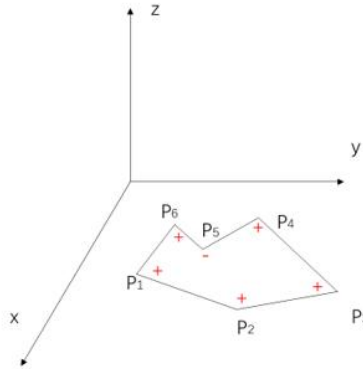


图 3.8 多边形逆时针排列图

Fig. 3.8 Polygon counterclockwise arrangement diagram

当多边形在 xz 平面上，首先遍历多边形的各个顶点，找出 x 最大的顶点，然后获得该顶点的前点以及后点，按照顺序组成两向量 $\vec{n_1}$ 以及 $\vec{n_2}$ ，判断 $\vec{n_1} \times \vec{n_2}$ 的 y 值是否大于 0，如果大于零，说明它们法向量指向 y 轴正方向，多边形为逆时针，否则为顺时针。同理，当多边形在 yz 平面时，遍历多边形的各个顶点，找出 y 值最大的顶点，然后获得该顶点的前点以及后点，按照顺序组成两向量 $\vec{n_1}$ 以及 $\vec{n_2}$ ，判断 $\vec{n_1} \times \vec{n_2}$ 的 x 值是否大于 0，如果大于零，说明它们法向量指向 x 轴正方向，多边形为逆时针，否则为顺时针。

如果判断多边形为顺时针排列，则可以反转点集合，把顺时针多边形转化为逆时针。

(3) 耳切法进行三角剖分

对于上述逆时针二维多边形，采用耳切法对其进行三角剖分，耳切法有以下四点定义：

1) 多边形的耳朵：在 V_1 是凸顶点的情况下，由 V_1 的前一个顶点 V_0 以及后一个顶点 V_2 组成的三角形，其内不包含任何其他顶点，其中顶点 V_1 称为多边形的耳尖。

2) 多边形的对角线：顶点 V_0 以及顶点 V_2 之间的连线称为多边形的对角线。

3) 一个多边形（边数大于 3）至少含有两个不相互重叠的耳尖。

4) 由 n 个顶点组成的多边形一共可以剖分为 $n-2$ 个三角形。

根据上述定义，采用循环的方式把多边形分解为一系列三角形，假设多边形有 n 个顶点，每个顶点为 $P_i (0 \leq i < n)$ ，每一次寻找到多边形的一个耳朵，把这个耳朵移除，并得到一个三角形，重复上述操作，直到多边形的点小于 3 个或者多边形的耳朵集合为空。耳切法剖分多边形的算法如下：

输入：多边形集合 $\text{List}\langle \text{Point} \rangle \text{polygon2d}$ 。

输出：三角形集合 $\text{List}\langle \text{Triangle} \rangle \text{triangles}$ 。

第一步：初始化集合 triangles ，用于存储剖分后的三角形，初始化集合 earPoints ，类型为 $\text{List}\langle \text{Point} \rangle$ ，用于储存多边形的耳朵。

第二步：遍历集合 polygon2d 中的每个顶点，判断其是否为耳朵，如果是，加入到集合 earPoints 中。

第三步：当集合 polygon2d 中的顶点个数 ≥ 3 并且集合 earPoints 不为空，进入 while 循环，否则，跳转到第四步。获取 earPoints 中的第一个元素 Point ear ，并把点 ear 从集合 earPoints 中移除，根据点 ear 可得其在多边形中的索引 earIndex ，以及前一个顶点索引 $\text{earIndex}-1$ 、后一个点索引 $\text{earIndex}+1$ ，根据索引可得多边形中三个相邻顶点，这三个顶点实例化成为一个三角形。把这个三角形加入到集合 triangles 中。接着把此耳朵顶点从多边形集合 polygon2d 中移除，由于把耳朵移除后，相邻的前后两顶点属性有可能发生变化，即原来不是耳朵，后变为耳朵，原来是耳朵，后不是耳朵两种情况。对于第一种情况，把此顶点加入到集合 earPoints 中，对于第二种情况，则把点从集合 earPoints 中移除。

第四步：输出三角形集合 $\text{List}\langle \text{Triangle} \rangle \text{triangles}$ 。

下文以图 3.9 (a) 所示多边形为例，具体展示运用耳切法对多边形进行三角剖分。首先通过遍历多边形集合中的顶点列表，得到初始凸点集 $A=\{0, 1, 3, 4, 6, 9\}$ ，初始耳朵集合 $B=\{3, 4, 6, 9\}$ ，顶点 0 和 1 不被视为耳朵，这是因为在由顶点 9、0 和 1 构成的三角形中，顶点 8 位于该三角形内部；同样，在由顶点 0、1 和 2 构成的三角形中，顶点 7 也位于该三角形内部。初始凹点集 $C=\{2, 5, 7, 8\}$ 。遍历初始耳朵集合 B ，当顶点 3 被移除，其对应的三角形为 $T_0=\{2, 3, 4\}$ ，图 3.9 (b) 展示了移除一个耳朵后的效果。相邻点 2 原本是个凹顶点，当顶点 3 被移除后顶点 2 依旧是个凹顶点，顶点 4 原本是个耳朵，当顶点 3 被移除后顶点 4 依旧是个耳朵，所以凹点集 C 不变，耳朵集合变成了 $B=\{4, 6, 9\}$ (3 被移除)。继续移除顶点 4，此时三角形对应的是 $T_1=\{2, 4, 5\}$ ，图 3.9 (c) 展示移除两个耳朵之后的效果。相邻顶点 2 依旧是个凹顶点，相邻顶点 5 之前是个凹顶点，移除之后变为了凸顶点，经过测试发现其为耳朵。因此移除顶点 4 后，列表的变化结果为，凹点集 $C=\{2, 7, 8\}$ ，耳朵集合为 $B=\{5, 6, 9\}$ (移除 4，新加入 5)。继续移除顶点 5，此时对应的三角形为 $T_2=\{2, 5, 6\}$ ，图 3.9 (d) 展示了移除三

个耳朵之后的效果。相邻顶点 2 起初是个凹顶点，但是移除后，变成了一个凸顶点，经过检测后发现顶点 7 位于由顶点 1、2、6 构成的三角形内，所以顶点 2 不是耳朵。移除之前顶点 6 是个耳朵，现在依旧是。所以变化后的点列表为，凹点集为 $C=\{7, 8\}$ ，耳朵集合 $B=\{6, 9\}$ （移除了顶点 5）。通过不断的重复上述操作完成多边形的剖分。下图 3.10 展示了多边形剖分的详细结果。

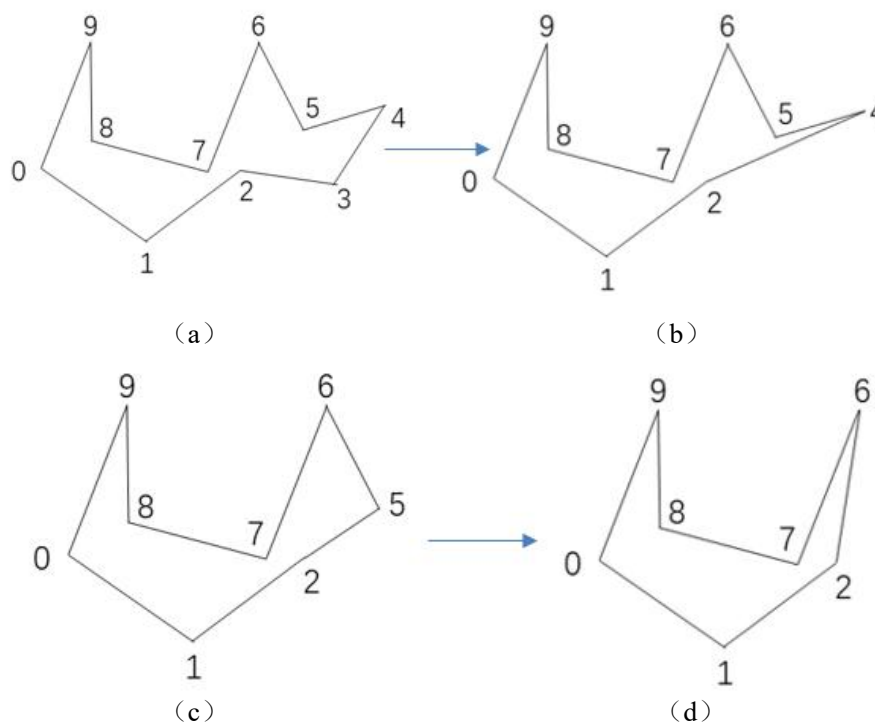


图 3.9 三角剖分过程图

Fig. 3.9 Triangulation process diagram

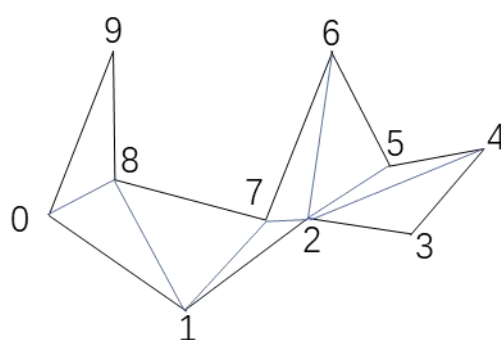


图 3.10 三角剖分结果图

Fig. 3.10 Triangulation results

(4) 坐标匹配

封闭多边形被剖分为一系列三角形，这些三角形都是二维三角形，即在平面 xy 、 xz 以及 yz 上，现在根据多边形中顶点的 id 号以及三角形中顶点的 id 号进行

匹配，当三角形中顶点的 id 号等于多边形中顶点的 id 号，把多边形这个顶点的坐标赋值给三角形的顶点。详细算法如下：

输入：Triangle t, List<Point>polygon。

输出：Triangle t(三维)。

1) 遍历三角形 $\triangle t$ 的三个顶点，和集合 polygon 中所有点进行 id 号匹配，如果匹配成功，把 polygon 的点的 x, y, z 赋值给 $\triangle t$ 的顶点。

2) 当 $\triangle t$ 的三个顶点都匹配成功并赋值完成，结束算法。

由于多边形被剖分为一系列三角形，所以使用上述算法可以使每个被剖分的三角形都还原成三维三角形。多边形在交线环内，由它剖分出的三角形则是内部三角形，反之则为外部三角形。

至此，交线环经过的三角面片重构完毕，并以交线环为边界，区分出内外三角面片，通过布尔运算把这些三角面片进行取舍与重组。

3.2 三角形内外分类

模型重构完成后，需要对交线环未经过的三角形进行内外分类，本文以巷道模型为例，其内部三角形是处在地层顶底之间的三角形，外部三角形是在顶底之外的三角形。如图 3.11 所示，箭头所指为外部巷道三角形，为了方便观看内部三角形，把地层的侧壁三角形删除，如图 3.12 所示，箭头所指为巷道内部三角形。图 3.11 以及图 3.12 都为第二章图 2.5 中理论模型的局部放大。

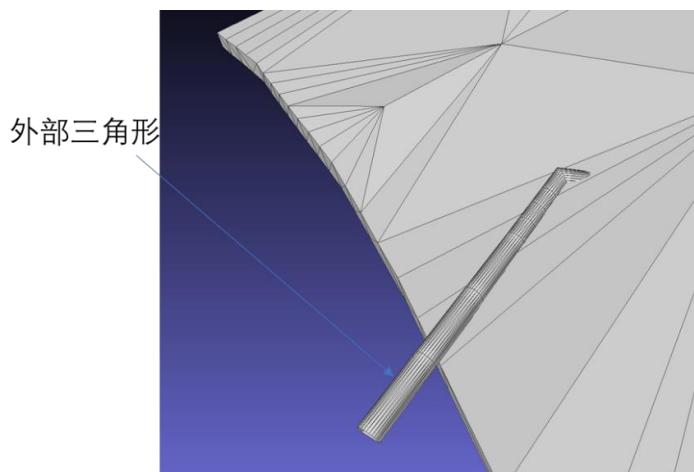


图 3.11 外部巷道三角形图

Fig. 3.11 Triangular diagram of external roadway

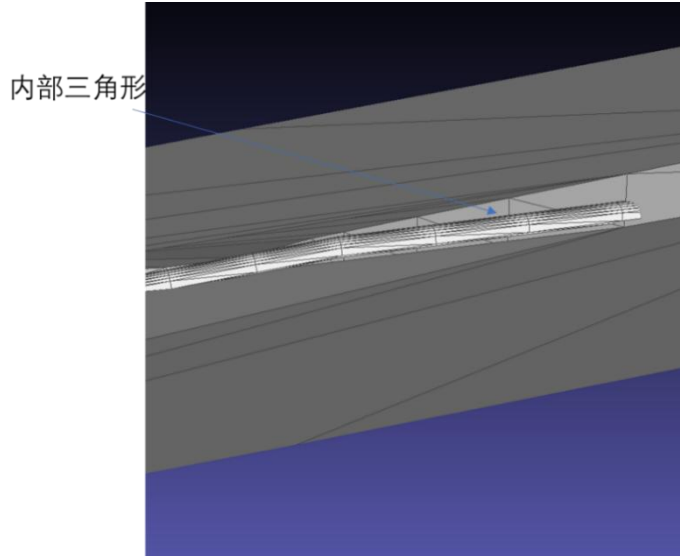


图 3.12 内部巷道三角形图

Fig. 3.12 Triangular diagram of inner workings

要区分三角形是在地层的内部还是外部，可以采用射线法，即如果一个三角形的三个顶点分别向 z 轴发射一条射线，如果每条射线与地层三角形相交的个数为奇数，说明这个三角形在地层的内部，为内部三角形，反之则为外部三角形。

以内部三角形为例，具体算法步骤如下：

输入：巷道三角形集合 `roadwayTriangles`、地层三角形集合 `layerTriangles`。

输出：内部巷道三角形集合 `inTriangles`。

(1) 初始化集合 `inTriangles` 用来存储内部巷道三角形。

(2) 遍历每个巷道三角形，用三角形的三个顶点 A ， B ， C 分别向 z 轴发射射线，得到射线 `rayA`，`rayB`，`rayC`。运用 2.3.2 中射线和三角形求交方法分别进行三条射线与地层三角形求交，并记录交点个数，如果三条射线产生的交点都为奇数个，说明此巷道三角形为内部三角形，加入到集合 `inTriangles` 中。

(3) 遍历结束，返回集合 `inTriangles`。

用射线法进行三角形内外分类虽然思路比较简单，当模型数据量比较小时可以采用此算法进行计算，但是当巷道模型以及地层模型的三角形个数比较多时，算法的效率比较低。

本文采用基于广度优先搜索（BFS）算法的图搜索技术，对三角网格中的面片进行内外分类。该算法的核心思想在于利用三角网格的几何和拓扑连续性，通过交线环将网格面片划分为内部分片和外部片，并保证在同一区域内，面片的内外属性是一致的。在这种结构中，三角面片的拓扑信息以图的形式呈现，而交线环则将这些区域分割成若干连通的部分。因此，只要从任意一个内部三角形或内部顶点开始，就可以扩展并识别出整个模型内所有其他的内部面片。同理，搜索外部三角形也是如此。利用 BFS 算法，能够高效地对三角面片进行搜索和分类，

从而快速完成整个模型的内外面片划分。本方法的重点在于先找到交线环内或者外部一点，以此点为起点，以交线环为限制边界，进行搜索遍历。以搜索模型内部三角形为例，算法的详细步骤如下：

（1）初始化集合 inTriangles 用来储存所需要的内部三角形，类型为 $\text{Set}\langle\text{Triangle}\rangle$ ，传入交线环经过的三角形集合用于边界搜索。

（2）找到交线环内的一点 surePoint ，根据上述区域分类算法可得一个三角形的内部多边形区域，如果一个点即属于内部多边形，又属于三角形顶点，此点即为 surePoint 点。

（3）初始化队列 queue ，把 surePoint 点储存入队列。

（4）处理队首顶点，根据三角网格的点-面存储结构，搜索所有包含此点的三角面片。遍历包含该顶点的每一个三角面片，如果它还没有被分类（即不是交线环经过的边界三角形也不是 inTriangles 内的三角形），加入集合 inTriangles 中，并处理其所含的 3 个顶点，若顶点不在 queue 中，则添加到队列 queue 末尾。这些三角面片处理完成后，队首顶点处理结束，从 queue 中删除此顶点。

（5）继续处理新的队首顶点，并把符合要求的三角形加入到集合 inTriangles 中。

（6）若 queue 中顶点为空，结束算法并返回集合 inTriangles 。

提取内部三角形的具体算法可以用图 3.13 表示：初始状态如图（a），先根据三角形 t_1 提取到的内部多边形找到点 p_1 ，并把 p_1 加入到队列中去，这时候队列中只有点 p_1 。根据三角网格点-面的数据结构搜索包含点 p_1 且不是交线环经过的所有三角形，找到了三角形 t_2 ， t_3 ， t_4 ，把这三个三角形加入到储存内部三角形的集合中，并把点 p_2 ， p_3 ， p_4 ， p_5 加入到队列中，将点 p_1 弹出，如图 3.13（b）所示，展示了 BFS 搜索过程。根据队列的队首顶点不断地搜索未知的内部三角形，直到队列为空，停止搜索，即可找出所有的内部三角面片，如图 3.13（c）所示。

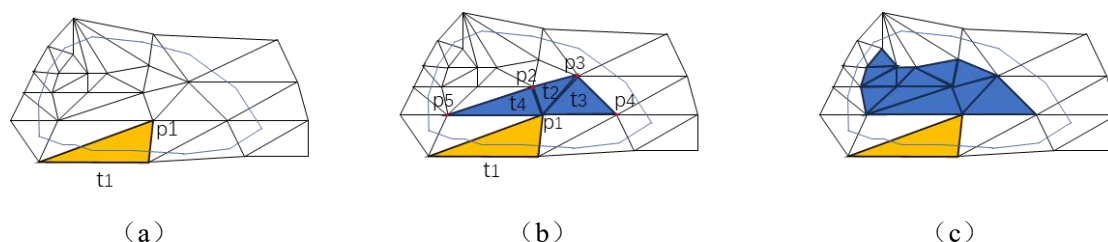


图 3.13 三角形内外分类算法示意图

Fig. 3.13 Triangle inside and outside classification algorithm diagram

以第二章图 2.5 中两模型为例，为了得到完整的内部三角形，以三个交线环经过的三角形为边界，根据内部点 p_1 搜索出全部的巷道内部三角形，如图 3.14 所示。

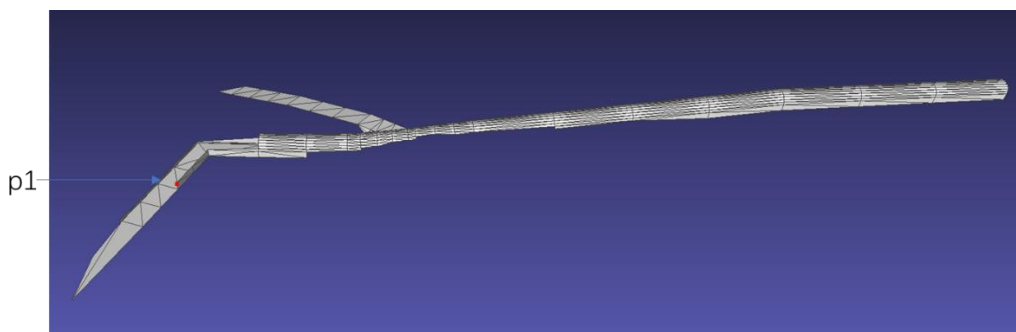


图 3.14 完整巷道内部三角形

Fig. 3.14 Complete roadway interior triangle

3.3 模型合并

一般情况下，三维地质模型之间的布尔运算可以用以下简单的数学公式来概括：

$$\begin{cases} A \cap B = A_{in}B + B_{in}A \\ A \cup B = A_{out}B + B_{out}A \\ A/B = A_{out}B + (B_{in}A)^{-1} \end{cases} \quad (3.1)$$

其中 A 、 B 表示两三维地质模型， $\cap$ 、 $/$ 、 $\cup$ 分别表示集合的交集、并集以及差集，其中 $A_{in}B$ 表示 A 模型在 B 模型内的所有三角形， $B_{in}A$ 表示 B 模型在 A 模型内的所有三角形，所以 $A \cap B$ 表示 A 模型在 B 模型内的所有三角形加上 B 模型在 A 模型内的所有三角形。 $A_{out}B$ 表示 A 模型在 B 模型之外的所有三角形，同理 $B_{out}A$ 表示 B 模型在 A 模型之外的所有三角形，所以 $A \cup B$ 表示 A 模型在 B 模型之外的所有三角形加上 B 模型在 A 模型之外的所有三角形。 $()^{-1}$ 代表将模型中的三角面的法向量取反。所以 A/B 代表在模型 A 中减去与模型 B 相互重叠的部分，得到 A 模型剩余部分。

三角形内外分类结束后，把需要的三角形根据布尔运算公式进行合并重组。本文将布尔运算的结果分为两部分，以两模型的布尔运算交集为例，三角形包括模型重构之后的内部三角形在加上三角形内外分类之后的完整内部三角形。以第二章图 2.5 中两模型为例，重构之后的巷道内部三角形由模型重构算法可以得出，如图 3.15 所示。完整的巷道内部三角形可由三角形内外分类算法得出，如图 3.14 所示，同理，所需地层三角形也是如此。两者合并重组可得到两模型布尔求交的结果，如图 3.16 所示。

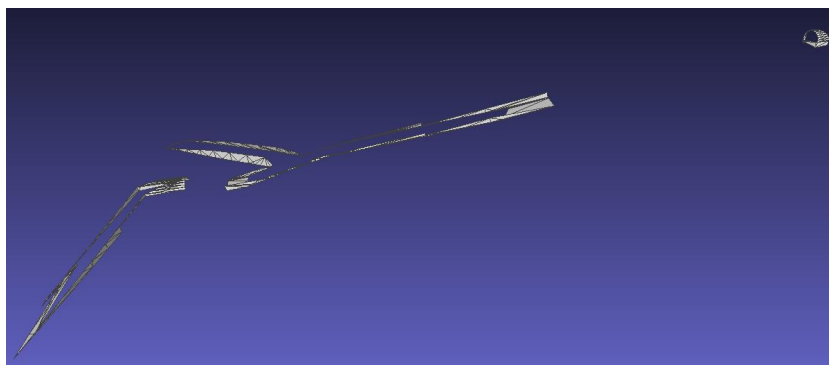


图 3.15 完整内部重构三角形

Fig. 3.15 Complete internal reconstruction triangle

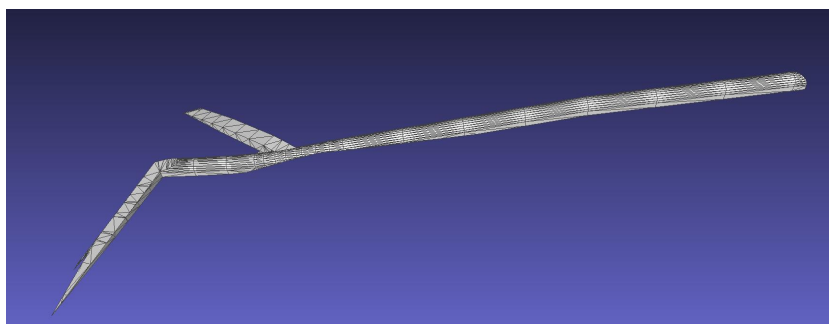


图 3.16 布尔运算交集结果图

Fig. 3.16 Boolean operation intersection result graph

根据布尔运算公式，巷道模型和地层模型的并集如图 3.17 所示，差集如图 3.18 所示。

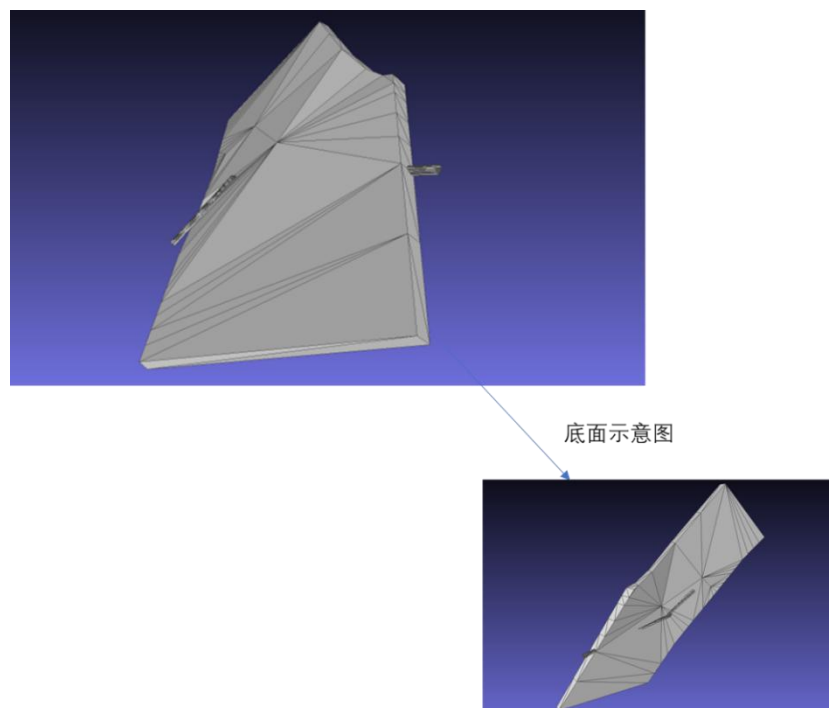


图 3.17 布尔运算并集结果图

Fig. 3.17 Boolean operation union result graph

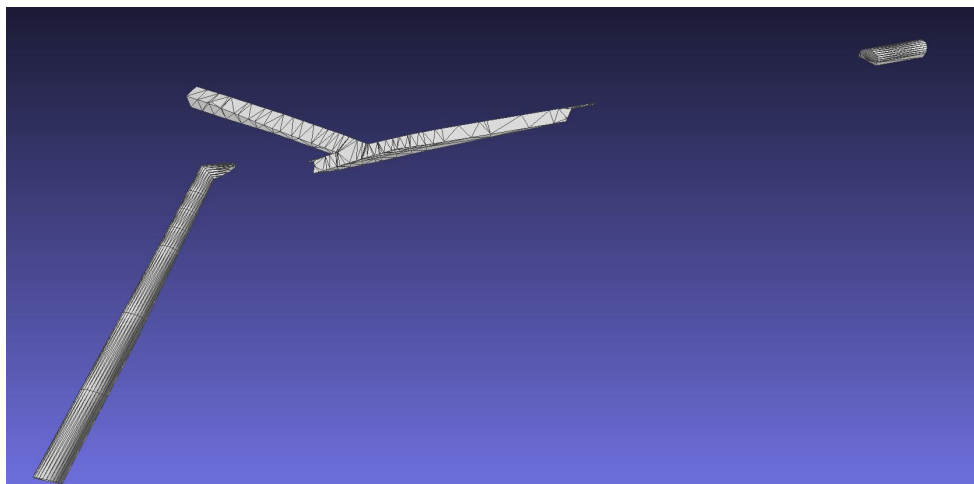


图 3.18 布尔运算差集结果图

Fig. 3.18 Boolean operation difference set result graph

3.4 本章小结

本章主要介绍在布尔运算中模型如何重构以及三角形分类，共分为以下几点：

（1）针对被交线环经过的三角形情况，记录交线环经过的三角形，对这些三角形进行多边形提取、区域分类、三角剖分，最后完成模型的重构。

（2）针对未被交线环经过的三角形，基于图的广度优先遍历算法（BFS）区分出内外三角形，最后把重构完成的三角形以及基于 BFS 算法区分的内外三角形依据布尔运算公式进行重组，得出三维地质模型中的布尔运算算法。

4 应用实例

4.1 研究区域背景

4.1.1 位置与交通

邢台矿坐落在河北省邢台市西南方向。该矿区行政上隶属于邢台市。在矿区的东侧，京广铁路穿行而过，矿区东北部到邢台火车站大约有 7 公里的距离，东部到小康庄站大约 4 公里。矿区设有与煤矿相连的铁路线，加之矿区内公路四通八达，因此交通十分便利。

4.1.2 地质概况

本区域属于石炭二叠系煤田，根据已知的钻孔数据，该地区的地层年代由老至新分别是奥陶系、石炭系、二叠系以及第四系。

邢台矿区地处山西中台凸起的东部。该区的断层类型主要为高角度正断层，方向主要为东西向、南北向及北东向，且以北东向断裂为主。此次调查共发现了 33 条落差超过 20 米的断层，平均 1.03 条/平方公里。大、中型断层是沿近平行走向分布的高角度正断层，形成狭长的地垒、地堑构造，其中 20 米以上的断层常作为采区的界线。

区内构造特点可归纳如下：

- 1) 构造线方向特性：地层走向主要沿着 $N10^{\circ}$ 至 $25^{\circ}E$ 的新华夏系方向延伸。
- 2) 本区域地层向东南方向倾斜，角度介于 5° 至 25° 之间。受到本区域内断层的影响，区域内有许多复杂地质构造产生，包括褶皱、大型断裂等。这些复杂地质情况的产生导致本区域内产生很多小型正断层。这些正断层的大致落差在 8-12 米，对矿区的生产以及维护造成影响。
- 3) 断层特征：对于同一断层，不同部位的落差以及断距不尽相同；同一断层自上而下其断距不断变大；断层在垂直方向上运动明显，在水平方向上运动不明显，并且由于构造运动的影响不同部位的倾向也有区别。
- 4) 构造成因：本地区的地质构造形成于燕山期，喜马拉雅运动的产生导致本区地质构造更加活跃。

在本区域内，南北向和北东向的构造线占据了主导地位，这些构造线起源于燕山期。西部的 F1、F15、F27、F26 等断层以及东部的 F2-1、F2 断层，分别构成了井田西部和东部的边界。

邢台矿的地质构造特征显著，其地表完全为第四系的松散沉积物所掩盖，形

成了典型的沉积覆盖地貌。第四系地层与下伏地层之间的接触关系表现为角度不整合，这一地质现象揭示了该地区复杂的地质历史演化过程。通过详细的钻孔数据分析及对已揭露地层的深入研究，该区域的地层从上到下依次为：奥陶系中统峰峰组（ O_2f ）、石炭系中统本溪组（ C_2b ）、石炭系上统太原组（ C_3t ）、二叠系下统山西组（ P_{1s} ）、二叠系下统下石盒子组（ P_{1x} ）二叠系上统上石盒子组（ P_{2s} ）、新生界第四系（ Q ）。

邢台矿钻孔揭露的地层主要是中奥陶统以上地层。分述如下：

①奥陶系（ O ）：

该区域主要岩石类型为黄灰色、深灰色和灰色厚层状的石灰岩，某些地层中包含白云质石灰岩，特征为鲕粒状。根据勘测资料显示本区地层的厚度在 590 米-810 米，本矿区通过对已揭露地层的深入研究，发现其揭露的岩层厚度在 0.15 米至 235 米之间。

②石炭系中统本溪组（ C_2b ）：

该组地层底部由黑褐色的铝土岩组成，其中包括菱铁矿颗粒和结核。地层的上部是由黑灰色的泥岩所构成，此组地层在沉积时水力条件稳定，所以形成水平层理。根据勘测资料显示整个区域厚度在 19 米至 26 米之间，通过钻孔揭露的地层厚度大致在 1.60 米至 25.70 米之间。该组地层与下方地层形成平行不整合接触。

③石炭系上统太原组（ C_3t ）：

该组地层由深灰色和灰色的泥岩、粉砂岩、灰色的砂岩、灰白色砂岩以及四层灰色至深灰色的石灰岩。以 9#煤为主，该层为本组主要的可采煤层。整个区域的这一地层厚度大约为 150 米，通过钻孔揭露的厚度在 117 米至 160 米之间。与下方地层呈现整合接触。

④二叠系下统（ P_1 ）：

I) 山西组（ P_{1s} ）：本组地层主要由黑灰色粉砂岩，以及黑灰色至灰色的砂岩构成，本组地层含有几层煤层，其中 2#煤是本组的主要可采煤层。通过详细的钻孔数据分析及对已揭露地层的深入研究，本组揭露的地层厚度在 39 米至 85 米之间，平均厚度为 60 米左右。

II) 下石盒子组（ P_{1x} ）：本组地层主要由灰黄色的细粒砂岩以及灰黑色的泥岩组成，其中含有少量的铝土质泥岩。这些泥岩含有为斑状结构。通过勘探资料可知整个区域该组地层厚度大约在 170 米至 210 米之间，通过详细的钻孔数据分析及对已揭露地层的深入研究，本组揭露的地层厚度在 40 米至 120 米之间。该组地层与下方地层呈现整合接触。

⑤二叠系上统（ P_2 ）：

I) 上石盒子组（ P_{2s} ）：该组地层主要由灰色以及黄绿色的中粒砂岩以及黑

灰色的泥岩构成。在底部，地层以灰黄色粗粒砂岩为标志，与下石盒子组形成分界面。而在顶部，地层以黑紫色的细粒砂岩为界线，与石千峰组进行区分。

II) 二叠系上统石千峰组 (P_2sh)：本组地层主要由暗紫色砂岩以及黑灰色泥岩所构成。通过勘探资料可知整个区域的这组地层厚度大致为 245 米左右。

⑥第四系 (Q)：

该区域主要由松散沉积物所构成，其中卵砾石层发育良好，展现出独特的地貌和沉积特征。这些沉积物对于研究该地区的地质历史、地貌演变以及沉积环境具有重要意义。整个区域的这一地层厚度在 110 米至 195 米之间，通过钻孔揭露的厚度则在 12.80 米至 288.50 米之间，平均厚度为 189.74 米。与下方的各个地层呈现不整合接触。

4.2 应用实例

应用实例一：

本文以邢台某矿为研究背景，以三维巷道模型（图 4.1）与 22 层地层模型（图 4.2）为对象进行布尔求交运算。其中地层模型中包括断层、陷落柱等。

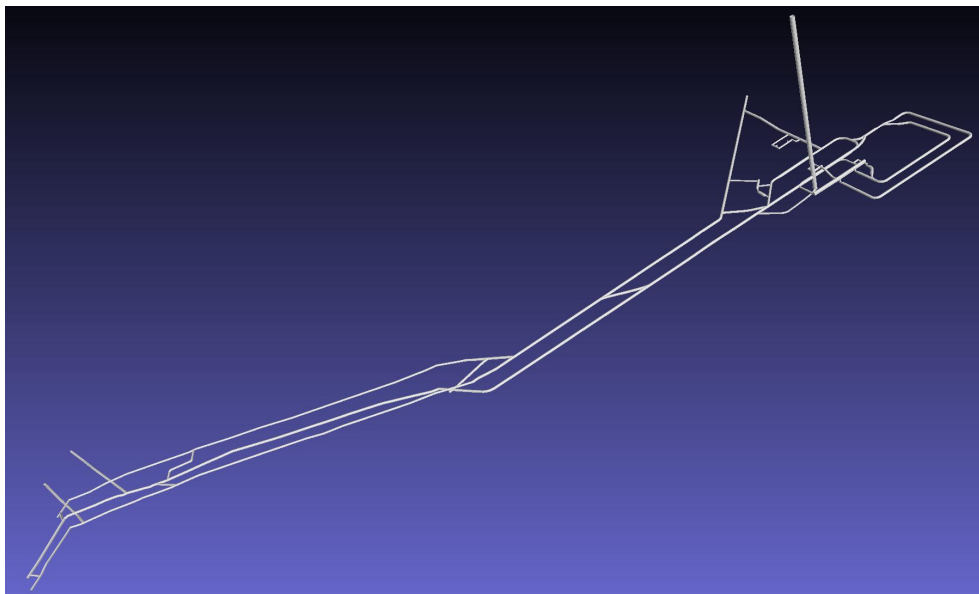


图 4.1 三维巷道模型

Fig. 4.1 Three-dimensional roadway model

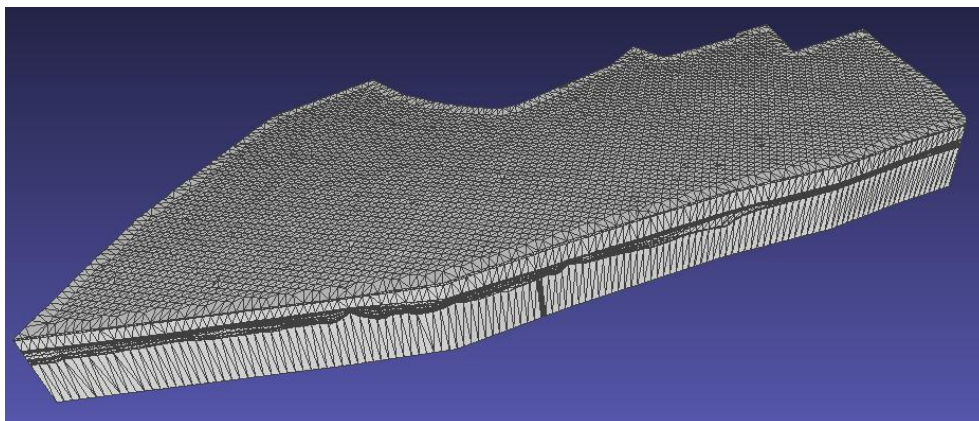


图 4.2 总体三维地层模型

Fig. 4.2 Overall 3D stratigraphic model

在进行地层建模时是按照含水层、隔水层、煤层分别建立的，所以本文把 22 层地层分为三类，即含水层、隔水层以及煤层，分别对它们进行介绍。

（1）含水层

含水层是指在地质学中位于土壤通气层以下的部分，其中物质的孔隙被水分所填满。含水层不仅可以储存一定量的水，而且还具有一定的水流通过能力。它们可以是多种不同类型岩石或土壤组成，包括空隙细小的粘土层（主要含结合水）、空隙较大的砂砾石层（主要含重力水）。

根据含水介质和孔隙类型，邢台矿自上到下划分为九个主要含水层：第四系顶砾孔隙含水层（I）、第四系底砾孔隙含水层（II）、石盒子组砂岩裂隙含水层（III）、大煤顶板砂岩裂隙含水层（IV）、野青灰岩裂隙岩溶含水层（V）、伏青灰岩裂隙岩溶含水层（VI）、大青灰岩裂隙岩溶含水层（VII）、本溪灰岩裂隙岩溶含水层（VIII）和奥陶系灰岩岩溶裂隙含水层（IX）。

（2）隔水层

隔水层（aquitard）是指透水性能较差的岩层或土层，它们能够阻止水分的透过或流动。隔水层的作用主要是隔断毛细水，防止地下水向上渗透到地面基层。隔水层可以由透水性良好的材料或不透水材料组成，用以在基层与土基之间铺设的垫层。隔水层通常包括孔隙极细小的土层和致密岩层，尽管这些岩层可能含有一定量的水分，甚至具有极微弱的透水能力。隔水层的确定通常是以岩层的渗透性大小为划分标准，例如黏性土、由黏粒和粉粒组成的坚硬岩类（如页岩、黏土岩、板岩），以及裂隙极不发育的块状岩类（如结晶岩、变质岩，但碳酸盐岩类变质岩除外）都被视为隔水层。

在邢台矿井田内，各个含水层之间分布着厚度不均的隔水层，这些隔水层主要由中粉砂岩、泥岩、铝土岩以及不同厚度的岩浆岩侵入体构成。正常情况下，这些岩层能够有效地隔离水分。隔水层以及含水层主要特征见表 4.1。

表 4.1 主要隔水层及含水层特征一览表

Table 4.1 List of main aquifer and aquifer features

含隔水层名称	厚度 (m)	岩性描述
第四纪顶部砾石含水层 (I)	15~20	为石英质砾石, 粗粒石英砂和砂土、砂组成, 砾径在 0.6~0.1m 之间。
隔水层 1	100~190	粘土、砂质粘土, 分布连续、稳定, 隔水性一般较好。
第四纪底部砾石含水层 (II)	0.52~15.96	红色石英岩、砂和粘土组成。
隔水层 2	1.54~28.95	粘土、砂质粘土, 分布连续、稳定, 隔水性一般较好。
石盒子组砂岩含水层 (III)	8.89~29.41	灰白色中粗砂岩或白色中粒砂岩, 泥质胶结, 裂隙不发育。
隔水层 3	10~30	致密薄层泥岩、砂质泥岩。
大煤顶板砂岩含水层 (IV)	0~21	灰白和白色中细粒及中粒砂岩, 含线性层理, 泥质胶结, 裂隙不发育。
2#煤	2.42~9.46	
隔水层 4	34~50	砂质泥岩
野青灰岩含水层 (V)	0.2~4.42	灰色至青灰色质地, 含大量珊瑚等化石, 区内非常稳定。
隔水层 5	12.32	砂岩、砂质页岩, 含黄铁矿结核。
5#煤	0~2.77	
隔水层 6	28.36	砂岩、砂质页岩和煤层组成, 岩层较稳定。
伏青灰岩含水层 (VI)	0.20~3.85	青灰色, 质较纯, 有时还有燧石结核。
隔水层 7	23~44	致密薄层泥岩和薄层砂岩, 隔水性良好。
大青灰岩含水层 (VII)	2.14~10.52	深灰色, 含数层燧石薄层, 裂隙发育。
隔水层 8	4~26.15	砂、泥岩隔水层。
本溪灰岩含水层 (VIII)	1.5~8.21	灰色岩石比较质密均一, 个别地段含硅质结核。
隔水层 9	5~12	致密铝质泥岩, 分布连续稳定, 隔水性能良好。
奥陶系灰岩含水层 (IX)	600~800	青灰色, 岩性, 质纯质密, 中下部有豹皮状呢灰岩及薄层泥灰岩, 裂隙发育。

（3）煤层

模型中主要包含 2#煤层、5#煤层以及 9#煤层。2#煤层的直接顶板通常是灰黑色的粉砂岩，但在某些地区，由于河流的冲刷作用，直接顶板会变成中粒到粗粒的石英砂岩。该煤层的底板一般是深灰色的泥岩和砂质泥岩，岩性变化较为显著。5#煤层的顶板由黑色粉砂岩和泥岩组成，底板通常为深黑色泥岩，岩性变化不明显。9#煤层的顶板是灰色粉砂岩，底板为深灰色的砂质泥岩和粉砂岩，岩性变化不明显。

根据每层地层的岩性描述，为每层地层附上相应的颜色。如图 4.3 含水层、隔水层、煤层相对位置关系图所示。图 4.4 为带颜色的三维地层模型。

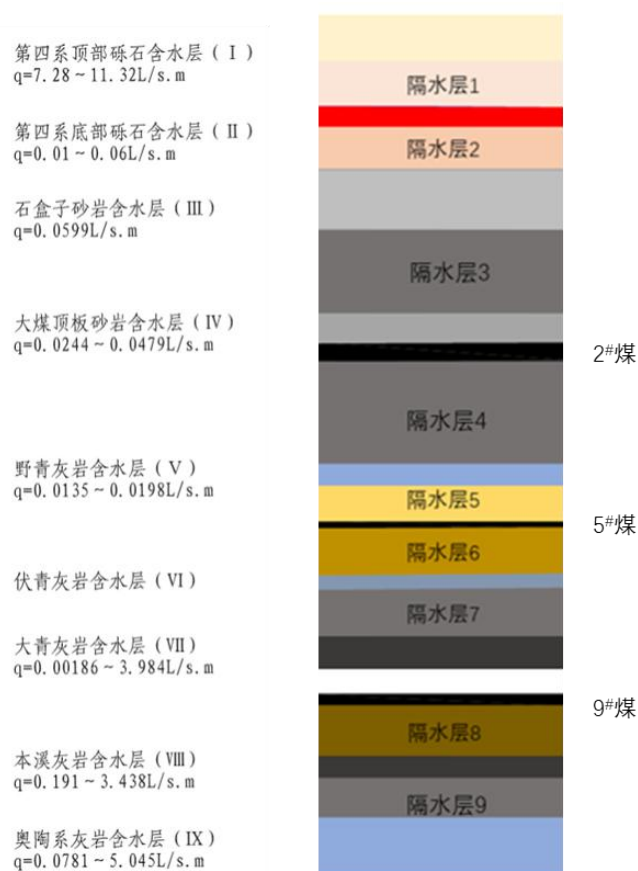


图 4.3 含水层、隔水层、煤层相对位置关系图

Fig. 4.3 Aquifer, aquifer and coal seam relative position relation diagram

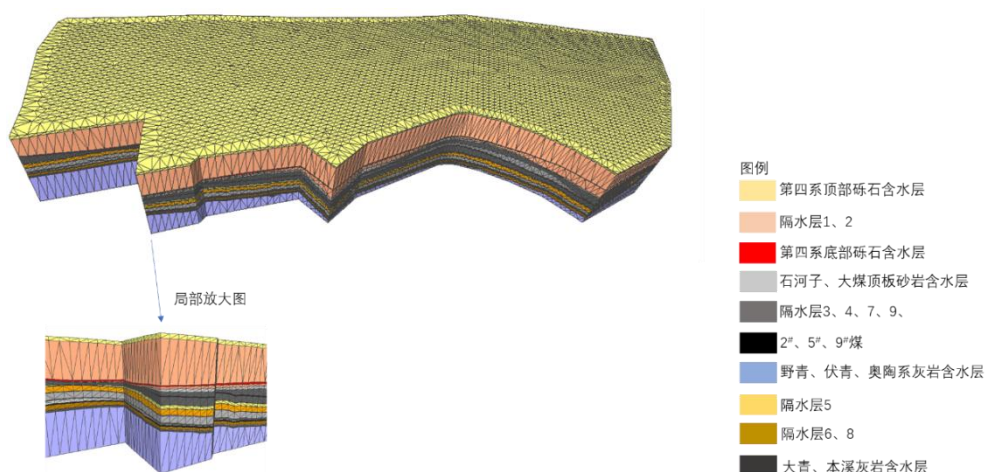


图 4.4 带颜色的三维地层模型图

Fig. 4.4 Color 3D stratigraphic model diagram

为了直观区分巷道的不同部位都经过哪些地层，下面分别进行巷道与含水层、巷道与隔水层、巷道与煤层的布尔求交运算。之后根据地层岩性的不同为巷道交集部分附上地层的颜色，这种技术为巷道岩性贴图，本文运用可视化软件 Meshlab 对巷道进行贴图上色，巷道岩性贴图就是在三维巷道模型以及地层模型的基础上展开的。主要涉及两方面技术：1：首先用每层地层和巷道进行布尔求交运算，得出每层地层与巷道的交集。2：根据地层岩性的不同对每层地层与巷道的交集进行相应的岩性贴图。巷道岩性贴图是一种用于描述巷道周围岩石或土壤的地质特征的图示方法。巷道岩性贴图的主要作用是：1：提高设计的真实感，通过模拟岩石的纹理和颜色，可以使巷道模型看起来更加真实^[71-73]。2：辅助支护设计，不同的岩石性质需要不同的支护方法，岩性贴图可以帮助工程师们直观地识别岩石类型，从而进行合理的支护设计^[74]。3：风险评估，通过分析贴图，可以预测巷道在开采过程中可能出现的稳定性问题，如岩石破碎、塌陷等^[75-76]。4：施工指导，在施工过程中，岩性贴图可以作为参考，帮助施工人员了解即将遇到的地质条件，提前做好施工准备^[77-78]。

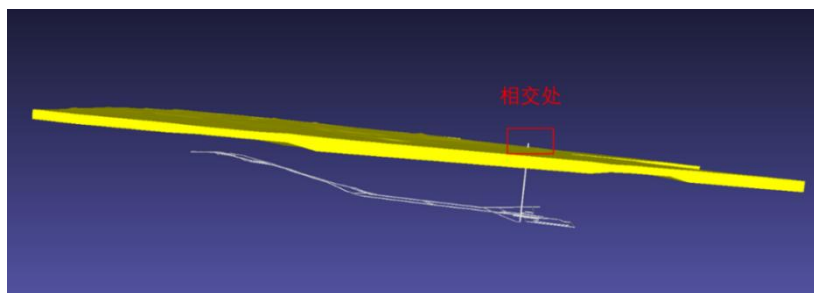
这种贴图技术广泛应用于采矿、隧道工程和地质勘探等地下环境领域，使工程师和地质学家能够更清晰地了解地下环境的地质条件。通过对各种岩性特征进行详细描述和分析，巷道岩性贴图能够帮助工程师评估巷道工程的难度和风险，进而制定出更为合理的设计方案和施工措施。借助数字化和可视化技术的应用，能够大幅度减少人工操作^[79]，提升工作效率和管理水平，实现矿山的智能化能源生产和智能管理^[80-82]。

（1）巷道与含水层布尔求交运算

由图 4.3 所知，在 22 层地层中，第 I、II、III、IV、V、VI、VII、VIII、IX 层为含水层，分别和巷道进行布尔求交运算并得到相对应结果。以巷道与第 I、V 层

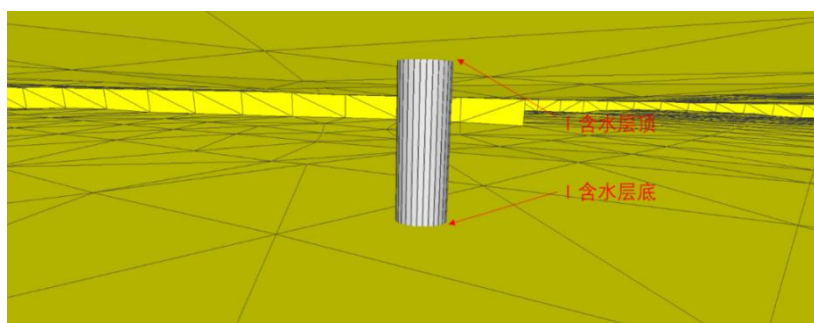
含水层相交为例，可得以下结果。

其中，在图 4.5（a）中，黄色模型代表I含水层，白色模型代表巷道模型。红色方框中的部分代表两模型相交处。图 4.5（b）中，巷道模型分别与I含水层顶以及I含水层底相交，根据布尔运算得出两层之间的巷道部分即是两模型的交集结果，此交集结果为巷道竖井的一部分，如图 4.5（c）所示。



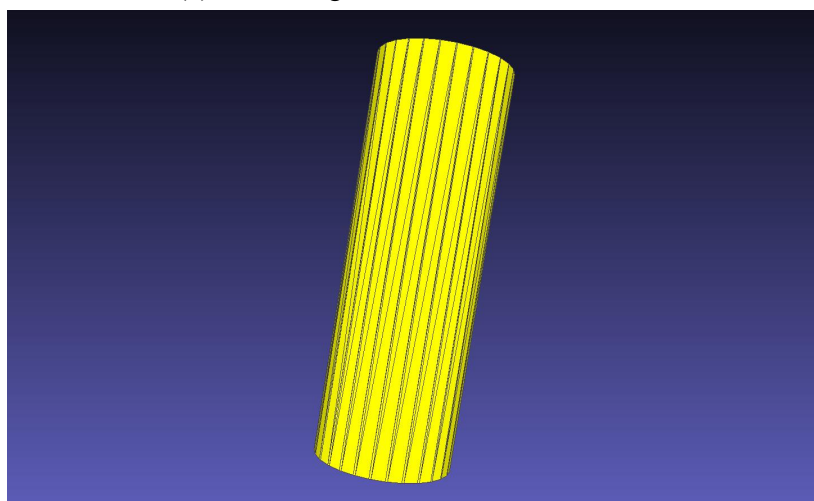
（a）巷道与I含水层相交图

(a) Diagram of intersection between roadway and aquifer I



（b）相交处局部放大图

(b) Local magnification at the intersection



（c）相交结果图

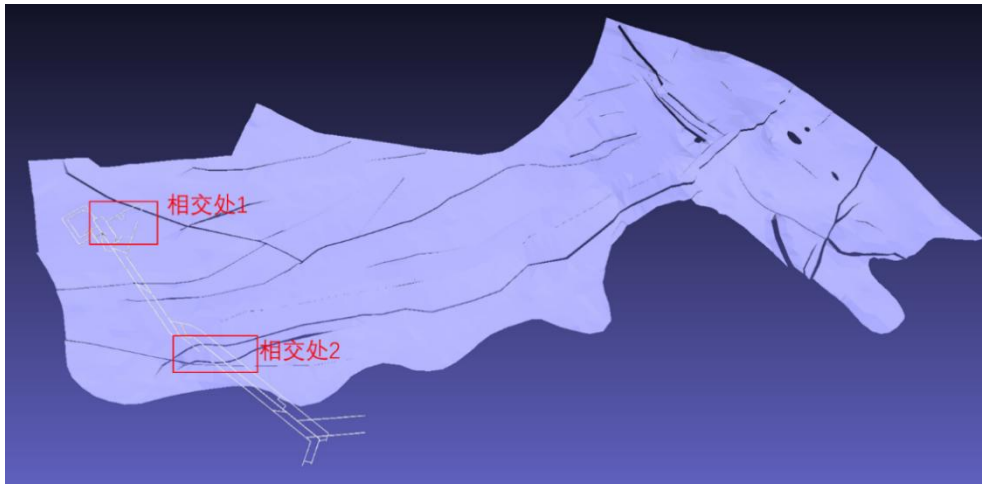
(c) Intersecting result graph

图 4.5 巷道与I含水层布尔求交运算

Fig. 4.5 Boolean intersection of roadway and aquifer I

在图 4.6（a）V含水层与巷道相交图中，青灰色代表V含水层模型，白色代表

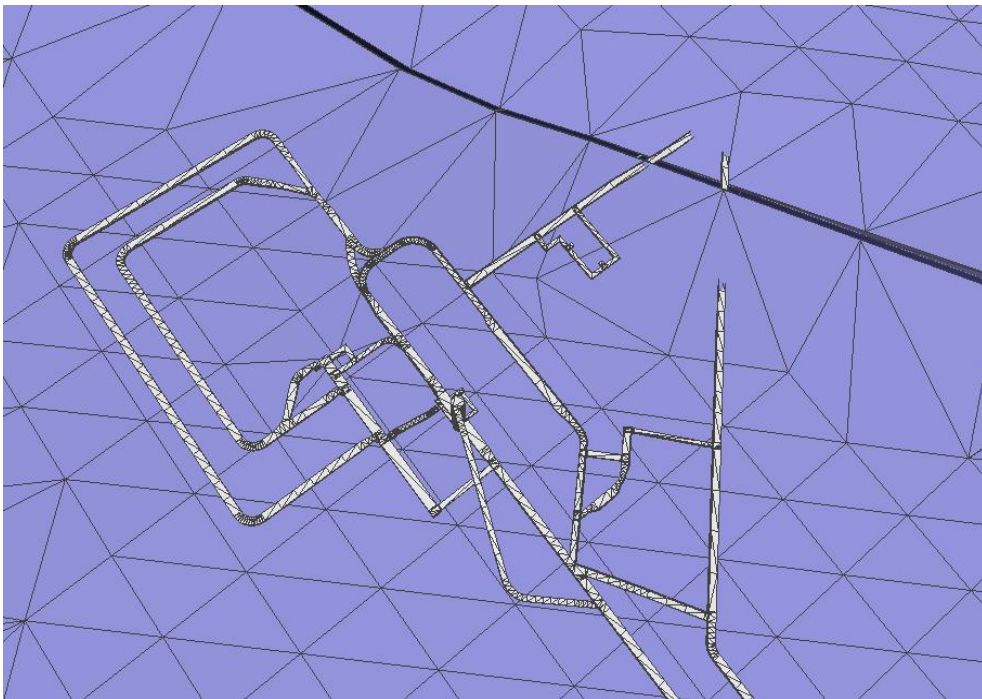
巷道模型，其中巷道与V含水层共有两部分相交，分为相交处1以及相交处2。



(a) 巷道与V含水层相交图

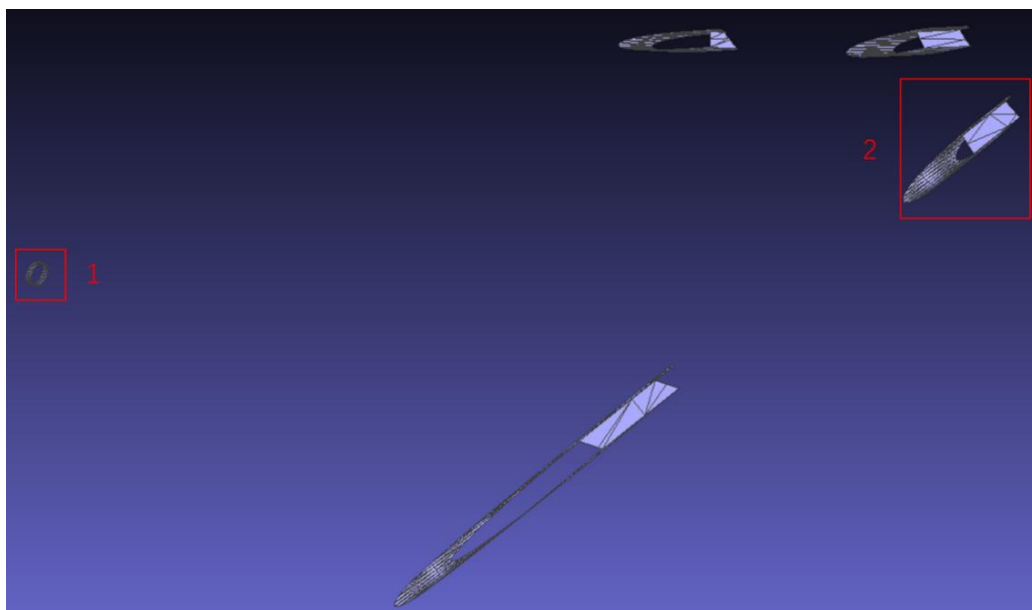
(a) Diagram of intersection of roadway and aquifer V

其中相交处1的局部放大图如图4.6(b)所示，图4.6(c)表示巷道与V含水层在相交处1的布尔运算交集结果，为了观察布尔运算结果的细节，把图4.6(c)中的1、2两处进行放大，得到图4.6(d)、(e)。



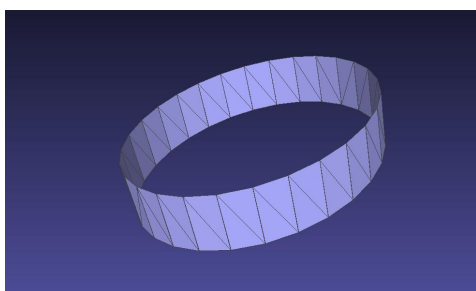
(b) 相交处1局部放大图

(b) Intersection 1 Partial magnification



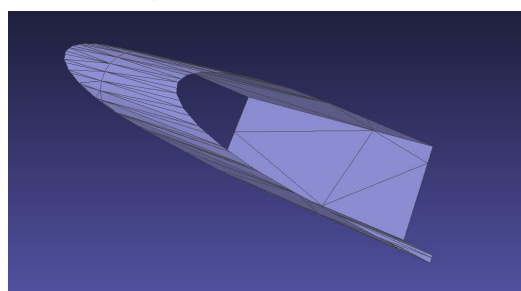
(c) 相交处 1 结果图

(c) Intersection 1 Result diagram



(d) 1 放大

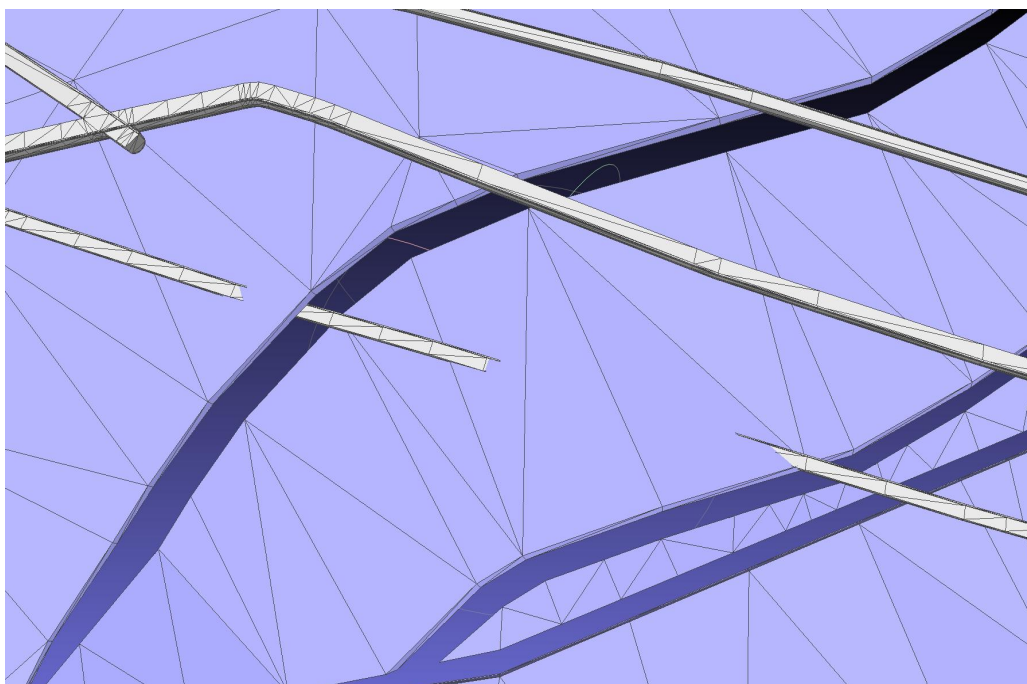
(d) 1 zoom in



(e) 2 放大

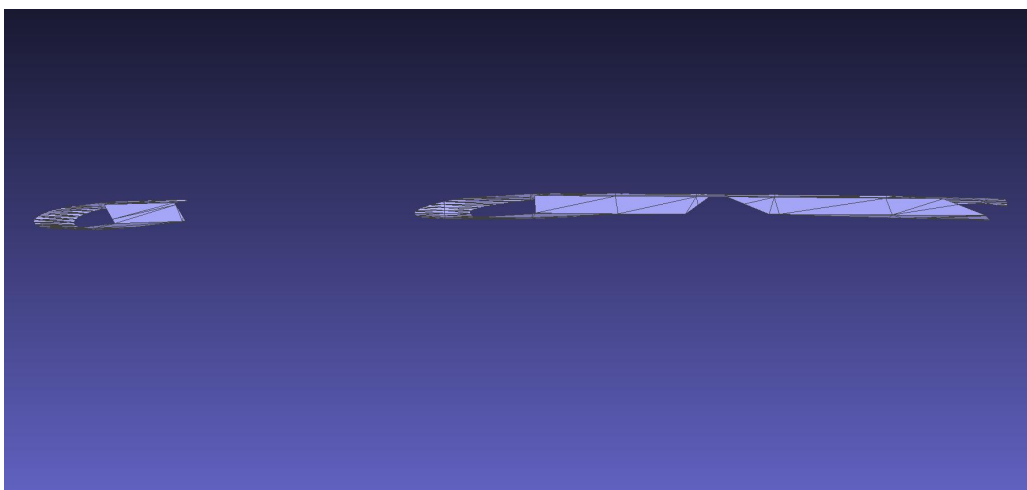
(e) 2 zoom in

相交处 2 的局部放大图如图 4.6 (f) 所示，其中共有两处相交部分，结果如图 4.6 (g) 所示。



(f) 相交处 2 局部放大图

(f) Intersection 2 partial magnification



(g) 相交处 2 结果

(g) Intersection 2 results

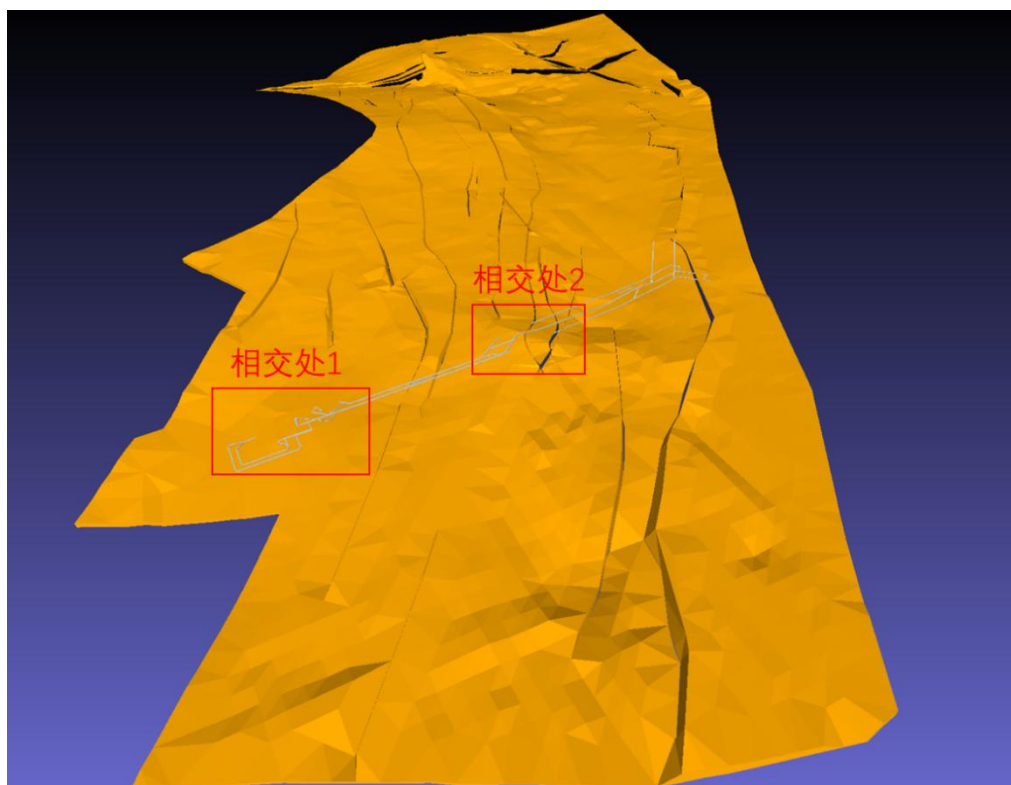
图 4.6 巷道与V含水层布尔求交运算

Fig. 4.6 Boolean intersection of roadway and aquifer V

(2) 巷道与隔水层布尔求交运算

由图 4.3 所知，在 22 层地层中，隔水层共有 9 层，分为隔水层 1、隔水层 2……隔水层 9，分别和巷道进行布尔求交运算并得到相对应结果。以巷道与隔水层 5 相交为例，可得以下结果。

在图 4.7 (a) 隔水层 5 与巷道相交图中，土黄色代表隔水层 5 模型，白色代表巷道模型，其中巷道与隔水层 5 共有两部分相交，分为相交处 1 以及相交处 2。



(a) 巷道与隔水层 5 相交图

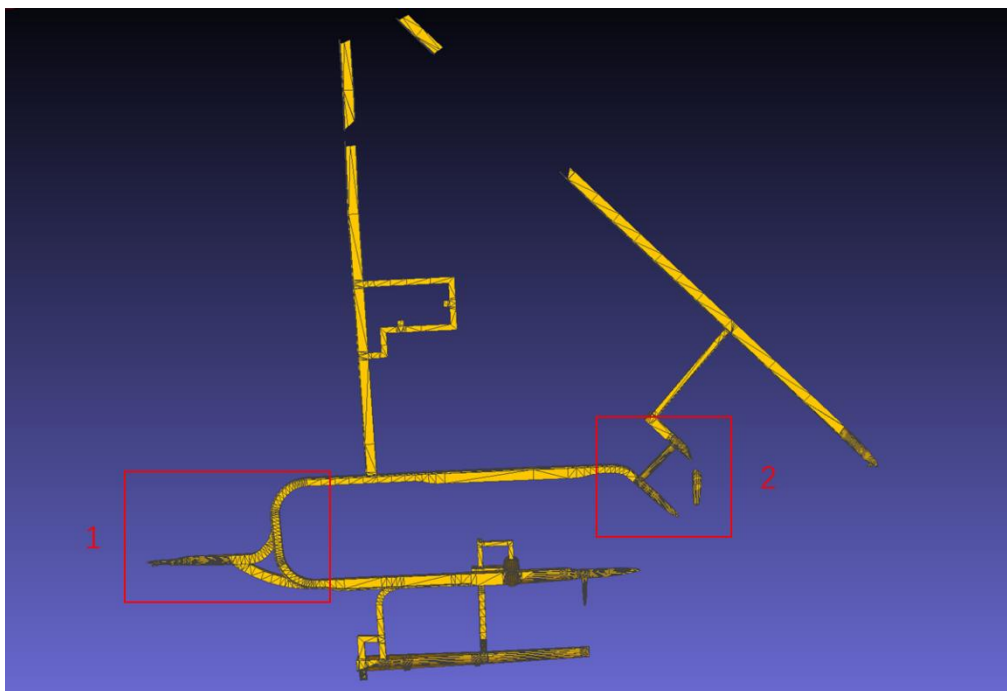
(a) Intersection diagram of roadway and aqueduct 5

其中相交处 1 的局部放大图如图 4.7 (b) 所示，图 4.7 (c) 表示巷道与隔水层 5 在相交处 1 的布尔运算交集结果，为了观察布尔运算结果的细节，把图 4.7 (c) 中的 1、2 两处进行放大，得到 (d)、(e)。



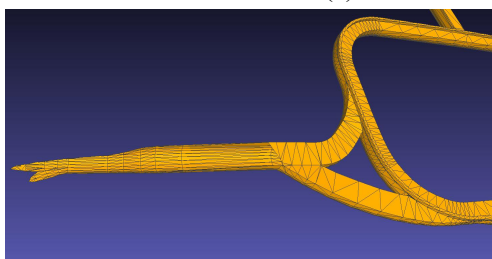
(b) 相交处 1 局部放大图

(b) Intersection 1 Partial magnification



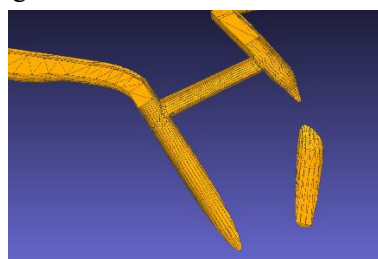
(c) 相交处 1 结果图

(c) Intersection 1 Result diagram



(d) 1 放大

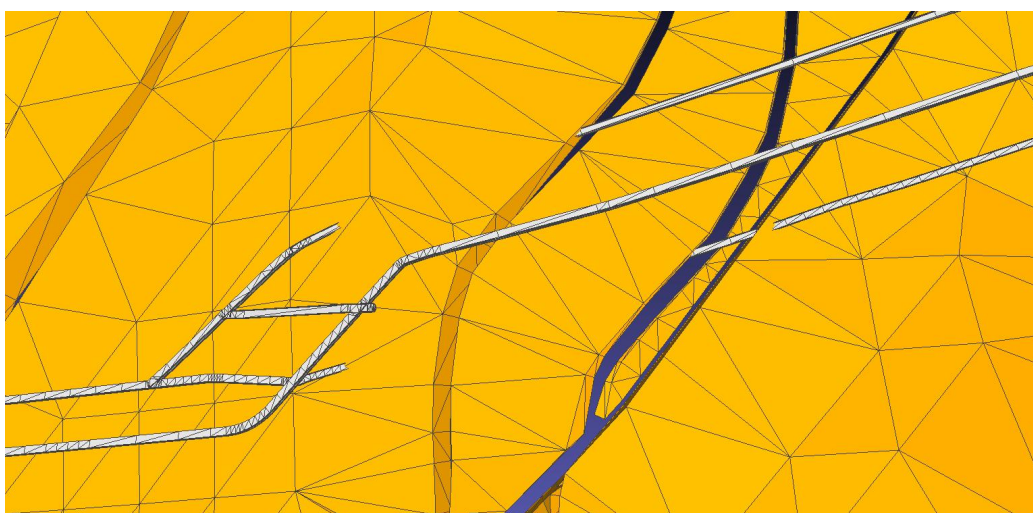
(d) 1 zoom in



(e) 2 放大

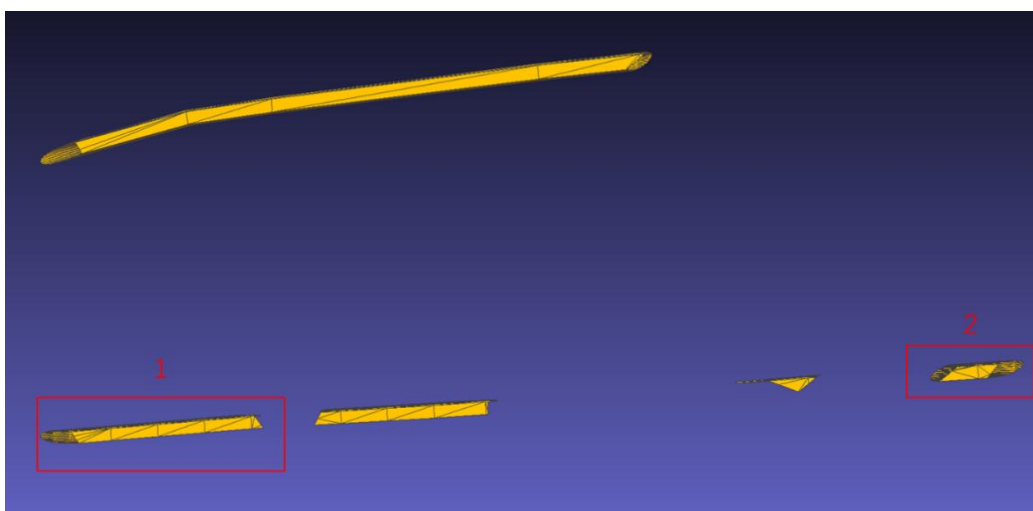
(e) 2 zoom in

图 4.7 (f) 表示图 4.7 (a) 中相交处 2 的局部放大图，图 4.7 (g) 表示巷道与隔水层 5 在相交处 2 的布尔运算交集结果，为了观察布尔运算结果的细节，把图 4.7 (g) 中的 1、2 两处进行放大，得到图 4.7 (h)、(i)。



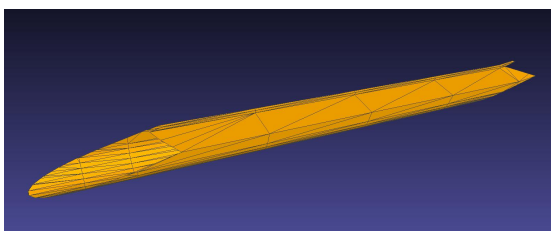
(f) 相交处 2 局部放大图

(f) Intersection 2 partial magnification



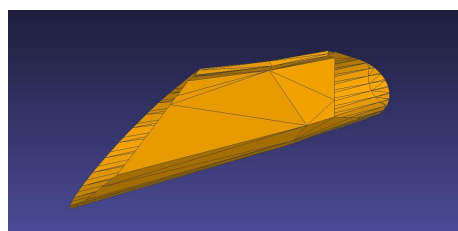
(g) 相交处 2 结果图

(g) Intersection 2 result diagram



(h) 1 放大

(h) 1 zoom in



(i) 2 放大

(i) 2 zoom in

图 4.7 巷道与隔水层 5 布尔求交运算

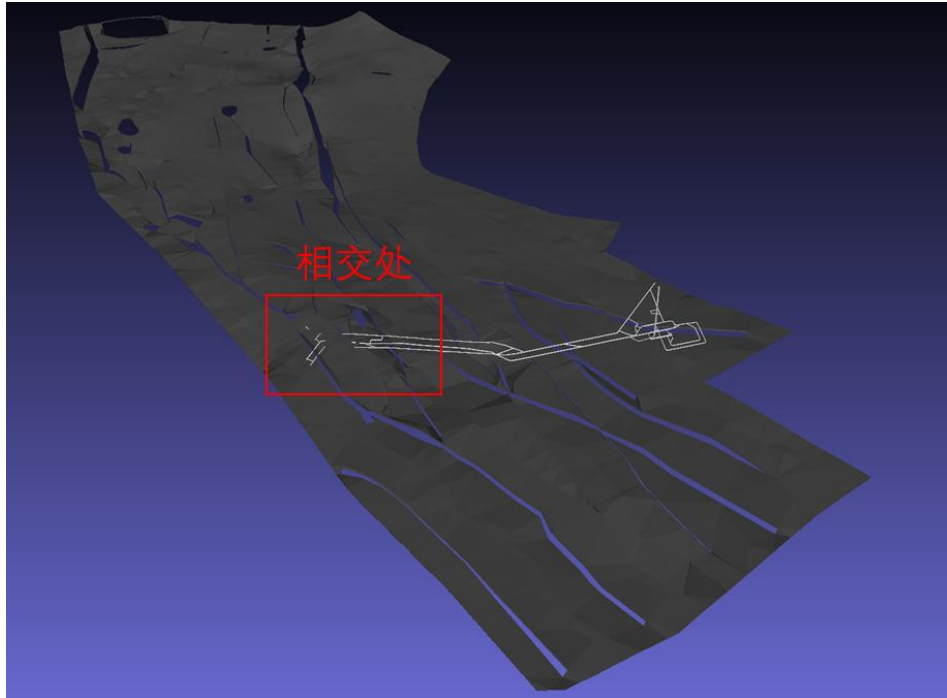
Fig. 4.7 Boolean intersection operation between roadway and barrier 5

(3) 巷道与煤层布尔求交运算

模型中主要包含 2#煤层、5#煤层以及 9#煤层。以巷道与 9#煤相交为例，可得

如下的结果。

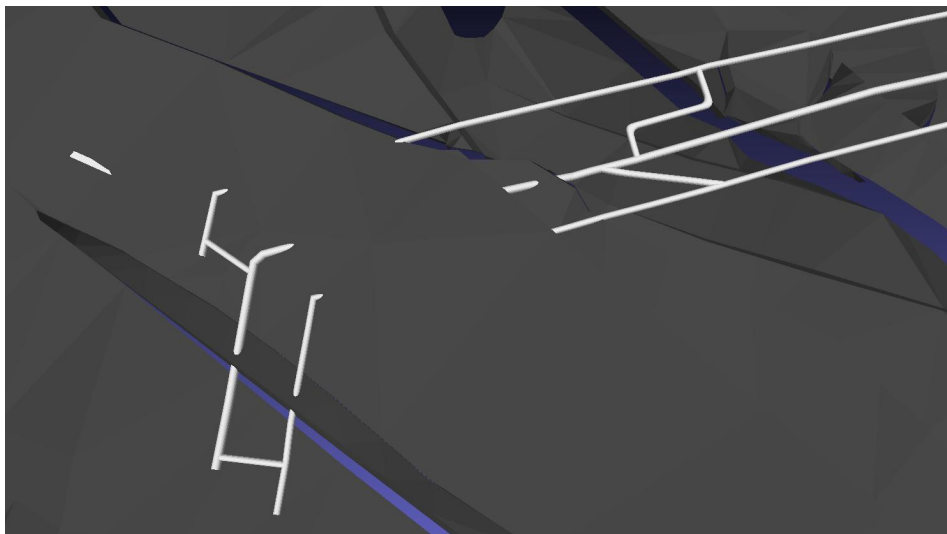
如下图 4.8 (a) 的 9#煤与巷道相交图中，白色代表三维巷道模型，黑灰色代表 9#煤地层，红色框中代表巷道与 9#煤相交的部分。



(a) 巷道与 9#煤相交图

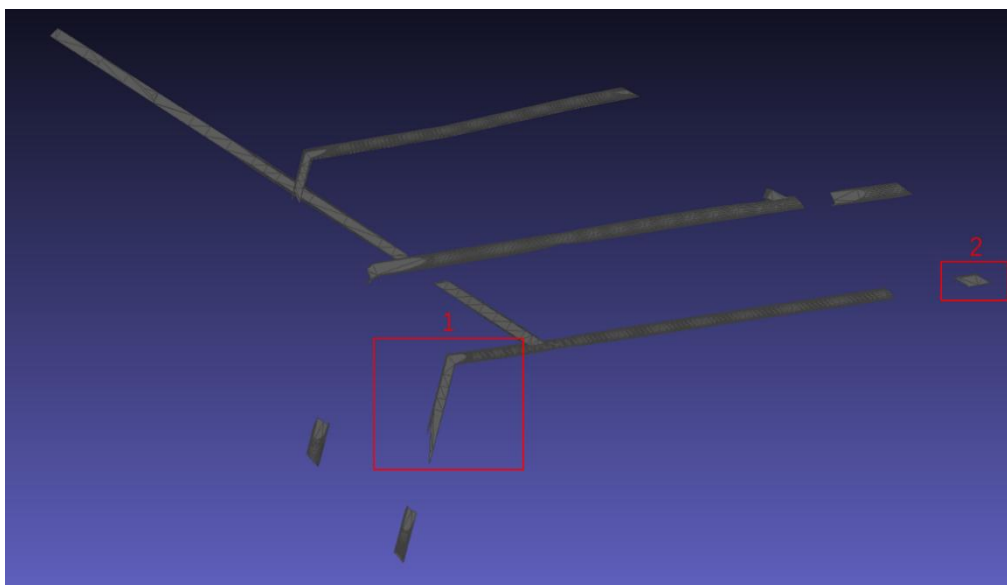
(a) Diagram of intersection of roadway and 9# coal

图 4.8 (b) 表示图 4.8 (a) 中相交处的局部放大图，巷道模型与 9#煤进行布尔求交运算，得到结果如图 4.8 (c)，为了观察布尔运算结果的细节，把图 4.8 (c) 中的 1、2 两处进行放大，得到图 4.8 (d)、(e)。



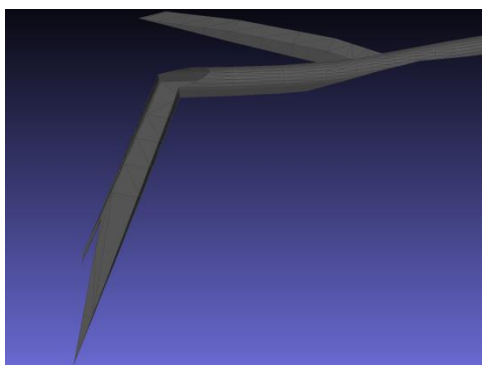
(b) 相交处局部放大图

(b) Local magnification at the intersection



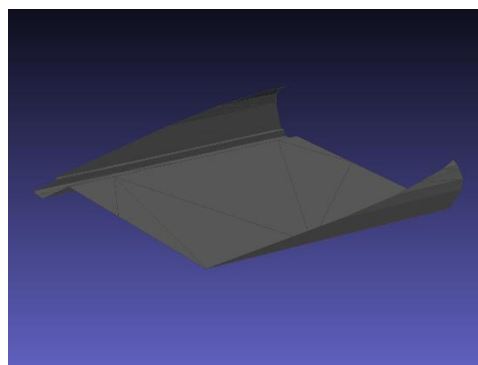
(c) 相交处结果图

(c) The result diagram at the intersection



(d) 1 放大

(d) 1 zoom in



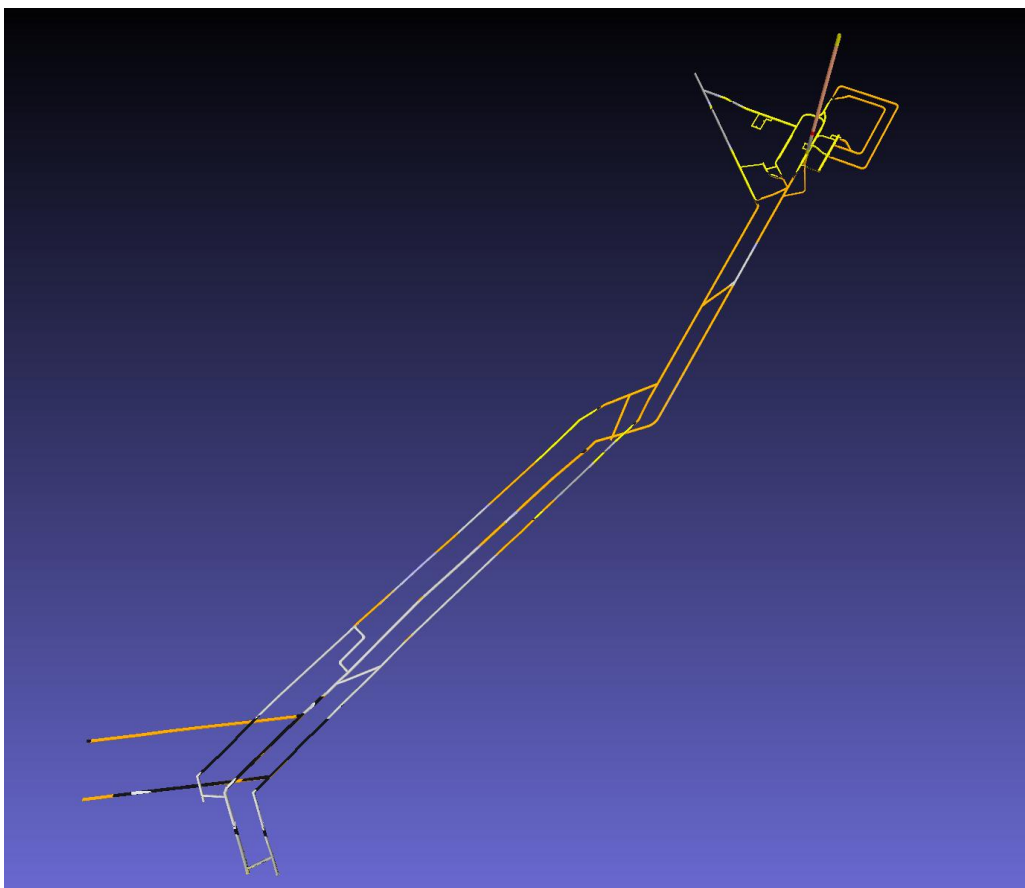
(e) 2 放大

(e) 2 zoom in

图 4.8 巷道与 9#煤布尔求交运算

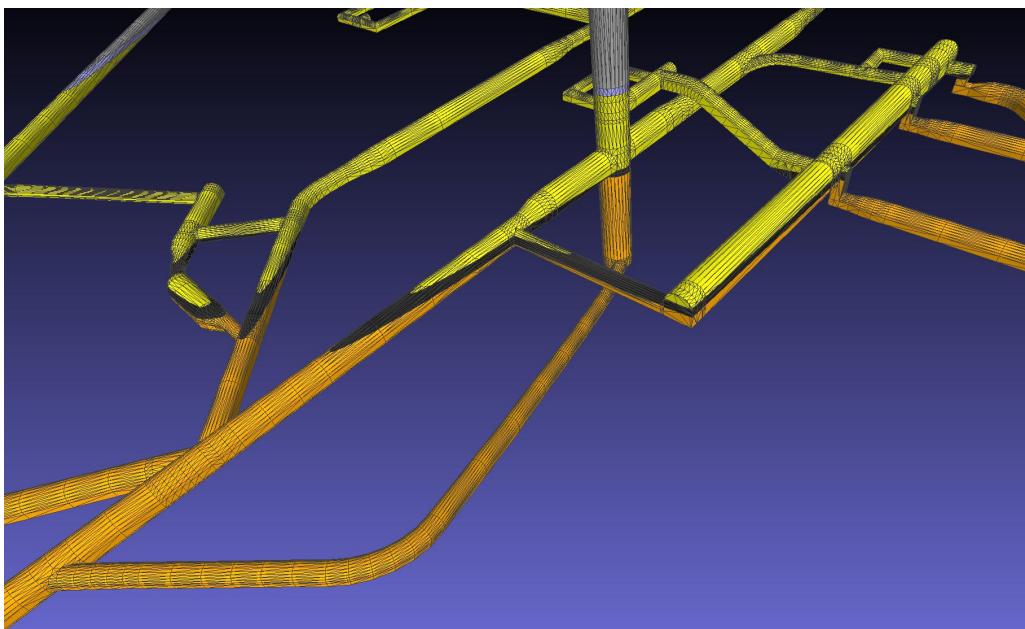
Fig. 4.8 The intersection of roadway and 9# coal Boolean

巷道分别与含水层、隔水层、煤层进行布尔求交之后，对求交之后的结果进行组合，得到图 4.9 (a)，为巷道和所有地层求交并附上相应地层颜色的总图。图 4.9 (b) 以及图 4.9 (d) 为局部结果图。图 4.9 (c) 为细节放大图。



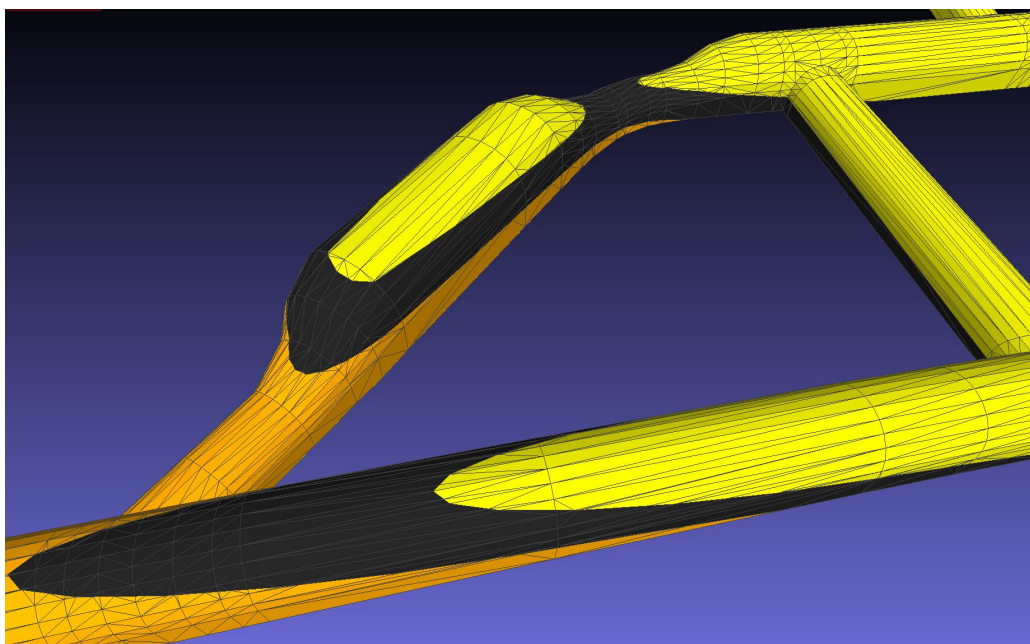
(a) 布尔求交运算结果总图

(a) Summary of Boolean quadrature results



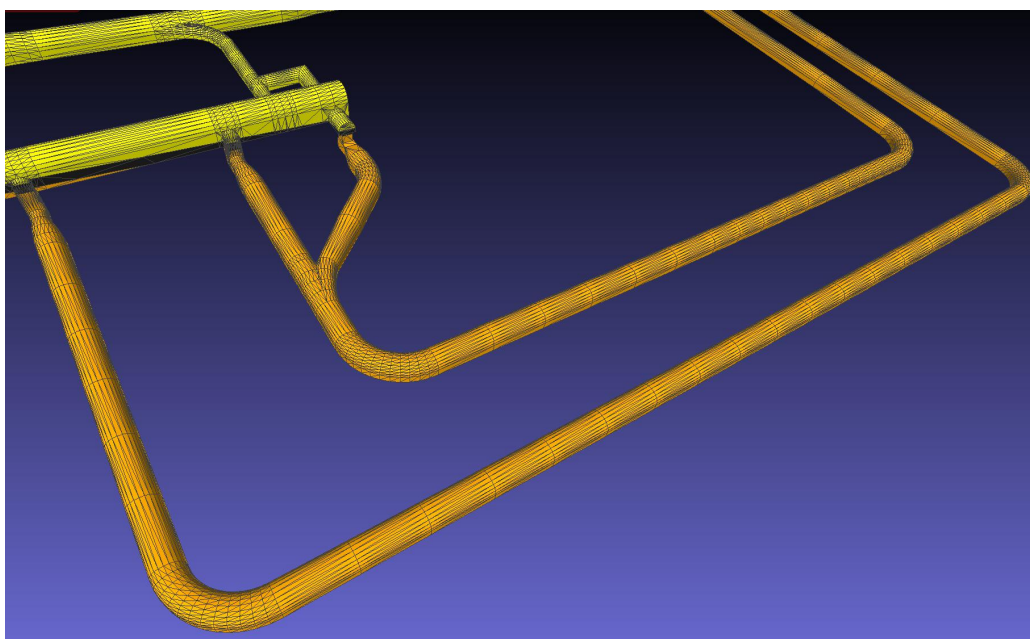
(b) 布尔求交运算局部结果图 1

(b) The local result of Boolean operation is shown in Figure 1



(c) 细节放大图

(c) Detail enlargement



(d) 布尔求交运算局部结果图 2

(d) The local result of the Boolean operation is shown in Figure 2

图 4.9 详细结果图

Fig. 4.9 Detailed result chart

由上图 4.9 (c) 可知算法在细节处理上表现良好，模型重构效果较好，没有出现缝隙或重叠等现象，并可以根据地层岩性的不同为三维巷道进行岩性划分并设置相应的颜色，实现三维巷道表面岩性划分及可视化。

应用实例二：

在三维地质建模和采矿工程中，布尔差运算常被用来模拟地层开挖的过程。当需要在三维地质模型中模拟地表或地下空间的挖掘行为时，布尔差运算能够帮助用户从原始地质模型中减去代表开挖区域的几何体，以得到开挖后的地质模型。

具体应用中，比如在规划矿山地层开采或地下工程建设时，可通过以下步骤使用布尔差运算：

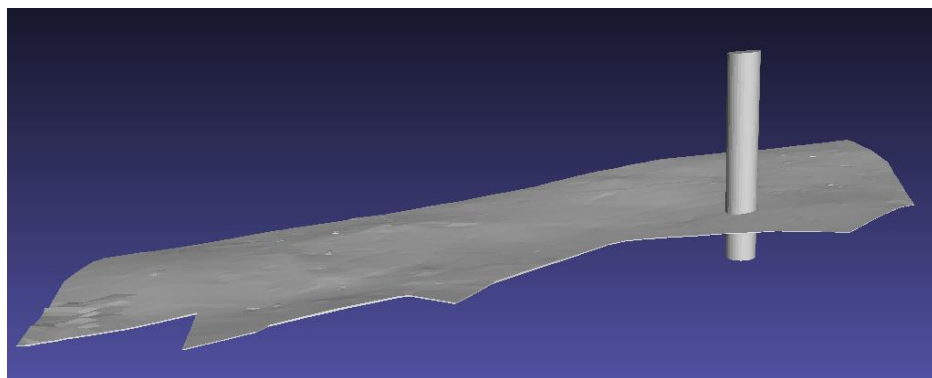
1：首先，建立目标地层的完整三维地质模型，包括各种岩层、断层和其它地质结构。

2：根据开采设计或工程施工方案，构建一个表示开挖轮廓或隧道路径的三维几何模型。

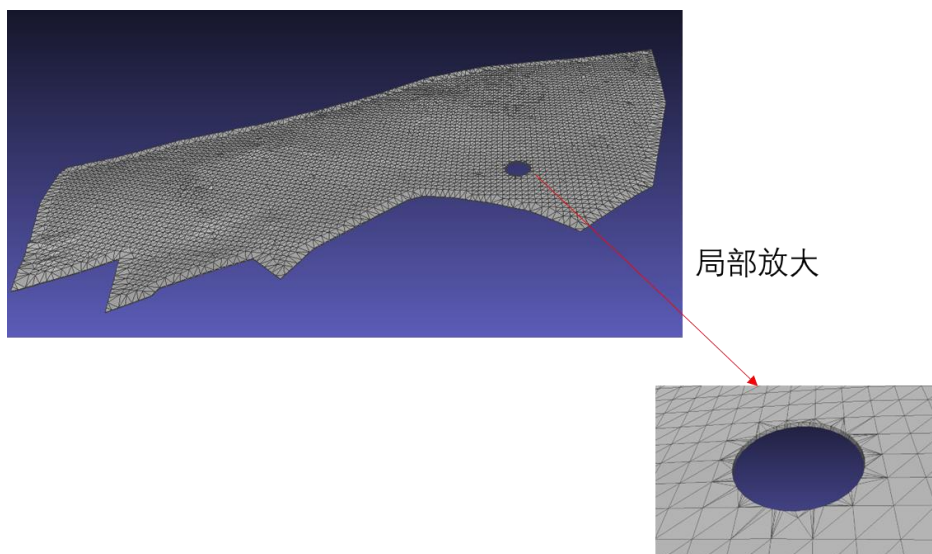
3：使用布尔差运算符，将开挖几何体从原始地质模型中减去，这样就可以得到一个反映开挖后状态的新地质模型。

4：这个新的模型显示了开挖后暴露的岩层结构、预计的空腔形状等信息。

如下图 4.10（a）表示目标地层以及圆柱形的开挖模型，图 4.10（b）表示地层的开挖结果，为了观察开挖后的地层细节，对其进行局部放大。

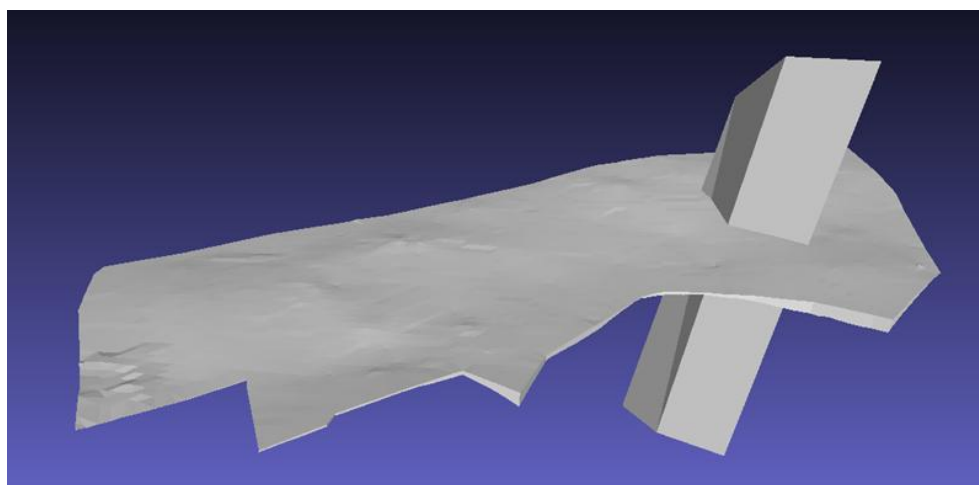


（a）

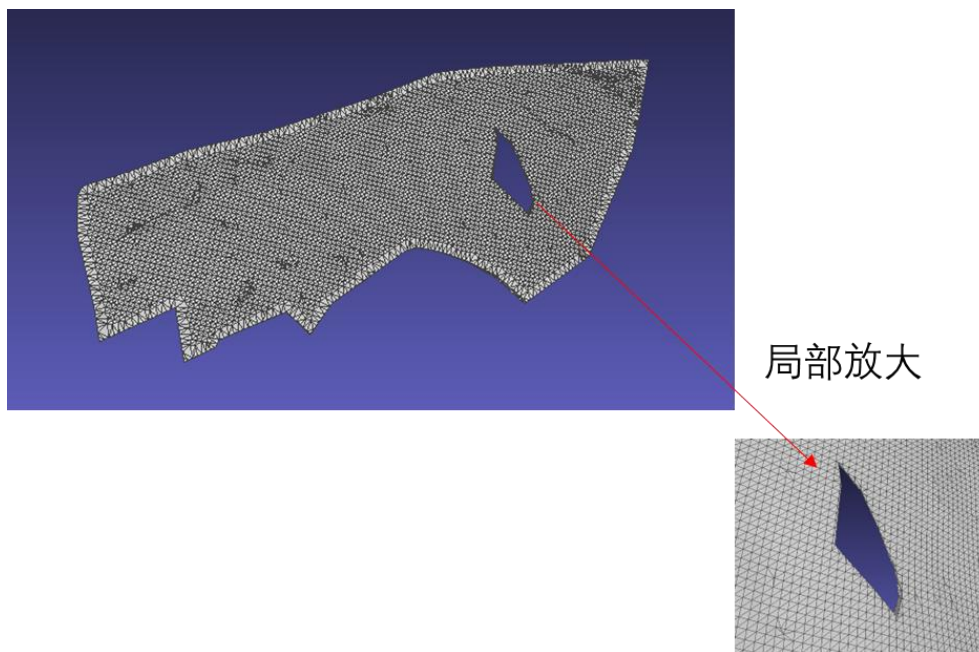


（b）

图 4.10 (c) 表示目标地层以及不规则六面体开挖模型，图 4.10 (d) 表示地层的开挖结果，为了观察开挖后的地层细节，对其进行局部放大。



(c)



(d)

图 4.10 地层开挖

Fig. 4.10 Stratum excavation

通过这种方法，工程师可以清晰地看到开挖作业对地质环境的影响，进而进行安全评估、稳定性分析以及支撑设计等工作，有助于更科学地进行决策和优化工程方案。

本文相关算法全都基于 java 面向对象编程语言实现，开发的工具为 IntelliJ IDEA，用 Meshlab 可视化软件为工具对交集结果进行可视化。

4.3 本章小结

本章主要以巷道与地层布尔运算交集以及对地层求取差集为例，以 Meshlab 可视化软件为工具对交集、差集结果进行可视化分析，可分为以下几点：

（1）把地层分为含水层、隔水层、煤层分别和巷道进行布尔求交运算，根据地层岩性的不同，对布尔运算的结果附上相应的颜色图片。

（2）把带有颜色的巷道交集部分进行重组，得到了含有多多种颜色的完整三维巷道，此巷道不同区域带有不同颜色，每种颜色分别对应相应的地层岩性，实现了通过观察三维巷道表面确定周围岩性的方法。为辅助支护设计、风险评估、施工指导奠定基础。

（3）以布尔差运算为例，对地层进行开挖，获取开挖后暴露的地层结构、预计的空腔形状等信息。

5 结论与展望

5.1 结论

“数字矿山”作为煤矿信息化的重要发展趋势，已经成为煤矿以及地质行业的研究热点。对已有的三维地质模型进行布尔运算以及相关应用在数字矿山领域占有十分重要的地位。因此，本文对三维地质模型之间的布尔运算以及相关应用进行了研究，并取得了以下成果：

（1）对三维地质模型进行导入工作，实现了模型数据结构以及拓扑关系的构建。

（2）对模型的层次包围盒算法进行研究，得到快速检测三角形之间是否发生碰撞的算法。对于三角形对求交产生的一些离散交线段，基于模型本身的拓扑结构，实现离散交线段连接成交线环的方法。

（3）对三角形被交线环切割成多个多边形区域的情况，使用 Ear-Clipping 算法将多边形区域剖分成一系列新的三角形，实现了模型重构算法。针对未被交线环切割的三角形，基于图的广度优先遍历算法（BFS）对这些三角形进行内外区分，把重构完成的三角形以及基于 BFS 算法区分的内外三角形依据布尔运算公式进行重组，实现了三维地质模型中的布尔运算算法。

（4）对三维地质模型布尔运算的结果进行应用分析，把地层分为含水层、隔水层、煤层分别和巷道进行布尔求交运算，根据地层岩性的不同，对布尔运算的结果附上相应的颜色图片，最后把带有颜色的巷道交集部分进行重组，得到了含有多种颜色的完整三维巷道，此巷道不同区域带有不同颜色，每种颜色分别对应相应的地层岩性，实现了通过观察三维巷道表面确定周围岩性的方法。为辅助支护设计、风险评估、施工指导奠定基础。以布尔差运算为基础，对地层进行开挖，获取开挖后暴露的岩层结构、预计的空腔形状等信息，使工程师可以清晰地看到开挖作业对地质环境的影响。

5.2 展望

尽管本文在三维地质模型的布尔运算及应用方面做了相关研究，但由于个人的能力有限，仍然有许多问题需要进一步研究与解决：

（1）在模型进行碰撞检测阶段，本文采用的算法虽然可以提高碰撞检测效率，但是对于包含大量三角形的模型，此算法的效率有提高的空间。

（2）对于布尔运算在其他方面的应用，本文没有对其进行深入探讨，因此，

可以结合矿山的实际生产需求，对布尔运算在其他方面的应用进行分析研究，如运用布尔运算对地层进行剖面、运用布尔运算对巷道进行建模等。

参考文献

- [1] 武玥. 矿山巷道路径搜索及可视化[D]. 西安科技大学, 2014.
- [2] 黄峰. 矿山三维巷道建模关键技术研究[D]. 中南大学, 2014.
- [3] 李凯. 三维巷道建模方法研究与实现[D]. 中南大学, 2012.
- [4] 谭正华, 潘梅, 王李管等. 实测导线数据的巷道系统三维自动建模方法[J]. 测绘科学, 2017, 42(11): 164-170.
- [5] 李彩琴. 东升庙铅锌矿矿井巷道和矿体三维建模与应用研究[D]. 内蒙古科技大学, 2021.
- [6] 贾庆仁, 车德福, 朱继文. 煤矿巷道三维建模方法设计与系统开发应用[J]. 矿山测量, 2018, 46(04): 1-4+28.
- [7] 王聪. 巷道三维自动建模技术及可视化研究[D]. 山东科技大学, 2018.
- [8] 谢景龙, 张得群. 矿山三维巷道的参数化建模及其可视化[J]. 矿山测量, 2017, 45(05): 5-9.
- [9] 石磊. 矿山测量数据处理与三维巷道建模方法分析[J]. 能源与节能, 2017, (05): 42-43.
- [10] 董文耀. 五阳煤矿采矿工程三维巷道建模研究[J]. 山东煤炭科技, 2016, (11): 15-17.
- [11] 赵小平, 刘强. 基于点云数据的三维巷道快速建模研究[J]. 有色金属(矿山部分), 2016, 68(06): 90-93.
- [12] 郭玉东. 井下巷道三维建模及可视化方法研究与实现[D]. 辽宁工程技术大学, 2016.
- [13] 江记洲, 郭甲腾, 吴立新, 等. 基于三维激光扫描点云的矿山巷道三维建模方法研究[J]. 煤矿开采, 2016, 21(02): 109-113.
- [14] 郭帆, 鲁远祥, 苟怡. 煤矿井下三维巷道建模技术研究[J]. 中州煤炭, 2015, (10): 71-75.
- [15] Nan Li, Xianglong Song, Cangbai Li. 3D Geological Modeling for Mineral System Approach to GIS-Based Prospectivity Analysis: Case Study of an MVT Pb–Zn Deposit[J]. Natural Resources Research, 2019, 28(03): 995-1019.
- [16] Kaidong Zheng. 3D Reservoir Modeling Based on Mobile Platform and OpenGL ES[J]. Journal of Physics: Conference Series, 2019, 1237(05): 052016.
- [17] Lin, Zhou, Lv, Zhu. 3D geological modelling based on 2D geological map[J]. Anna ls of GIS, 2017, 23(02): 117-129.
- [18] 杨义辉. 基于 MultiPath 模型的矿井巷道三维建模方法的研究与实现[J]. 矿山测量, 2013, (01): 46-50+55+7.

- [19] 万金. 三维地下巷道的研究及其应用[D]. 武汉科技大学, 2012.
- [20] 张海明. 基于 AutoCAD 的巷道三维建模设计与实现[D]. 南京农业大学, 2014.
- [21] 刘乐, 樊娟. 矿井巷道数据提取及三维巷道可视化研究[J]. 无线互联科技, 2016, (09): 26-28.
- [22] 谭正华, 王李管, 熊书敏等. 基于实测边界线的地下巷道三维建模方法[J]. 中南大学学报(自然科学版), 2012, 43(02): 626-631.
- [23] 王红娟, 卢新明, 李金龙等. 基于布尔运算的巷道精准建模研究及应用[J]. 软件导刊, 2021, 20(01): 21-25.
- [24] 邹艳红, 褚慧慧, 毛先成. 基于面-体布尔运算的断层切割矿体三维模型[J]. 金属矿山, 2019(07): 153-160.
- [25] 吴佳明, 丁烈云, 肖明清等. 基于 BIM 技术的地质体与结构体模型深度融合方法[J]. 应用基础与工程科学学报, 2023, 31(06): 1410-1426.
- [26] 陈骞. 基于两级体素模型的 STL 布尔运算研究[硕士学位论文]. 华中科技大学, 2020
- [27] Frisken, Sf, Perry, Rn. Adaptively sampled distance fields: a general representation of shape for computer graphics[C]. ACM, 2000.
- [28] Museth K, Breen D E, Whitaker R T, et al. Level set surface editing operators[C]. International Conference on Computer Graphics and Interactive Techniques, 2002, 21(3): 330-338.
- [29] Kobbelt L, Botsch M, Schwannecke U, et al. Feature sensitive surface extraction from volume data[C]. International Conference on Computer Graphics and Interactive Techniques, 2001: 57-66.
- [30] Ju T, Losasso F, Schaefer S, et al. Dual contouring of hermite data[C]. International Conference on Computer Graphics and Interactive Techniques, 2002, 21(3): 339-346.
- [31] 李妞. 基于八叉树球形体素模型的动态局部更新虚拟加工仿真算法研究[D]. 青岛理工大学, 2013.
- [32] Varadhan G, Krishnan S, Sriram T, et al. Topology preserving surface extraction using adaptive subdivision[C]. Symposium on Geometry Processing, 2004: 235-244.
- [33] Varadhan G, Krishnan S, Kim Y J, et al. Feature-sensitive subdivision and isosurface reconstruction[C]. IEEE Visualization, 2003.
- [34] 郭强强. 面向大型网格模型的高效布尔运算和曲面分层算法研究[D]. 华中科技大学, 2022.
- [35] 何思维. 基于包围盒的三维图像包裹算法研究[D]. 西安工业大学, 2023.
- [36] 侯铂. 基于 BVH 光子树的均匀参与介质渲染方法研究[D]. 山东师范大学, 2

023.

- [37] 陈豪, 赵继, 徐秀玲, 等. 基于凸多面体碰撞检测的虚拟砂轮建模研究. 中国机械工程, 2022, 33(02): 127-133
- [38] 王磊. 基于混合型包围盒碰撞检测技术的研究及其在 Web3D 漫游中的应用[D]. 上海大学, 2015.
- [39] 金汉均, 王晓荣, 王萌. 基于层次包围盒的碰撞检测算法的存储优化[J]. 计算机工程与应用, 2007(16): 61-63.
- [40] 杨凯祥. STL 模型布尔运算研究[D]. 中北大学, 2017.
- [41] 杨兰. 三维网格模型实体布尔运算方法的研究与实现[D]. 中南大学, 2011.
- [42] 车德福, 张春丽, 杜海鹰. BSP 树与格网划分相结合的数字城市三维模型组织方法[J]. 矿山测量, 2019, 47(04): 81-84.
- [43] 刘凯正. 复杂开曲面算的鲁棒布尔运算[D]. 中国科学技术大学, 2021.
- [44] 季莘翔, 王宇钢, 林一鸣. 基于四叉树的 ORB 特征阶梯分布算法[J]. 机械工程师, 2024(03): 26-30+33.
- [45] 刘昊. 用于三维光场显示的复杂三维模型切割与优化方法研究[D]. 北京邮电大学, 2023.
- [46] 翟晨龙, 朱冬梅, 贺可太等. 基于外存八叉树 STL 模型的拓扑重建方法[J]. 机电产品开发与创新, 2024, 37(01): 115-119.
- [47] 李颖颖, 黄文培. 基于优化八叉树的场景视锥体裁剪算法[J]. 计算机与现代化, 2024(01): 103-108.
- [48] 张雷. 建筑设计中曲面网格的划分方式及优化策略[D]. 西安建筑科技大学, 2023.
- [49] Thibault W C, Naylor B F. Set operations on polyhedra using binary space partitioning trees[C]. International Conference on Computer Graphics and Interactive Techniques, 1987, 21(4): 153-162.
- [50] Naylor B F, Amanatides J, Thibault W C, et al. Merging BSP trees yields polyhedral set operations[C]. International Conference on Computer Graphics and Interactive Techniques, 1990, 24(4): 115-124.
- [51] Mei G, Tipper J C. Simple and Robust Boolean Operations for Triangulated Surfaces[J]. arXiv: Computational Geometry, 2013.
- [52] 林宇鹏. 三维网格模型的布尔运算算法研究[D]. 中国科学技术大学, 2017.
- [53] 李亚琪. 多视图目标视角估计与三维重建方法研究[D]. 武汉理工大学, 2023.
- [54] Campen M, Kobbelt L. Exact and Robust (Self-)Intersections for Polygonal Meshes[J]. Computer Graphics Forum, 2010, 29(2): 397-406.
- [55] 熊心一, 姚宇. 基于混合八叉树的动态对象碰撞检测算法[J]. 计算机应用,

- 2019, 39(S1): 96-99.
- [56] Feito F R, Ogayar C J, Segura R J, et al. Fast and accurate evaluation of regularized Boolean operations on triangulated solids[J]. *Computer-aided Design*, 2013, 45(3): 705-716.
- [57] 姜旭东. 三角网格模型的规则化布尔计算优化研究[D]. 上海交通大学, 2015.
- [58] Franklin W R. Mathematical, algorithmic and data structure issues: adaptive grids for geometric operations[J]. *Cartographica the International Journal for Geographic Information & Geovisualization*, 1984, 21(2-3):160-167.
- [59] Franklin W R, Chandrasekhar N, Kankanhalli M, et al. Efficiency of uniform grids for intersection detection on serial and parallel machines[C]. In: Magnenat-Thalmann Nadia, Thalmann D, editors. *New trends in computer graphics (Proc. computer graphics international'88)*. Springer-Verlag; 1988:288-97.
- [60] Magalhaes S V G, Andrade M V A, Franklin W R, et al. PinMesh-Fast and exact 3 D point location queries using a uniform grid[J]. *Computers & Graphics*, 2016, 58: 1-11.
- [61] Magalhaes S V G, Franklin W R, Andrade M V A. Exact fast parallel intersection of large 3-D triangular meshes[C]. *International Meshing Roundtable*. Springer, Cham, 2018.
- [62] Magalhes S V G, Franklin W R, Andrade M V A. An efficient and exact parallel algorithm for intersecting large 3-D triangular meshes using arithmetic filters[J]. *Computer Aided Design*, 2019, 120:102801.
- [63] T. Möller. A fast triangle-triangle intersection test. *Journal of graphics tools*, 1997, 2(2): 25-30
- [64] M. Held. Erit—a collection of efficient and reliable intersection tests. *Journal of Graphics Tools*, 1997, 2(4): 25-44
- [65] P. Guigue, O. Devillers. Fast and robust triangle-triangle overlap test using orientation predicates. *Journal of graphics tools*, 2003, 8(1): 25-32
- [66] Jiang X ,Peng Q ,Cheng X , et al.Efficient Booleans algorithms for triangulated meshes of geometric modeling[J].*Computer-Aided Design and Applications*,2016,13 (4):419-430.
- [67] M. Livesu, G. Cherchi, R. Scateni, M. Attene. Deterministic Linear Time Constrained Triangulation using Simplified Earcut. *IEEE Transactions on Visualization and Computer Graphics*, 2021
- [68] Moller T, Trumbore B. Fast, Minimum storage ray/triangle intersection [J]. *Journal of Graphics Tools*, 1997, 2(1):21-28.
- [69] 初剑, 魏志强, 孟祥宾等. 基于 Delaunay 三角剖分的曲面求交技术[J]. *系统仿真学报*, 2009, 21(S1): 155-158.
- [70] 王超, 王立勇, 苏清华等. 一种改进 Delaunay 三角剖分的临时道路检测方法

- [J]. 重庆理工大学学报(自然科学), 2023, 37(06): 85-92.
- [71] 王子叶. 基于深度学习的断层识别与曲面重建研究[D]. 河北地质大学, 2024.
- [72] 李学勇. 面向角点网格地质模型的三维可视化方法研究[D]. 西安石油大学, 2023.
- [73] 朱书良. VR/AR 场景下的环境感知系统应用研究[D]. 南京邮电大学, 2023.
- [74] 陈攀, 杨涵, 徐雨等. 高应力软岩巷道全断面支护方案设计及优选[J]. 煤炭技术, 2023, 42(12): 50-55.
- [75] 李季, 张荣光, 余洋等. 采动应力偏转诱发回采巷道顶板断层活化风险评估[J]. 岩石力学与工程学报, 2023, 42(S2): 4109-4120.
- [76] 周涛, 张广宁, 周广飞. 基于冲击风险量化评估模型的采掘方案优化案例研究[J]. 煤炭工程, 2023, 55(05): 8-13.
- [77] 许慧. 基于 BIM 技术的市政工程项目全过程造价管理研究[D]. 河北地质大学, 2024.
- [78] 于晓淼. 水利工程边坡支护施工技术管理方法分析[J]. 水上安全, 2023(15): 46-48.
- [79] XIA J, ZHANG Y, SHE D, et al. Urban water system theory and its model development and application[J]. Science China Earth Sciences, 2024, 67(03): 704-724.
- [80] Wenwei Z, Shaodong C, Shaobin H. Development and Prospects of Large-Scale Underground Oil(Gas)Storage Technology in China[J]. China Oil & Gas, 2023, 30(06): 9-16.
- [81] International Journal of Minerals, Metallurgy and Materials CONTENTS[J]. International Journal of Minerals, Metallurgy and Materials, 2023, 30(12): 2469-2480.
- [82] Xue G, Yilmaz E, Wang Y. Progress and prospects of mining with backfill in metal mines in China[J]. International Journal of Minerals, Metallurgy and Materials, 2023, 30(08): 1455-1473.

致 谢

时间犹如白驹过隙，研究生三年转瞬即逝，回想这短暂的三年，心里有万千感触，这三年，对我来说是一次成长，也是一次蜕变，让我学习了很多专业的知识以及为人处世的原则。对此，我想表达我内心的感谢。

首先，我衷心地感谢我的导师，武强院士。是您引领我步入科研的殿堂，让我得以深入学习专业知识。每当我遇到困惑，您总是耐心解答，使我受益匪浅。能够成为您的学生，我深感荣幸。您对学生的责任感、对工作的严谨态度以及对科研的不懈追求，都深深地影响着我，成为我人生的楷模和奋斗的目标。

感谢徐华老师、赵颖旺老师以及杜沅泽老师。在项目的推进过程中，您们给予了我悉心的指导；在生活的点滴中，您们给予了我无私的支持与帮助。您们的引领使我不断进步，不断拓宽视野，教会我以客观、严谨的态度面对问题。在面对学习和生活的压力与困扰时，您们总是给予我最大的理解与鼓励，使我得以成长。特别是您们对待项目和科研的认真与负责，对我产生了深远的影响。

感谢曾一凡老师以及刘守强老师在学业上对我的悉心指导；感谢杜鑫老师对我生活上的照顾；感谢师兄张帅、王潇、刘源、孙千军以及师姐张小燕、赵琳、贾明琿、李雨宸对我在学业以及生活上的帮助；感谢同实验室的孟金洪、刘东、陈飞、管国萌、张瑶、包函、张智豪、于超、刘泽洋、魏华铭、崔雅帅、韦仙宇、王舒杨、杨义娜、赵菱尔，我们一起度过了研究生充实快乐的三年，这将是我以后难忘的回忆；感谢师弟柯峻伟、蒲耿萌、程子豪、查安中以及师妹陈梦在工作方面的支持与帮助。感谢这三年你们的帮助，使我受益匪浅。

感谢我的室友王宇乾、刘东、孟金洪、张浩然、乔嘉熙，我们一起共同努力，互帮互助，但愿未来你们能一帆风顺，前途似锦。

最后我要感谢的是我的家人田喜林、杜焕娇以及程晨，你们是最坚强的后盾，感谢你们对我的疼爱与照顾，我会带着这份爱与责任一路走下去。

最后祝大家在以后的道路中“所行皆是平坦，所想皆能如愿”。

作者简介

田华威，男（1998.11.01—），2021年毕业于华北水利水电大学，获学士学位；2021年9月-2024年6月在中国矿业大学（北京）攻读硕士专业学位，攻读领域为地质工程。

在学期间参与科研项目

1. 国家自然科学基金资助项目“矿井突（透）水灾情演变机制与智能研判方法研究”主要研究人员。项目编号：42202274。2023年1月-2025年12月。
2. 横向课题“文家坡煤矿水害防控与智能应急管理平台研发与应用”研究人员，2022年5月-2023年12月。